# MANAJEMEN PERANGKAT LUNAK STANDAR ISO27001/2022 *SECURE CODING* UNTUK PENINGKATAN DAN KONVERSI VERSI WEB APP DENGAN MEMANFAATKAN AI DAN PERANGKAT *OPENSOURCE*.

SKRIPSI




***THE SUSTAINABLE DEVELOPMENT GOALS***
***Industry, Innovation and Infrastructure***
***Affordable and Clean Energy***
***Climate Action***

**Disusun oleh:**

**Muhammad Farrel Akbar**
**22/492806/TK/53947**

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI**
**DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI**
**FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA**
**YOGYAKARTA**
**3 Juni 2026**

**HALAMAN PENGESAHAN**

**MANAJEMEN PERANGKAT LUNAK STANDAR ISO27001/2022 *SECURE CODING* UNTUK PENINGKATAN DAN KONVERSI VERSI WEB APP DENGAN MEMANFAATKAN AI DAN PERANGKAT *OPENSOURCE***

**SKRIPSI**

Diajukan sebagai Salah Satu Syarat untuk Memperoleh

Gelar Sarjana Teknik

pada Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik

Universitas Gadjah Mada

**Disusun oleh :**

**Muhammad Farrel Akbar**
**22/492806/TK/53947**

Telah disetujui dan disahkan

pada tanggal, 12 Juni 2026

| Dosen Pembimbing I | Dosen Pembimbing II |
|---|---|
| Dani Adhipta, S.Si., M.T. | Ir. Addin Suwastono, S.T., M.Eng., IPM. |
| NIP. 196907262000121001 | NIP. 197111171998031003 |

**SKRIPSI**

# MANAJEMEN PERANGKAT LUNAK STANDAR ISO27001/2022 *SECURE CODING* UNTUK PENINGKATAN DAN KONVERSI VERSI WEB APP DENGAN MEMANFAATKAN AI DAN PERANGKAT *OPENSOURCE*

Dipersiapkan dan disusun oleh

**Muhammad Farrel Akbar**
**22/492806/TK/53947**

Telah dipertahankan di depan dewan penguji
pada tanggal : **12 Juni 2026**

**Susunan Dewan Penguji**

| Pembimbing Utama | Pembimbing Pendamping |
|---|---|
| **Dani Adhipta, S.Si., M.T.** | **Ir. Addin Suwastono, S.T., M.Eng., IPM.** |

Anggota Dewan Penguji Lain

| | |
|---|---|
| **Dr. Ir. Rudy Hartanto, M.T., IPM.** | **Syukron Abu Ishaq Alfarozi, S.T., Ph.D.** |

Skripsi ini telah diterima sebagai salah satu persyaratan
untuk memperoleh gelar Sarjana Teknik
Tanggal: 17 Juli 2026
Pengelola Program Studi: Sarjana Teknologi Informasi

**Dr. Ir. Ahmad Nasikun, S.T., M.Sc., IPM., ASEAN Eng.**
NIP 198801082024061001

Mengetahui,
Ketua Departemen Teknik Elektro dan Teknologi Informasi

**Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE.**
NIP 197802242002121001

# PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

| | |
|---|---|
| Nama | : Muhammad Farrel Akbar |
| NIM | : 22/492806/TK/53947 |
| Tahun terdaftar | 2022 |
| Program | : Sarjana |
| Program Studi | : Teknologi Informasi |
| Fakultas | : Teknik Universitas Gadjah Mada |

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 3 Juni 2026




Muhammad Farrel Akbar
22/492806/TK/53947

# HALAMAN PERSEMBAHAN

Tugas akhir ini saya persembahkan kepada kedua orang tua saya yang telah mendukung dan mendoakan saya hingga titik ini. Saya persembahkan pula kepada keluarga dan teman-teman semua, serta untuk bangsa, negara, dan agamaku.

# KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE. selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Dr. Ahmad Nasikun, S.T., M.Sc. selaku Ketua Program Studi Teknologi Informasi.
3. Dani Adhipta, S.Si., M.T. dan Addin Suwastono, Ir., S.T., M.Eng., IPM. selaku dosen pembimbing yang telah membimbing dan memberikan arahan serta ilmu yang sangat berharga selama penyusunan skripsi ini.
4. Muhammad Aidiel Rachman Putra, S.Kom., M.Kom. dan Dr. Warsun Najib, S.T., M.Sc. selaku dosen DTETI FT UGM yang telah bersedia meluangkan waktu sebagai *rater* independen dalam eksperimen validasi efektivitas *AI Engine* pada penelitian ini.
5. Kedua Orang tua penulis, Bapak Andrian Nugroho dan Ibu Retny Anggeriana yang telah memberikan dukungan serta doa yang tiada henti sehingga penulis dapat menyelesaikan studi ini.
6. Para Dosen Program Studi S1 Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada yang telah memberikan bekal ilmu kepada penulis.
7. Teman-teman tim capstone *smart grid* A05, Sakti Cahya Buana, Stasya Adelia, Christy Clarrimond Kewas, serta Ahmad Maydanul Ilmi yang selama satu tahun terakhir telah mendampingi penulis dalam menyelesaikan tugas akhir di departemen bersama. Penulis senantiasa mendoakan kesuksesan dan kebahagiaan bagi seluruh pihak yang telah disebutkan.
8. Mas Ridho Agusta Anugrah, Mas Wahyu Rinastomo, serta seluruh teman-teman KKN di Desa Rejoagung yang telah mengubah penulis memberi motivasi skripsi dan pribadi yang lebih baik dan memberikan motivasi untuk menyelesaikan skripsi ini.

Penulis menyadari bahwa dalam penyusunan skripsi ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi kesempurnaan skripsi ini. Semoga skripsi ini dapat bermanfaat bagi pembaca pada umumnya dan bagi penulis pada khususnya. Amin.

Yogyakarta, 24 Mei 2026

Penulis

# DAFTAR ISI


HALAMAN PENGESAHAN ..... ii
PERNYATAAN BEBAS PLAGIASI ..... iii
HALAMAN PERSEMBAHAN ..... iv
KATA PENGANTAR ..... v
DAFTAR ISI ..... vi
DAFTAR TABEL ..... x
DAFTAR GAMBAR ..... xi
DAFTAR SINGKATAN ..... xii
INTISARI ..... xiii
ABSTRACT ..... xiv
BAB I Pendahuluan ..... 1
1.1 Latar Belakang ..... 1
1.2 Rumusan Masalah ..... 3
1.3 Tujuan Penelitian ..... 4
1.4 Batasan Penelitian ..... 4
1.5 Keamanan dan Privasi Data ..... 5
1.6 Manfaat Penelitian ..... 6
1.6.1 Manfaat Akademis ..... 6
1.6.2 Manfaat Praktis ..... 6
1.7 Sistematika Penulisan ..... 6
BAB II Tinjauan Pustaka dan Dasar Teori ..... 8
2.1 Tinjauan Literatur ..... 8
2.1.1 Perlindungan Otomatis Aplikasi PHP dari Injeksi SQL ..... 8
2.1.2 Menggunakan LLM untuk Menemukan dan Memantau Kerentanan 8
2.1.3 Transformasi Kode dengan Bantuan LLM ..... 9
2.1.4 Pendekatan Neuro-Simbolik untuk Deteksi Kerentanan Keamanan 9
2.1.5 Dampak Jangka Panjang Transformasi Kode Otomatis ..... 10
2.1.6 Transformasi Kode Berbasis Pola untuk Migrasi Aplikasi ..... 10
2.1.7 Migrasi Kode Berbasis AST di Lingkungan Produksi ..... 11
2.1.8 Ringkasan Perbandingan ..... 11
2.1.9 Pertanyaan Penelitian ..... 14
2.2 Dasar Teori ..... 14
2.2.1 PHP dan Perubahan Antar Versi ..... 14
2.2.2 ISO/IEC 27001:2022 dan *Secure Coding* ..... 15
2.2.3 OWASP Top 10 dan Kerentanan Aplikasi Web PHP ..... 16
2.2.4 DevSecOps dan Integrasi Keamanan dalam *Pipeline* Pengembangan 16

2.2.5 Metodologi *Research and Development* dengan Model Prototipe .. 17
2.2.6 *Abstract Syntax Trees* dan *Static Analysis* ..... 17
2.2.7 Rector: Alat Transformasi Kode Berbasis AST ..... 17
2.2.8 PHPStan: Alat Analisis Statis untuk PHP ..... 18
2.2.9 Semgrep: Alat Analisis Statis Ringan untuk Keamanan ..... 18
2.2.10 Model-Model LLM untuk Analisis Kode ..... 18
2.2.11 Metrik Evaluasi Kualitas Keluaran LLM ..... 19
2.2.12 Ollama: Menerapkan LLM Secara Lokal ..... 21
2.2.13 Algoritma Ratcliff/Obershelp untuk Pengukuran *Similarity* Kode . 21
2.2.14 Evaluasi Berbasis Penilaian Manusia ..... 22
BAB III Metodologi Penelitian ..... 23
3.1 Alat dan Bahan Penelitian ..... 23
3.1.1 Perangkat Keras ..... 23
3.1.2 Perangkat Lunak ..... 23
3.1.3 Model LLM yang Dievaluasi ..... 24
3.1.4 Bahan Penelitian ..... 24
3.2 Metode Penelitian ..... 25
3.3 Arsitektur Sistem ..... 26
3.3.1 Scanner (scanner.py) ..... 27
3.3.2 Converter (converter.py) ..... 28
3.3.3 Analyzer (analyzer.py) ..... 29
3.3.4 AI Engine (ai_engine.py) ..... 29
3.3.5 ISO Mapper (iso_mapper.py) ..... 29
3.3.6 Main (main.py) ..... 30
3.3.7 Composer Analyzer (composer_analyzer.py) ..... 31
3.4 Alur Data Pipeline ..... 31
3.5 Desain Prompt LLM ..... 32
3.5.1 Struktur Prompt ..... 32
3.5.2 Keputusan Desain Prompt ..... 35
3.6 Desain Pemetaan ISO/IEC 27001:2022 ..... 36
3.6.1 Mekanisme Klasifikasi Temuan ..... 37
3.7 Rancangan Eksperimen Komparasi Model LLM ..... 38
3.7.1 Baseline Pembanding ..... 38
3.7.2 Kondisi Eksperimen ..... 39
3.7.3 Metrik Evaluasi ..... 39
3.7.4 Interpretasi *Trade-off* Antar Metrik ..... 40
3.8 Rancangan Eksperimen Validasi Efektivitas *AI Engine* ..... 40
3.8.1 Definisi Kondisi B0 dan T1 ..... 40
3.8.2 Dataset ..... 40

3.8.3 Desain *Blind Rating* ..... 40
3.8.4 Instrumen Penilaian ..... 41
3.8.5 Metode Analisis Statistik ..... 41
3.9 Metode Pengujian dan Evaluasi ..... 42
BAB IV Hasil dan Diskusi ..... 43
4.1 Studi Kasus 1: FT UGM (Dashboard TCK) ..... 43
4.1.1 Deskripsi Dataset ..... 43
4.1.2 Hasil Konversi Rector ..... 43
4.1.2.1 Contoh Transformasi Rector ..... 44
4.1.2.2 File yang Gagal Dikonversi ..... 46
4.1.3 Hasil Pemindaian Keamanan Semgrep ..... 47
4.1.3.1 Pre-Scan: Baseline Keamanan Kode Asli ..... 47
4.1.3.2 Post-Scan: Perbandingan Setelah Konversi ..... 47
4.1.4 Hasil Validasi PHPStan ..... 48
4.1.5 Status Kepatuhan ISO/IEC 27001:2022 ..... 50
4.1.6 Hasil Komparasi Lima Model LLM (Kondisi A) ..... 51
4.1.6.1 Format Compliance Rate ..... 51
4.1.6.2 Syntax Validity Rate ..... 52
4.1.6.3 Statistik Ringkasan ..... 53
4.1.6.4 Trade-off dan Implikasinya ..... 55
4.1.7 Hasil Komparasi Lima Model LLM (Kondisi B) ..... 56
4.1.7.1 Format Compliance Rate ..... 56
4.1.7.2 Syntax Validity Rate ..... 57
4.1.7.3 Statistik Ringkasan ..... 58
4.1.8 Perbandingan Kondisi A dan Kondisi B ..... 59
4.1.9 Optimasi Parameter Qwen2.5-Coder (Kondisi C) ..... 60
4.1.9.1 Pengaruh Temperature ..... 60
4.1.9.2 Pengaruh Context Window ..... 61
4.1.9.3 Implikasi Kondisi C ..... 61
4.1.10 Analisis Dependensi Composer ..... 61
4.1.11 Validasi Keakuratan Hasil Konversi ..... 62
4.1.11.1 Metrik Pipeline-Level ..... 62
4.1.11.2 Validasi 24 file ..... 63
4.1.12 Diskusi dan Keterbatasan ..... 66
4.1.12.1 Keterbatasan Dataset ..... 66
4.1.12.2 Keterbatasan Kompatibilitas PHP 8.4 dan PHP 8.5 ..... 67
4.1.12.3 Keterbatasan Teknis Pipeline ..... 67
4.1.12.4 Interpretasi Confidence Score ..... 68
4.2 Studi Kasus 2: Sistem CBT DTI UGM ..... 68

4.2.1 Deskripsi Dataset ..... 68
4.2.2 Hasil Konversi Rector ..... 69
4.2.2.1 File yang Gagal Dikonversi ..... 69
4.2.3 Hasil Pemindaian Keamanan Semgrep ..... 70
4.2.3.1 Pre-Scan: Baseline Keamanan Kode Asli ..... 70
4.2.3.2 *Post-Scan* dan *AI Engine* ..... 71
4.2.4 Hasil Validasi PHPStan ..... 72
4.2.5 Status Kepatuhan ISO/IEC 27001:2022 ..... 73
4.2.6 Analisis Dependensi Composer ..... 73
4.3 Perbandingan Kedua Studi Kasus ..... 74
4.3.1 Perbandingan Temuan Keamanan Semgrep ..... 74
4.3.2 Perbandingan Tingkat Konversi Rector ..... 74
4.3.3 Perbandingan Temuan PHPStan ..... 75
4.3.4 Ringkasan Perbandingan dan Analisis Dependensi Composer ..... 76
4.3.5 Hasil Validasi Efektivitas *AI Engine* ..... 77
4.3.5.1 Desain Eksperimen ..... 77
4.3.5.2 Skor Rata-rata per Temuan ..... 78
4.3.5.3 Perbandingan per Dimensi ..... 78
4.3.5.4 Analisis Statistik ..... 79
4.3.5.5 Diskusi ..... 79
4.3.5.6 Limitasi ..... 80
BAB V Kesimpulan dan Saran ..... 81
5.1 Kesimpulan ..... 81
5.2 Saran ..... 82
DAFTAR PUSTAKA ..... 84
LAMPIRAN ..... L-1
Lampiran A — Repositori GitHub ..... L-1
Lampiran B — Pseudocode Pipeline Migrasi PHP ..... L-3
Lampiran C — Teks Prompt Kondisi B ..... L-22
Lampiran D — Instrumen Penilaian *Blind Rating* ..... L-26

# DAFTAR TABEL


Tabel 2.1 Perbandingan Penelitian Terdahulu dan Karakteristik Penelitian ..... 12
Tabel 2.2 Perbandingan Penelitian Terdahulu dan Aspek Metodologi..... 12

Tabel 3.1 Pemetaan argumen `-target-php` ke konfigurasi Rector ..... 28
Tabel 3.2 Pemetaan sumber temuan ke kontrol ISO/IEC 27001:2022 Annex A 37
Tabel 3.3 Pemetaan kata kunci `rule_id` Semgrep ke jenis kerentanan dan kontrol ISO/IEC 27001:2022 ..... 38

Tabel 4.1 Status kepatuhan ISO/IEC 27001:2022 hasil *pipeline* pada dataset FT UGM ..... 50
Tabel 4.2 Statistik lengkap komparasi kelima model LLM pada Kondisi A .... 53
Tabel 4.3 Statistik lengkap komparasi kelima model LLM pada Kondisi B..... 58
Tabel 4.4 Perbandingan metrik utama Kondisi A vs Kondisi B untuk kelima model LLM ..... 59
Tabel 4.5 Status kompatibilitas PHP 8.x dependensi *composer* FT UGM..... 62
Tabel 4.6 Status kepatuhan ISO/IEC 27001:2022 hasil *pipeline* pada dataset CBT DTI UGM..... 73
Tabel 4.7 Ringkasan perbandingan metrik *pipeline* dan status dependensi *composer* kedua studi kasus ..... 76
Tabel 4.8 Skor rata-rata B0 dan T1 per temuan per rater (skala Likert 7, rata-rata 4 dimensi)..... 78
Tabel 4.9 Rata-rata skor B0 dan T1 per dimensi penilaian (gabungan 2 rater, $n = 11$ temuan)..... 79

# DAFTAR GAMBAR


Gambar 3.1 Alur penelitian dengan model prototipe ..... 25
Gambar 3.2 Arsitektur sistem *pipeline* dengan enam modul fungsional dan satu orkestrator ..... 27
Gambar 3.3 Alur data *pipeline*: tahap masukan: Scanner, Converter, dan Composer Analyzer ..... 31
Gambar 4.4 Ringkasan hasil konversi Rector terhadap 245 *file* PHP dataset FT UGM ..... 44
Gambar 4.5 Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A ..... 49
Gambar 4.6 *Format compliance rate* kelima model LLM pada Kondisi A (*zero-shot*, *prompt* identik) ..... 52
Gambar 4.7 *Syntax validity rate* dan *code block extraction rate* kelima model LLM pada Kondisi A ..... 52
Gambar 4.8 *Format compliance rate* kelima model LLM pada Kondisi B (*prompt* dioptimasi per model) ..... 57
Gambar 4.9 *Syntax validity rate* dan *code block extraction rate* kelima model LLM pada Kondisi B ..... 57
Gambar 4.10 Perbandingan metrik Qwen2.5-Coder pada variasi *temperature* dan *context window* (Kondisi C) ..... 60
Gambar 4.11 Akurasi konversi Rector pada dataset FT UGM ..... 62
Gambar 4.12 Durasi eksekusi *pipeline* pada dataset FT UGM ..... 63
Gambar 4.13 Distribusi cakupan *file* sampel validasi 24 file berdasarkan direktori asal ..... 64
Gambar 4.14 Perbandingan metrik validasi antara keluaran Rector dan referensi LLM pada 12 *file* sampel dari `application/` ..... 64
Gambar 4.15 Distribusi *similarity score* antara keluaran Rector dan referensi LLM pada 12 *file* sampel (*line-level SequenceMatcher*) ..... 65
Gambar 4.16 Ringkasan hasil konversi Rector terhadap 326 *file* PHP dataset CBT DTI UGM ..... 69
Gambar 4.17 Temuan Semgrep *pre-scan* vs *post-scan* berdasarkan tingkat keparahan pada dataset CBT DTI UGM ..... 70
Gambar 4.18 Temuan Semgrep *pre-scan* vs *post-scan* berdasarkan jenis kerentanan pada dataset CBT DTI UGM ..... 71
Gambar 4.19 Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A pada dataset CBT DTI UGM ..... 72
Gambar 4.20 Perbandingan temuan Semgrep *pre-scan* berdasarkan jenis kerentanan pada kedua studi kasus ..... 74
Gambar 4.21 Perbandingan hasil konversi Rector pada kedua studi kasus ..... 75
Gambar 4.22 Perbandingan *error* PHPStan berdasarkan kontrol ISO/IEC 27001:2022 pada kedua studi kasus ..... 75

# DAFTAR SINGKATAN

| | | |
|---|---|---|
| $CR$ | = | *Conversion Rate* (tingkat konversi) |
| $FCR$ | = | *Format Compliance Rate* |
| $SVR$ | = | *Syntax Validity Rate* |
| $\Delta F$ | = | *Security Finding Delta* (selisih temuan keamanan) |
| $\bar{C}_m$ | = | *Mean Confidence Score* model $m$ |
| $\sigma_m^2$ | = | variansi *confidence score* model $m$ |
| AI | = | *Artificial Intelligence* |
| API | = | *Application Programming Interface* |
| AST | = | *Abstract Syntax Tree* |
| AWS | = | *Amazon Web Services* |
| CBT | = | *Computer-Based Test* |
| CI/CD | = | *Continuous Integration/Continuous Delivery* |
| CLI | = | *Command Line Interface* |
| CVE | = | *Common Vulnerabilities and Exposures* |
| CWE | = | *Common Weakness Enumeration* |
| GGUF | = | *GGML Unified Format* |
| GPU | = | *Graphics Processing Unit* |
| IDE | = | *Integrated Development Environment* |
| IEC | = | *International Electrotechnical Commission* |
| ISMS | = | *Information Security Management System* |
| ISO | = | *International Organization for Standardization* |
| JIT | = | *Just-In-Time* |
| JSON | = | *JavaScript Object Notation* |
| LLM | = | *Large Language Model* |
| LoRA | = | *Low-Rank Adaptation* |
| MVC | = | *Model-View-Controller* |
| NIST | = | *National Institute of Standards and Technology* |
| NVD | = | *National Vulnerability Database* |
| OWASP | = | *Open Web Application Security Project* |
| PDO | = | *PHP Data Objects* |

# INTISARI

Sebagian besar situs web masih menjalankan aplikasi PHP pada versi 7.x atau lebih lama yang sudah tidak mendapatkan pembaruan keamanan resmi, sehingga sistem menghadapi akumulasi kerentanan yang terus bertambah tanpa kemungkinan diperbaiki. Migrasi manual pada ratusan *file* kode tidak realistis, dan alat transformasi yang tersedia saat ini tidak menyertakan pemeriksaan keamanan sebagai bagian terintegrasi dari alur migrasi. Organisasi yang beroperasi di bawah standar ISO/IEC 27001:2022 juga harus melakukan audit keamanan sebagai langkah terpisah setelah migrasi selesai, sehingga menambah beban operasional.

Penelitian ini membangun *platform* berbasis Python yang mengintegrasikan konversi otomatis aplikasi web PHP lama ke versi PHP target yang dapat dikonfigurasi, dipadukan dengan pemeriksaan kepatuhan *secure coding* berbasis ISO/IEC 27001:2022 dalam satu alur kerja *open source* yang dapat dijalankan secara lokal. *Platform* ini menggunakan Rector untuk transformasi kode berbasis AST, Semgrep untuk pemindaian keamanan, PHPStan untuk validasi pasca-konversi, serta lima model LLM *open source* yang dijalankan secara lokal melalui Ollama—DeepSeek Coder 6.7b, Qwen2.5-Coder 7b, CodeLlama 7b, Mistral 7b, dan Llama 3.1 8b—untuk analisis semantik dan saran perbaikan kerentanan. Studi kasus dilakukan pada kode PHP milik FT UGM (Dashboard TCK, 245 *file*) dan CBT DTI UGM (326 *file*).

Rector mencatat tingkat konversi 97,6% pada dataset FT UGM dan 91,7% pada dataset CBT DTI UGM, dengan tingkat validitas sintaks pasca-konversi 100% pada kedua studi kasus. Nilai $\Delta F = 0$ mengonfirmasi bahwa proses konversi tidak memperkenalkan kerentanan baru. Laporan JSON yang dihasilkan secara otomatis mencakup pemetaan per temuan ke kontrol ISO/IEC 27001:2022 Annex A yang relevan dan dapat langsung digunakan sebagai dokumentasi audit. Dari eksperimen komparasi lima model LLM, Qwen2.5-Coder 7b direkomendasikan untuk integrasi ke dalam sistem otomatis karena menghasilkan *Syntax Validity Rate* (SVR) 100% secara konsisten di semua kondisi eksperimen. *Platform* ini membuktikan bahwa integrasi antara konversi kode otomatis dan pemeriksaan kepatuhan ISO/IEC 27001:2022 dapat dijalankan sepenuhnya secara lokal tanpa mengirimkan kode sumber ke layanan eksternal.

Kata kunci : migrasi PHP, *secure coding*, ISO/IEC 27001:2022, *large language model*, transformasi kode AST

# ABSTRACT

*The majority of websites continue to run PHP web applications on end-of-life versions that no longer receive official security updates, leaving known vulnerabilities permanently unpatched. Manual migration of hundreds of source files is impractical, and existing transformation tools do not incorporate security compliance checks as an integrated part of the migration workflow. Organizations operating under ISO/IEC 27001:2022 are further burdened by having to conduct security audits as a separate step after migration is complete.*

*This research develops a Python-based platform that integrates automated conversion of legacy PHP web applications to a configurable target PHP version with ISO/IEC 27001:2022-aligned secure coding compliance checks in a single, fully local, open source workflow. The platform combines Rector for AST-based code transformation, Semgrep for security scanning, PHPStan for post-conversion type validation, and five locally executed open source large language models via Ollama—DeepSeek Coder 6.7b, Qwen2.5-Coder 7b, CodeLlama 7b, Mistral 7b, and Llama 3.1 8b—for semantic vulnerability analysis and remediation suggestions. Case studies were conducted on PHP codebases from FT UGM (Dashboard TCK, 245 files) and CBT DTI UGM (326 files).*

*Rector achieved a conversion rate of 97.6% on the FT UGM dataset and 91.7% on the CBT DTI UGM dataset, with 100% post-conversion syntax validity in both cases. A security finding delta of zero ($\Delta F = 0$) confirmed that the conversion process introduced no new security vulnerabilities. Automatically generated JSON reports include per-finding mappings to relevant ISO/IEC 27001:2022 Annex A controls and can be used directly as audit documentation. Among the five evaluated LLMs, Qwen2.5-Coder 7b is recommended for automated pipeline integration, consistently achieving 100% Syntax Validity Rate across all experimental conditions. The platform demonstrates that automated PHP version migration combined with ISO/IEC 27001:2022 compliance reporting can be executed entirely on-premises without transmitting source code to any external service.*

**Keywords** : PHP migration, secure coding, ISO/IEC 27001:2022, large language model, AST code transformation

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Penelitian ini dilatarbelakangi oleh isu kerentanan keamanan yang melekat pada aplikasi web PHP yang masih berjalan di versi lawas (*end-of-life*). PHP tetap menjadi bahasa sisi *server* paling dominan, digunakan oleh 71,7% situs web yang bahasa sisi *server*-nya diketahui [1], namun hanya 58,1% di antaranya yang telah beralih ke PHP 8.x; sekitar 41,8% sisanya masih berjalan di PHP 7.x (33,1%) atau PHP 5.x (8,7%) [1]. Dukungan resmi untuk PHP 7.4, versi terakhir seri 7.x, telah berakhir pada 28 November 2022 [2], yang berarti pembaruan keamanan resmi telah dihentikan. NVD yang dikelola NIST mencatat bahwa sistem pada fase *end-of-life* secara efektif membiarkan celah keamanan terekspos tanpa kemungkinan perbaikan dari pihak pengembang bahasa pemrograman [3], dan akumulasi risiko ini membesar seiring lamanya sistem dibiarkan tidak diperbarui.

Permasalahan serupa secara nyata terjadi pada sistem di lingkungan FT UGM. Institusi ini mengelola aplikasi *Dashboard* TCK yang dibangun di atas *framework* CodeIgniter 3.x dengan PHP 7.4.9, terdiri dari 245 *file* PHP yang sebagian besar berisi logika bisnis spesifik institusi. Pemindaian awal dengan Semgrep [4] terhadap basis kode ini menemukan 11 temuan kerentanan aktif, terdiri dari 1 kasus injeksi SQL dan 10 kasus *Server-Side Request Forgery* (SSRF) yang terkonsentrasi pada satu *file* pemrosesan *output*, sementara PHPStan [5] mengidentifikasi 2.790 masalah tipe dan kompatibilitas yang berpotensi memicu kegagalan saat dijalankan di PHP versi baru. Status kepatuhan keseluruhan terhadap kontrol *secure coding* ISO/IEC 27001:2022 Annex A pada kondisi ini adalah `NON_COMPLIANT`, dengan empat dari enam kontrol yang dipetakan (A.8.25, A.8.26, A.8.28, A.8.29) berstatus tidak terpenuhi. Temuan ini menunjukkan dampak operasional yang nyata: 11 kerentanan keamanan tetap terbuka dan dapat dieksploitasi publik selama proses migrasi berlangsung, sementara 2.790 isu kompatibilitas tipe harus ditelusuri satu per satu sebelum kode dapat dinyatakan siap berjalan stabil di PHP 8.x – pekerjaan yang tidak realistis dilakukan secara manual oleh tim pengembang yang terbatas dalam batas waktu migrasi yang wajar.

Migrasi ke versi PHP yang masih didukung secara resmi bukan langkah yang sederhana untuk dilakukan secara manual. PHP 8.x membawa sejumlah *breaking changes* dari seri 7.x, termasuk penghapusan fungsi seperti `create_function()`, `each()`, dan seluruh ekstensi `ext/mysql`, serta sistem tipe yang lebih ketat dengan tipe *union* dan tipe campuran yang dapat memicu kegagalan *runtime* pada kode yang sebelumnya berjalan normal [2]. Penanganan kesalahan pun berubah: fungsi

yang sebelumnya mengembalikan `false` saat gagal kini melempar pengecualian. Menerapkan seluruh perubahan ini secara manual pada ratusan *file* tidak realistis, dan risiko regresi fungsional meningkat sejalan dengan ukuran basis kode yang dimigrasikan – inilah yang membuat sebagian organisasi memilih migrasi bertahap (PHP 7.4 dahulu, baru PHP 8.x) [2] sebagai mitigasi risiko, bukan sekadar preferensi teknis.

Alat otomatis yang tersedia saat ini membantu namun tidak saling terintegrasi, dan tidak satu pun yang mampu menjembatani celah antara deteksi kerentanan dan pembuktian kepatuhan yang dapat diaudit. Rector [6] terbukti andal untuk transformasi sintaksis berbasis AST pada skala industri [7], tetapi tidak dapat menangani perubahan yang membutuhkan pemahaman semantik, seperti penggantian `mysql_*()` dengan PDO/MySQLi. Semgrep [4] dan PHPStan [5] masing-masing dapat mendeteksi pola kerentanan dan masalah kompatibilitas tipe, namun keluarannya masih berbentuk simbolik – ID *rule*, nomor baris, dan potongan pola yang cocok – yang harus diterjemahkan secara manual oleh pengembang menjadi penjelasan risiko, langkah remediasi, dan pemetaan ke kontrol ISO/IEC 27001:2022 yang relevan. Akibatnya, bagi organisasi yang harus memenuhi ISO/IEC 27001:2022 Annex A.8, bukti kepatuhan *secure coding* tetap harus disusun lagi secara manual setelah migrasi selesai – sebagaimana ditunjukkan oleh status `NON_COMPLIANT` pada kode FT UGM di atas, di mana temuannya hanya tersaji sebagai data mentah Semgrep dan PHPStan, bukan penjelasan yang siap diaudit. Strobl dkk. [8] juga menemukan bahwa ketiadaan mekanisme validasi yang melekat pada proses transformasi menimbulkan risiko yang baru terasa setelah sistem berjalan di produksi, sehingga validasi pasca-konversi bukan langkah opsional melainkan kebutuhan inti desain.

Oleh karena itu, pendekatan berbasis kecerdasan buatan (AI) diusulkan untuk menjembatani keterbatasan tersebut. Pekerjaan penerjemahan temuan Semgrep menjadi penjelasan yang dapat diaudit – yang selama ini memakan waktu dan tidak konsisten antar pengembang – merupakan tugas yang relevan bagi LLM mengingat model ini tidak memerlukan eksekusi kode maupun akses ke basis data produksi, tetapi mampu memberikan pemahaman semantik tentang konteks kerentanan yang tidak dapat dihasilkan oleh alat berbasis aturan. *Large Language Model* (LLM) telah terbukti mampu mendeteksi sekaligus menjelaskan kerentanan perangkat lunak dalam bahasa natural [9] [10], dan Li, Dutta, dan Naik [11] menunjukkan bahwa menggabungkan LLM dengan alat analisis statis menghasilkan hasil yang lebih baik daripada masing-masing pendekatan yang berjalan sendiri – temuan yang menjadi dasar konseptual langsung bagi peran AI dalam penelitian ini. Dengan demikian, LLM diposisikan bukan untuk mengganti Semgrep sebagai detektor, melainkan untuk mengisi celah interpretasi antara temuan mentah dan tindakan remediasi yang dapat diaudit.

Kendala utama penggunaan LLM untuk tujuan ini adalah privasi data.

Model performa tinggi seperti GPT-4 hanya dapat diakses melalui API *cloud*, yang mengharuskan kode sumber dikirim ke *server* pihak ketiga – pilihan yang tidak dapat diterima untuk kode aplikasi internal FT UGM. Perkembangan model LLM *open source* berukuran kecil-menengah seperti DeepSeek Coder 6.7b [12], Qwen2.5-Coder 7b [13], CodeLlama 7b [14], Mistral 7b [15], dan Llama 3.1 8b [16] mengubah situasi ini: model-model ini dapat dijalankan secara lokal pada perangkat keras konsumen ber-VRAM 8 GB melalui Ollama [17], sehingga seluruh inferensi terjadi di mesin lokal tanpa data yang keluar dari lingkungan organisasi.

Penelitian ini membangun *platform* berbasis AI dan alat *open source* yang dapat secara otomatis mengubah aplikasi web PHP lawas ke versi PHP target yang dapat dikonfigurasi, dengan dukungan teknis dari PHP 5.2 hingga PHP 8.x, sekaligus menyertakan pemeriksaan *secure coding* berbasis ISO/IEC 27001:2022. *Platform* ini menggunakan Rector [6] untuk transformasi kode berbasis AST, Semgrep [4] untuk pemindaian keamanan, PHPStan [5] untuk validasi pasca-konversi, serta lima model LLM *open source* yang dijalankan secara lokal melalui Ollama [17] untuk menerjemahkan temuan Semgrep menjadi penjelasan dan saran remediasi yang siap diaudit. Studi kasus dilakukan pada kode PHP milik FT UGM dan CBT DTI UGM.

## 1.2 Rumusan Masalah

Berdasarkan latar belakang dan fakta-fakta yang telah diuraikan, akar permasalahan yang teridentifikasi adalah: (a) aplikasi PHP lawas milik FT UGM dan DTI UGM menyimpan kerentanan keamanan aktif (terkonfirmasi melalui pemindaian Semgrep dan PHPStan) yang tidak lagi mendapat pembaruan resmi; (b) migrasi manual ke versi PHP yang didukung memerlukan pemahaman semantik yang tidak dapat ditangani sepenuhnya oleh alat berbasis aturan seperti Rector, sehingga berisiko menimbulkan regresi fungsional bila dipaksakan otomatis tanpa validasi; (c) deteksi kerentanan, transformasi kode, dan pembuktian kepatuhan ISO/IEC 27001:2022 berjalan sebagai proses-proses terpisah, sehingga temuan keamanan tersaji sebagai data mentah yang masih harus diterjemahkan secara manual menjadi bukti audit; dan (d) belum diketahui model LLM *open source* lokal mana yang paling sesuai untuk tugas penerjemahan temuan keamanan kode PHP tersebut. Berdasarkan akar permasalahan tersebut, penelitian ini merumuskan empat permasalahan sebagai berikut:

1. Apakah *pipeline* yang dibangun berhasil memigrasikan aplikasi web PHP 7.x ke PHP 8.3 tanpa menimbulkan regresi fungsional pada kode sumber yang dikonversi?
2. Apakah hasil konversi mempertahankan fungsionalitas kode sumber asli sebagaimana dapat diverifikasi melalui pemeriksaan validitas sintaksis dan kompatibilitas tipe pasca-konversi?

3. Apakah deteksi kerentanan keamanan, transformasi kode, dan pembuktian kepatuhan terhadap kontrol ISO/IEC 27001:2022 Annex A.8 dapat disatukan dalam satu alur kerja otomatis sehingga temuan keamanan tidak lagi memerlukan penerjemahan manual menjadi bukti audit?
4. Apakah *AI Engine* berbasis LLM *open source* lokal yang diusulkan menghasilkan penjelasan dan saran remediasi yang secara terukur lebih informatif dibandingkan keluaran *pipeline* tanpa komponen AI?

## 1.3 Tujuan Penelitian

Penelitian ini memiliki empat tujuan yang menjawab rumusan masalah di atas:

1. Membangun *pipeline* berbasis AI menggunakan alat *open source* untuk mengonversi aplikasi web PHP 7.x ke PHP 8.3 secara otomatis.
2. Memvalidasi bahwa hasil konversi mempertahankan fungsionalitas kode sumber melalui pemeriksaan validitas sintaksis dan kompatibilitas tipe pasca-konversi.
3. Mengimplementasikan mekanisme pemeriksaan *secure coding* yang dipetakan ke kontrol ISO/IEC 27001:2022 Annex A.8 sebagai bagian dari alur konversi.
4. Mengevaluasi efektivitas *AI Engine* berbasis LLM *open source* lokal melalui perbandingan terukur antara keluaran *pipeline* dengan dan tanpa komponen AI, menggunakan eksperimen penilaian manusia secara buta (*blind rating*). Sebagai bagian dari evaluasi ini, performa lima kandidat model LLM dibandingkan untuk menentukan model yang paling sesuai menjalankan peran *AI Engine*.

## 1.4 Batasan Penelitian

Penelitian ini memiliki batasan-batasan sebagai berikut:

1. **Ruang lingkup konversi:** Secara teknis, *pipeline* yang dibangun mendukung konversi dari PHP 5.2 ke atas, karena Rector menggunakan rantai *ruleset* kumulatif yang mencakup semua perubahan dari PHP 5.2 hingga versi target yang dipilih [6]. Namun studi kasus dalam penelitian ini menggunakan kode sumber PHP 7.x milik FT UGM dan CBT DTI UGM sebagai representasi kondisi yang paling umum ditemukan saat ini [1]. Versi target yang dapat dikonfigurasi mencakup PHP 7.4 hingga PHP 8.3, yang merupakan rentang versi paling relevan untuk kebutuhan modernisasi institusi saat ini [2].
2. **Studi kasus:** Penelitian ini menggunakan dua studi kasus: (1) aplikasi *Dashboard* TCK (*Target Capaian Kerja*) milik Fakultas Teknik (FT) Universitas Gadjah Mada, dibangun di atas *framework* CodeIgniter 3 dengan versi PHP 7.4.9; dan (2) aplikasi *Computer-Based Test* (CBT) milik Direktorat Teknologi Informasi

(DTI) Universitas Gadjah Mada, dibangun di atas *framework* CodeIgniter 3 dengan persyaratan PHP $\geq$ 7.0. *Pipeline* bekerja pada level *file* PHP individual dan tidak bergantung pada *framework* yang digunakan, sehingga secara teknis dapat diterapkan pada aplikasi berbasis *framework* lain seperti Laravel atau Symfony maupun pada aplikasi PHP tanpa *framework*. Pengujian terhadap *framework* selain CodeIgniter 3 bukan bagian dari ruang lingkup penelitian ini.

3. **Kontrol keamanan:** Pemeriksaan keamanan hanya dilakukan pada kontrol ISO/IEC 27001:2022 Annex A yang relevan dengan *secure coding*: A.8.25, A.8.28, dan A.8.29, serta A.5.17, A.8.24, dan A.8.26 yang turut dipetakan oleh *pipeline* [18].
4. **Model AI:** Penelitian ini mengevaluasi dan membandingkan lima model LLM *open source* yang dijalankan secara lokal melalui Ollama [17] dalam eksperimen komparasi *zero-shot*: DeepSeek Coder 6.7b [12], Qwen2.5-Coder 7b [13], CodeLlama 7b [14], Mistral 7b [15], dan Llama 3.1 8b [16]. Semua model digunakan tanpa *fine-tuning*.
5. **Lingkungan validasi:** Pengujian dilakukan dalam pengaturan lokal menggunakan kode sumber dari lingkungan institusional. Pengujian dalam lingkungan produksi skala besar bukan bagian dari ruang lingkup penelitian ini.
6. **Bahasa pemrograman:** Penelitian ini terbatas pada aplikasi PHP. Bahasa lain tidak dicakup.
7. **Kompleksitas aplikasi:** Aplikasi uji memiliki kompleksitas tingkat menengah, mencakup ratusan *file* PHP dengan campuran logika bisnis, akses basis data, dan presentasi dalam satu basis kode *framework*.

## 1.5 Keamanan dan Privasi Data

Kode PHP dari FT UGM dan DTI UGM adalah aset institusi yang tidak boleh dikirimkan ke layanan eksternal. Batasan privasi data ini menjadi landasan dalam merancang arsitektur AI pada penelitian ini. Sebagai alternatif dari penggunaan API LLM berbasis *cloud*, penelitian ini menggunakan Ollama sebagai *runtime* lokal sehingga seluruh inferensi terjadi di mesin pengembangan.

Tidak ada potongan kode sumber aplikasi yang dikirimkan ke layanan inferensi eksternal atau API LLM berbasis cloud. Seluruh proses analisis berlangsung secara statis: tidak ada koneksi ke basis data produksi FT UGM, tidak ada eksekusi kode PHP. Satu-satunya koneksi eksternal yang terjadi adalah panggilan ke Packagist API oleh modul `composer_analyzer.py` untuk memeriksa metadata versi dependensi, bukan kode sumber aplikasi. Keputusan ini sejalan dengan ISO/IEC 27001:2022 yang mensyaratkan pemisahan lingkungan pengembangan dan pengendalian akses terhadap

aset informasi sensitif.

## 1.6 Manfaat Penelitian

### 1.6.1 Manfaat Akademis

Penelitian ini berkontribusi dalam mengintegrasikan kontrol *secure coding* ISO/IEC 27001:2022 ke dalam alur kerja transformasi kode otomatis [18], sebuah pendekatan yang belum banyak dieksplorasi pada penelitian terdahulu di bidang ini [19] [9] [20]. Selain itu, penelitian ini menghasilkan data empiris tentang performa model-model LLM *open source* lokal berukuran kecil hingga menengah dalam tugas analisis keamanan kode, yang relevan bagi institusi dengan sumber daya komputasi terbatas.

Desain modular *platform* memungkinkan pengembangan lebih lanjut oleh peneliti lain, misalnya dengan mengganti alat transformasi untuk mendukung bahasa pemrograman lain, menggunakan model AI yang lebih baru, atau memperluas pemetaan ke standar keamanan lain selain ISO/IEC 27001:2022.

### 1.6.2 Manfaat Praktis

*Platform* yang dibangun memudahkan proses migrasi aplikasi PHP lawas, khususnya bagi organisasi yang tidak memiliki banyak sumber daya pengembangan. Pemeriksaan kepatuhan keamanan disertakan langsung dalam proses migrasi [18], sehingga kerentanan dapat ditemukan bersamaan dengan konversi dan laporan kepatuhan dihasilkan secara otomatis. Dukungan versi target dari PHP 5.2 hingga PHP 8.x mengakomodasi berbagai strategi migrasi, baik yang bertahap maupun yang langsung. Seluruh *platform* bersifat *open source* dan dapat digunakan secara gratis.

## 1.7 Sistematika Penulisan

Skripsi ini terdiri dari lima bab.

**Bab I: Pendahuluan** memaparkan latar belakang masalah, rumusan masalah, tujuan penelitian, batasan penelitian, keamanan dan privasi data, manfaat penelitian, dan sistematika penulisan.

**Bab II: Tinjauan Pustaka dan Dasar Teori** membahas penelitian-penelitian sebelumnya yang relevan dan kerangka teoritis, mencakup perubahan versi PHP, standar ISO/IEC 27001:2022, metodologi *secure coding*, analisis kode berbasis LLM, serta alat-alat yang digunakan: Rector, Semgrep, PHPStan, dan Ollama.

**Bab III: Metodologi Penelitian** menjelaskan arsitektur sistem, tahapan penelitian, desain *prompt* LLM, pemetaan kontrol ISO/IEC 27001:2022, rancangan eksperimen komparasi model, serta metode pengujian dan evaluasi.

**Bab IV: Hasil dan Diskusi** menyajikan hasil implementasi *pipeline* pada dua studi kasus (FT UGM dan CBT DTI UGM), mencakup tingkat keberhasilan konversi, perbandingan temuan keamanan sebelum dan sesudah konversi, hasil komparasi lima model LLM, dan diskusi mengenai kekuatan dan keterbatasan sistem.

**Bab V: Kesimpulan dan Saran** merangkum temuan berdasarkan tujuan yang ditetapkan dan memberikan saran untuk penelitian berikutnya.

# BAB II
# TINJAUAN PUSTAKA DAN DASAR TEORI

## 2.1 Tinjauan Literatur

Bagian ini membahas penelitian-penelitian sebelumnya yang relevan dengan tiga fokus utama studi ini: analisis keamanan otomatis pada aplikasi PHP, deteksi kerentanan berbasis LLM, dan transformasi kode yang melibatkan LLM. Masing-masing karya dibahas dari sisi masalah yang diatasi, kontribusinya, dan keterbatasannya. Bagian ini ditutup dengan tabel perbandingan yang memperlihatkan posisi penelitian ini di antara karya-karya tersebut.

### 2.1.1 Perlindungan Otomatis Aplikasi PHP dari Injeksi SQL

Merlo, Letarte, dan Antoniol [19] pada 2007 sudah menunjukkan bahwa analisis keamanan dan perbaikan kode PHP dapat diotomatisasi. Pendekatan mereka menggabungkan analisis statis, analisis dinamis, dan rekayasa ulang kode untuk melindungi aplikasi PHP lama dari injeksi SQL tanpa mengubah fungsionalitasnya. Caranya adalah dengan mengekstrak penjaga berbasis model dari kasus uji tepercaya, lalu secara otomatis menyisipkannya ke titik-titik akses basis data dalam kode sumber.

Pendekatan ini terbukti efektif: pengujian pada phpBB versi 2.0.0 (37.193 baris kode PHP 4.2.2) berhasil menghentikan semua 176 serangan injeksi yang diuji tanpa satu pun *false positive*. Namun, pendekatan ini memiliki keterbatasan. Kualitas perlindungan sepenuhnya bergantung pada kelengkapan rangkaian pengujian yang digunakan selama fase analisis dinamis. Titik akses yang tidak tercakup pengujian tidak terlindungi.

Karya ini relevan bukan karena pendekatan teknisnya, melainkan karena membuktikan bahwa analisis keamanan dan perbaikan kode PHP skala besar sangat mungkin diotomatisasi. Mengingat studi ini terbit hampir dua dekade lalu namun solusi komprehensif berstandar modern belum juga tersedia, jelas ada celah penelitian yang harus segera diisi.

### 2.1.2 Menggunakan LLM untuk Menemukan dan Memantau Kerentanan

Akuthota dkk. [9] meneliti kemampuan GPT-3.5-Turbo untuk deteksi dan pemantauan kerentanan perangkat lunak. Pendekatannya murni berbasis *prompt engineering* tanpa pelatihan ulang model sama sekali. Model berhasil menemukan pola kerentanan yang umum, meskipun akurasinya menurun seiring meningkatnya kompleksitas pola yang dicari.

Li dkk. [10] mengambil pendekatan yang lebih terstruktur. Mereka menggabungkan analisis *patch* historis dengan analisis statis, lalu menambahkan

tabel fitur kerentanan dan templat penganalisis untuk membuat klasifikasi LLM lebih akurat, terutama pada kerentanan yang membutuhkan pemahaman aliran data lintas fungsi. Evaluasi pada basis kode Linux kernel berhasil menemukan 44 bug baru, 37 di antaranya dikonfirmasi dan 30 diperbaiki, dengan hasil yang lebih baik dibandingkan analisis statis saja.

Dari kedua studi tersebut, terlihat bahwa LLM umum mampu mendeteksi pola kerentanan murni melalui *prompt engineering*, dan penggabungan LLM dengan analisis statis terstruktur memberikan hasil yang lebih akurat untuk kasus yang kompleks. Namun, hambatan yang sama muncul di keduanya: model dijalankan via API *cloud*, yang menjadi kendala nyata ketika kode yang dianalisis bersifat rahasia. Penelitian ini mengatasi kendala tersebut dengan menjalankan semua model secara lokal melalui Ollama, sehingga tidak ada kode sumber yang keluar dari lingkungan organisasi.

### 2.1.3 Transformasi Kode dengan Bantuan LLM

Stümpfle dkk. [20] mengusulkan pendekatan untuk mengubah varian perangkat lunak yang dikloning menjadi *Software Product Line* (SPL) menggunakan LLM. Metodenya bekerja dalam tiga tahap: penyaringan titik variasi untuk menemukan perbedaan kode yang relevan secara semantik antar varian, rekayasa *prompt* untuk menghasilkan artefak SPL yang benar, dan *loop* umpan balik penyempurnaan diri yang mengirimkan hasil pengujian kembali ke model secara berulang hingga kode lolos kompilasi dan pengujian fungsional. *Loop* umpan balik ini terbukti secara signifikan mengurangi kegagalan akibat halusinasi model, meskipun efektivitasnya tetap bergantung pada kualitas langkah penyaringan awal.

Baik Stümpfle dkk. maupun penelitian ini menggunakan LLM untuk transformasi kode yang melampaui substitusi sintaksis dan membutuhkan pemahaman semantik. *Loop* umpan balik dalam Stümpfle dkk. memiliki fungsi konseptual yang mirip dengan peran validasi PHPStan dalam *pipeline* penelitian ini: keduanya menggunakan *output* alat analisis statis untuk memverifikasi apakah transformasi berhasil. Perbedaan utamanya terletak pada fokus: Stümpfle dkk. menangani ekstraksi fitur lintas varian untuk rekayasa lini produk, sedangkan penelitian ini berfokus pada migrasi versi dan verifikasi kepatuhan keamanan untuk kode warisan dalam satu bahasa.

### 2.1.4 Pendekatan Neuro-Simbolik untuk Deteksi Kerentanan Keamanan

Li, Dutta, dan Naik [11] memperkenalkan IRIS, sistem yang menggabungkan LLM dengan analisis statis untuk deteksi kerentanan pada tingkat seluruh repositori. Masalah yang diangkat cukup spesifik: alat analisis statis tradisional seperti CodeQL sangat bergantung pada spesifikasi *taint* yang dibuat manual, dan spesifikasi ini hampir pasti tidak lengkap karena jumlah API pihak ketiga terus bertambah. LLM juga tidak bisa

berjalan sendiri karena tidak mampu melakukan penalaran yang mencakup keseluruhan aliran kode dalam proyek berskala besar.

Solusi IRIS adalah membiarkan LLM menghasilkan spesifikasi *taint* secara otomatis berdasarkan pemahaman kontekstual terhadap API yang relevan, lalu menyerahkan analisis jalur ke alat analisis statis yang sudah ada. Pada *dataset* CWE-Bench-Java yang terdiri dari 120 kerentanan yang divalidasi secara manual, IRIS dengan GPT-4 berhasil mendeteksi 55 kerentanan dibandingkan hanya 27 oleh CodeQL saja, sekaligus menurunkan rata-rata *false discovery rate* sebesar 5,21 poin persentase.

Hasil ini menjadi dasar empiris pemilihan pendekatan gabungan dalam penelitian ini. Perbedaan utamanya adalah IRIS mengandalkan GPT-4 via *cloud* dan hanya diuji pada Java. Penelitian ini mengambil pendekatan konseptual yang serupa, tetapi dengan model lokal melalui Ollama dan menargetkan PHP.

### 2.1.5 Dampak Jangka Panjang Transformasi Kode Otomatis

Strobl dkk. [8] menganalisis tiga kasus transformasi kode otomatis skala besar di sebuah organisasi pemerintah, semuanya melibatkan konversi sistem warisan dari COBOL dan PL/I ke Java dengan ukuran lebih dari satu juta baris kode. Fokus utama studi ini bukan pada keberhasilan transformasi, melainkan pada dampak pascatransformasi.

Hasil studi tersebut mengungkapkan permasalahan yang signifikan. Transformasi otomatis seringkali menghasilkan kode yang sintaksis valid di bahasa target, tapi mempertahankan karakteristik struktural dari bahasa sumber sehingga sulit dipahami pengembang yang tidak kenal sistem lama. Lebih lanjut, sistem yang memiliki *test suite* otomatis sejak awal jauh lebih mudah dirawat pascatransformasi, sementara sistem tanpa pengujian otomatis menjadi beban jangka panjang.

Temuan ini menjadi alasan mengapa penelitian ini menyertakan PHPStan sebagai validasi pasca-konversi dan Semgrep sebagai pemindaian keamanan sebelum dan sesudah transformasi. Masalah yang ditinggalkan oleh konversi otomatis perlu terdeteksi segera, bukan bertahun-tahun kemudian.

### 2.1.6 Transformasi Kode Berbasis Pola untuk Migrasi Aplikasi

Cai dkk. [21] mengusulkan pendekatan transformasi kode berbasis pola untuk migrasi aplikasi ke lingkungan *cloud*. Prosesnya iteratif: pemindaian kode untuk mengidentifikasi blok yang perlu diubah, pencocokan pola menggunakan ekspresi reguler yang diperluas, perancangan templat untuk kode target, lalu eksekusi aturan transformasi. Pendekatan ini diuji pada 19 proyek *open source* Java dengan memigrasikannya ke Amazon Web Services (AWS).

Kontribusi utama dari pendekatan ini adalah membuktikan bahwa transformasi berbasis pola terbukti bisa mengurangi pekerjaan manual berulang pada migrasi skala besar. Namun, terdapat keterbatasan mendasar pada pendekatan tersebut. Ekspresi reguler yang diperluas tetap beroperasi pada level teks, bukan pada representasi struktural kode. Hasil ini membuatnya rentan gagal pada konstruksi kode yang kompleks atau tidak mengikuti pola yang telah didefinisikan sebelumnya.

Rector [6] bekerja pada AST, bukan teks, sehingga transformasinya lebih dapat diandalkan untuk kode produksi yang beragam. Selain itu, Cai dkk. [21] tidak menyertakan aspek keamanan atau kepatuhan standar sama sekali dalam pendekatan mereka. Perbedaan ini mencerminkan bagaimana kebutuhan dalam transformasi kode otomatis telah berkembang: dari sekadar memindahkan kode ke lingkungan baru, menjadi memastikan kode yang dipindahkan juga aman.

### 2.1.7 Migrasi Kode Berbasis AST di Lingkungan Produksi

Chen dkk. [7] mendokumentasikan penerapan migrasi kode berbasis AST pada skala produksi di Microsoft. Pendekatan mereka menggunakan penggantian simbol berbasis AST untuk memigrasikan basis kode TypeScript berskala besar secara otomatis. Kunci dari pendekatan ini adalah bahwa transformasi dilakukan pada tingkat pohon sintaksis, bukan teks, sehingga menghasilkan perubahan yang struktural konsisten dan tidak bergantung pada gaya penulisan kode masing-masing pengembang.

Signifikansi studi ini terletak pada validasi teknis pendekatannya di lingkungan produksi nyata, bukan pada skala operasi. Chen dkk. mendemonstrasikan bahwa transformasi berbasis AST dapat diterapkan secara otomatis pada basis kode TypeScript di Microsoft dan menghasilkan perubahan yang struktural konsisten, mengkonfirmasi kelayakan teknis pendekatan serupa yang mendasari Rector untuk migrasi kode dalam konteks institusional.

Ada dua perbedaan konteks penting. Pertama, migrasi yang dilakukan Chen dkk. adalah TypeScript ke TypeScript (*refactoring* dalam bahasa yang sama), sedangkan penelitian ini berhadapan dengan *breaking changes* antar versi PHP yang melibatkan penghapusan API dan perubahan sistem tipe. Kedua, seperti halnya Cai dkk. [21], studi ini tidak menyentuh aspek keamanan sama sekali. Temuan keamanan yang muncul selama atau setelah migrasi bukan bagian dari cakupan mereka.

### 2.1.8 Ringkasan Perbandingan

Tabel 2.1. Perbandingan Penelitian Terdahulu dan Karakteristik Penelitian

| **Studi** | **Bahasa** | **AI/LLM Digunakan** | **Tugas Utama** |
|---|---|---|---|
| Merlo dkk. [19] | PHP | Tidak ada | *Security remediation* |
| Akuthota dkk. [9] | Java | GPT-3.5-Turbo | *Vulnerability detection* |
| Li dkk. [10] | C (Linux kernel) | LLM + analisis statis | *Vulnerability detection* |
| Stümpfle dkk. [20] | C/C++ | LLM | *Code transformation* |
| Li, Dutta & Naik [11] | Java | GPT-4 | *Vulnerability detection* |
| Strobl dkk. [8] | COBOL/PL/I | Tidak ada | *Code transformation* |
| Cai dkk. [21] | Java | Tidak ada | *Cloud migration* |
| Chen dkk. [7] | TypeScript | Tidak ada | *Code migration* (AST) |
| **Penelitian ini** | PHP | Membandingkan 5 LLM lokal dan memilih AI terbaik untuk migrasi PHP | *Version migration* + *security compliance* |

Tabel 2.2. Perbandingan Penelitian Terdahulu dan Aspek Metodologi

| **Studi** | ***Deployment*** | **Validasi Pasca-Konversi** | **Standar Keamanan** | ***Open Source*** |
|---|---|---|---|---|
| Merlo dkk. [19] | N/A | Tidak | Tidak ada | Ya |
| Akuthota dkk. [9] | *Cloud* | Tidak | Tidak ada | Tidak |
| Li dkk. [10] | *Cloud* | Tidak | Tidak ada | Tidak |
| Stümpfle dkk. [20] | *Cloud* | Ya (*feedback loop*) | Tidak ada | Tidak |

*(bersambung ke halaman berikutnya)*

*(lanjutan Tabel 2.2)*

| **Studi** | ***Deployment*** | **Validasi Pasca-Konversi** | **Standar Keamanan** | ***Open Source*** |
|---|---|---|---|---|
| Li, Dutta & Naik [11] | *Cloud* | Tidak | Tidak ada | Ya (IRIS) |
| Strobl dkk. [8] | N/A | Tidak | Tidak ada | Tidak |
| Cai dkk. [21] | N/A | Tidak | Tidak ada | Tidak |
| Chen dkk. [7] | N/A | Tidak | Tidak ada | Tidak |
| **Penelitian ini** | Lokal (Ollama) | Ya (PHPStan + Semgrep) | ISO/IEC 27001:2022 | Ya |

Kedua tabel memperlihatkan kesenjangan yang konsisten. Studi-studi yang berfokus pada keamanan tidak menangani migrasi versi. Studi-studi yang berfokus pada transformasi kode, termasuk yang sudah terbukti di skala industri seperti Chen dkk. [7] tidak menyertakan pemeriksaan keamanan maupun pemetaan ke standar formal. Tidak satu pun dari kedelapan studi yang menggabungkan migrasi versi PHP, analisis keamanan berbasis LLM, validasi pasca-konversi, dan kepatuhan ISO/IEC 27001:2022 dalam satu alur kerja *open source* dengan *deployment* lokal.

Kedua tabel di atas membandingkan kehadiran fitur (*feature presence*), bukan performa absolut. Mengingat kedelapan studi pembanding menyasar bahasa target dan jenis kerentanan yang berbeda, komparasi kinerja secara langsung tidak dapat dilakukan. Kontribusi utama penelitian ini tidak sekadar memperbanyak cakupan fitur, melainkan mengintegrasikan kemampuan spesifik yang belum dieksplorasi pada konteks PHP: menggunakan LLM lokal untuk menerjemahkan temuan mentah Semgrep menjadi penjelasan dan saran remediasi yang dapat diaudit, tanpa mengubah peran Semgrep sebagai detektor maupun Rector sebagai mesin transformasi. Rector, Semgrep, dan PHPStan sendiri bukan kontribusi baru dari penelitian ini; ketiganya digunakan sebagaimana adanya. Kombinasi penempatan lapisan AI di antara ketiganya, dengan validasi pasca-konversi dan pemetaan ke ISO/IEC 27001:2022 yang melekat pada alur kerja, adalah kontribusi yang diklaim penelitian ini, dan klaim tersebut diuji secara empiris melalui implementasi pada Bab IV, bukan diasumsikan dari tabel perbandingan ini.

Berdasarkan tinjauan literatur di atas, terlihat jelas bahwa riset mengenai perbaikan keamanan, deteksi kerentanan berbasis LLM, dan transformasi kode masih berkembang secara terpisah. Kemajuan terkini menunjukkan bahwa kombinasi LLM dengan alat analisis statis adalah arah yang paling menjanjikan, sebagaimana dibuktikan oleh IRIS [11] dan St"umpfle dkk. [20]. Kesenjangan penelitian yang belum tertangani

adalah penerapan kombinasi ini pada konteks migrasi versi PHP dengan kendala privasi data. Penelitian ini mengisi kesenjangan tersebut dengan membangun *pipeline* yang menempatkan LLM lokal sebagai lapisan interpretasi di atas Semgrep dan Rector, serta memetakannya ke standar kepatuhan formal ISO/IEC 27001:2022 dalam satu alur kerja yang dapat diaudit.

### 2.1.9 Pertanyaan Penelitian

Berdasarkan kesenjangan yang teridentifikasi, penelitian ini mengajukan empat pertanyaan penelitian berikut:

1. Apakah *pipeline* yang dibangun berhasil memigrasikan aplikasi web PHP 7.x ke PHP 8.3 tanpa menimbulkan regresi fungsional pada kode sumber yang dikonversi?
2. Apakah hasil konversi mempertahankan fungsionalitas kode sumber asli sebagaimana dapat diverifikasi melalui pemeriksaan validitas sintaksis dan kompatibilitas tipe pasca-konversi?
3. Apakah deteksi kerentanan keamanan, transformasi kode, dan pembuktian kepatuhan terhadap kontrol ISO/IEC 27001:2022 Annex A.8 dapat disatukan dalam satu alur kerja otomatis sehingga temuan keamanan tidak lagi memerlukan penerjemahan manual menjadi bukti audit?
4. Apakah *AI Engine* berbasis LLM *open source* lokal yang diusulkan menghasilkan penjelasan dan saran remediasi yang secara terukur lebih informatif dibandingkan keluaran *pipeline* tanpa komponen AI?

## 2.2 Dasar Teori

### 2.2.1 PHP dan Perubahan Antar Versi

PHP (*Hypertext Preprocessor*) telah menjadi bahasa skrip sisi *server* yang paling banyak digunakan di web sejak akhir 1990-an [22]. W3Techs mencatat bahwa 71,7% dari seluruh situs web yang memiliki bahasa pemrograman sisi *server* menggunakan PHP [1].

Untuk memahami tantangan migrasi yang dihadapi kode lama, perlu dipahami dulu karakteristik PHP 5.x yang masih banyak ditemukan di sistem yang belum bermigrasi. PHP 5.x, aktif digunakan dari sekitar 2004 hingga awal 2010-an, memiliki karakteristik yang berbeda secara fundamental dari versi modern. Yang paling signifikan adalah ekstensi `ext/mysql` dengan fungsi-fungsi seperti `mysql_connect()`, `mysql_query()`, dan `mysql_fetch_array()`. Fungsi-fungsi ini tidak mendukung *prepared statements* secara asli, sehingga rentan terhadap injeksi SQL jika *input* pengguna tidak disanitasi secara manual. Ekstensi ini sudah dinyatakan *deprecated* sejak PHP 5.5 dan dihapus sepenuhnya di PHP 7.0 [2]. PHP 5.x juga menggunakan kata kunci `var` untuk deklarasi properti kelas, sintaks `array()` yang

panjang, dan belum mendukung *type hints* skalar. Rector dapat menangani sebagian besar transformasi sintaksis dari PHP 5.x melalui *ruleset* kumulatifnya [6], namun penggantian `mysql_*()` tidak dapat dilakukan oleh Rector karena membutuhkan pemahaman semantik tentang konteks penggunaan basis data. Keterbatasan inilah yang secara langsung memotivasi kehadiran AI Engine dalam *pipeline* penelitian ini.

PHP 7.x membawa peningkatan performa signifikan lewat Zend Engine 3.0, menambahkan deklarasi tipe skalar, petunjuk tipe pengembalian, dan operator penggabungan *null*. PHP 7.4, versi terakhir seri ini, menambahkan properti bertipe dan fungsi panah sebelum dukungan resminya berakhir 28 November 2022 [2].

PHP 8.x membuat sejumlah *breaking changes* yang membuat kode PHP 7.x tidak bisa langsung dijalankan [2]. Selain penghapusan fungsi-fungsi yang sudah *deprecated*, sistem tipe menjadi jauh lebih ketat dengan tipe *union* dan tipe campuran. Fitur baru seperti *match expression*, operator *nullsafe* (`?->`), argumen bernama, dan kompilasi JIT diperkenalkan. Penanganan kesalahan juga berubah: beberapa fungsi yang sebelumnya mengembalikan `false` kini melempar pengecualian. Untuk basis kode besar, melakukan semua perubahan ini secara manual jelas tidak realistis.

### 2.2.2 ISO/IEC 27001:2022 dan *Secure Coding*

ISO/IEC 27001:2022 adalah standar internasional untuk Sistem Manajemen Keamanan Informasi (ISMS) yang dikembangkan bersama oleh ISO dan IEC [18]. Standar ini menetapkan persyaratan untuk membuat, menerapkan, memelihara, dan terus meningkatkan ISMS dalam suatu organisasi. Annex A memuat 93 kontrol keamanan yang dibagi ke dalam empat domain: Organisasi, Manusia, Fisik, dan Teknologi. Dari 93 kontrol tersebut, penelitian ini hanya memetakan yang langsung relevan dengan pengembangan perangkat lunak:

- **A.8.25** (*Secure development lifecycle*) mengharuskan aturan pengembangan yang aman ditetapkan dan diikuti di setiap tahap, termasuk penggunaan pedoman *secure coding* dan alat analisis statis.
- **A.8.28** (*Secure coding*) mengharuskan aturan pengkodean yang aman diikuti untuk mencegah kerentanan umum seperti injeksi, autentikasi yang rusak, pengungkapan data sensitif, dan kriptografi yang tidak aman.
- **A.8.29** (Pengujian keamanan dalam pengembangan dan penerimaan) mengharuskan pengujian keamanan menjadi bagian dari proses pengembangan sebelum kode masuk produksi.

Ketiga kontrol ini, bersama A.5.17, A.8.24, dan A.8.26 yang turut dipetakan oleh *pipeline*, menjadi acuan resmi untuk mesin pengecekan keamanan dalam penelitian ini

[18]. Aturan Semgrep yang digunakan berhubungan langsung dengan A.8.28 dan A.8.29. Struktur *pipeline* secara keseluruhan mencerminkan kebutuhan siklus hidup A.8.25.

### 2.2.3 OWASP Top 10 dan Kerentanan Aplikasi Web PHP

OWASP Top 10 adalah referensi standar untuk risiko keamanan paling kritis pada aplikasi web yang diterbitkan secara berkala oleh OWASP berdasarkan data dari ratusan organisasi di seluruh dunia [23]. Versi terbaru, OWASP Top 10:2021, relevan langsung dengan kondisi aplikasi PHP lama.

Injeksi menempati posisi ketiga dan merupakan temuan paling umum pada kode PHP lama [23]. Injeksi SQL sering muncul dari penggunaan `mysql_query()` tanpa parameterisasi. XSS juga termasuk kategori injeksi dan terjadi saat *output* tidak dienkode sebelum ditampilkan ke peramban. Kegagalan kriptografi, yang berada di posisi kedua, kerap muncul dalam praktik penyimpanan kata sandi menggunakan `md5()` atau `sha1()`, padahal keduanya sudah lama tidak dianggap aman.

Dalam penelitian ini, Semgrep dijalankan dengan *ruleset* `p/php` dan `p/owasp-top-ten` [4], yang secara langsung dipetakan ke kategori-kategori dalam OWASP Top 10:2021. Temuan Semgrep kemudian dipetakan ke kontrol ISO/IEC 27001:2022 yang relevan melalui skrip `iso_mapper.py`.

### 2.2.4 DevSecOps dan Integrasi Keamanan dalam *Pipeline* Pengembangan

DevSecOps adalah praktik yang mengintegrasikan keamanan ke dalam setiap tahap pengembangan perangkat lunak [18]. Pendekatan tradisional menempatkan pengujian keamanan sebagai langkah terpisah di akhir proses, dengan tim keamanan yang baru bekerja setelah kode selesai. Sebaliknya, DevSecOps mengintegrasikan keamanan sebagai bagian dari setiap keputusan teknis selama pengembangan berlangsung.

Dalam praktiknya, DevSecOps diwujudkan melalui integrasi alat keamanan otomatis ke dalam *pipeline*: analisis statis kode sumber (SAST), pemindaian dependensi, dan pengujian keamanan dinamis. Kerentanan yang ditemukan lebih awal jauh lebih murah untuk diperbaiki dibandingkan yang baru ditemukan setelah sistem berjalan di produksi.

*Pipeline* dalam penelitian ini menerapkan prinsip DevSecOps secara langsung: Semgrep memindai sebelum dan sesudah konversi, PHPStan memvalidasi kompatibilitas kode, dan AI Engine menangani kasus yang butuh analisis semantik. Keamanan bukan langkah audit terpisah, tapi bagian dari proses migrasi itu sendiri [18].

### 2.2.5 Metodologi *Research and Development* dengan Model Prototipe

Metodologi *Research and Development* (R&D) adalah pendekatan penelitian yang menghasilkan produk atau sistem yang dapat diuji dan divalidasi secara empiris, bukan sekadar analisis teoritis. Model prototipe adalah salah satu pendekatan dalam R&D di mana sistem dibangun dalam versi awal, diuji, lalu diperbaiki berdasarkan hasil pengujian secara berulang hingga mencapai kondisi yang stabil [24]. Pendekatan ini sesuai untuk sistem yang keputusan desainnya bergantung pada perilaku komponen eksternal yang sulit diprediksi di awal, seperti respons model LLM terhadap format *prompt* tertentu atau cara Rector menangani pola kode yang tidak biasa.

### 2.2.6 *Abstract Syntax Trees* dan *Static Analysis*

Analisis statis adalah teknik pemeriksaan kode sumber tanpa menjalankan program. Dalam konteks keamanan perangkat lunak, teknik ini digunakan untuk mengidentifikasi pola kerentanan, praktik pengkodean tidak aman, dan pelanggaran kebijakan secara otomatis. Analisis statis sudah menjadi bagian standar dari *pipeline* DevSecOps modern karena bisa dijalankan terus-menerus selama pengembangan tanpa beban tambahan yang signifikan [18].

*Abstract Syntax Trees* (AST) adalah representasi pohon dari kode sumber di mana setiap *node* mewakili konstruksi sintaksis seperti ekspresi, pernyataan, atau deklarasi. Alat yang bekerja dengan AST bisa menganalisis dan mengubah kode secara struktural, berbeda dari pendekatan berbasis teks biasa yang digunakan oleh Cai dkk. [21]. Rector menggunakan PHP AST untuk menemukan pola yang perlu diubah dan menghasilkan kode penggantinya. Chen dkk. [7] mendemonstrasikan bahwa pendekatan berbasis AST bukan sekadar teori; mereka menerapkannya untuk migrasi kode TypeScript di Microsoft pada skala produksi.

### 2.2.7 Rector: Alat Transformasi Kode Berbasis AST

Rector adalah alat *open source* untuk transformasi kode PHP berbasis AST [6]. Prosesnya: kode sumber PHP diubah menjadi AST, aturan transformasi diterapkan, lalu kode PHP baru dihasilkan dari pohon yang sudah dimodifikasi. Rector dilengkapi aturan bawaan untuk berpindah antar versi PHP, termasuk `LevelSetList::UP_TO_PHP_80` hingga `UP_TO_PHP_85`, dan *ruleset*-nya bersifat kumulatif sehingga mencakup semua perubahan dari versi sumber hingga versi target.

Dalam penelitian ini, Rector berperan sebagai mesin transformasi utama. Transformasi sintaksis dan struktural dari PHP 5.x atau 7.x ke PHP 8.x ditangani sepenuhnya oleh Rector. Transformasi ekstensi `ext/mysql` tidak dapat ditangani

karena membutuhkan penulisan ulang semantik, bukan sekadar penggantian sintaks. Oleh karena itu, tugas tersebut didelegasikan kepada AI Engine.

### 2.2.8 PHPStan: Alat Analisis Statis untuk PHP

PHPStan adalah alat analisis statis untuk PHP yang memeriksa kebenaran tipe data, konsistensi pemanggilan fungsi, dan kesalahan logika tanpa menjalankan program [5]. Berbeda dari Semgrep yang mencari pola kerentanan keamanan, PHPStan fokus pada integritas kode dari sisi kompatibilitas dan tipe.

PHPStan bekerja dengan sistem level 0–9, di mana level lebih tinggi berarti pemeriksaan lebih ketat. Dalam *pipeline* penelitian ini, PHPStan digunakan pada level 5 sebagai komponen validasi pasca-konversi. Setelah Rector selesai, PHPStan memeriksa apakah kode hasil konversi kompatibel dengan versi PHP target dan bebas dari kesalahan tipe. Perannya mendukung kontrol A.8.29 ISO/IEC 27001:2022 [18] dengan memastikan kode yang dihasilkan secara otomatis tidak mengandung kesalahan yang bisa memengaruhi keandalan sebelum masuk produksi.

### 2.2.9 Semgrep: Alat Analisis Statis Ringan untuk Keamanan

Semgrep adalah alat analisis statis ringan yang mendukung lebih dari 30 bahasa pemrograman, termasuk PHP [4]. Aturan keamanannya ditulis dalam format YAML dengan sintaks yang mirip dengan kode sumber yang dianalisis, sehingga pengguna bisa membuat aturan sendiri tanpa perlu memahami detail AST bahasa target. Registri aturan publiknya mencakup berbagai kerentanan umum seperti injeksi SQL, XSS, kriptografi lemah, dan deserialisasi tidak aman.

Dalam penelitian ini, Semgrep digunakan dua kali: sebelum konversi untuk menetapkan *baseline* keamanan kode asli, dan sesudah konversi untuk memastikan tidak ada kerentanan baru yang muncul. Kedua pemindaian ini mendukung kontrol A.8.28 dan A.8.29 ISO/IEC 27001:2022 [18].

### 2.2.10 Model-Model LLM untuk Analisis Kode

LLM (*Large Language Model*) adalah model bahasa neural berbasis *transformer* yang dilatih pada data teks dan kode dalam jumlah besar. LLM yang dilatih pada kode bisa memahami semantik kode, mengidentifikasi pola kerentanan, menyarankan perbaikan, dan menghasilkan kode baru [9]. Kemampuan ini berbeda dari analisis berbasis aturan: LLM mempertimbangkan konteks dan tujuan kode, bukan hanya strukturnya.

Penelitian ini mengevaluasi lima model LLM *open source* yang dijalankan secara lokal. Kriteria pemilihan ada dua: model harus bisa dijalankan di GPU dengan VRAM

8 GB menggunakan kuantisasi Q4_0, dan harus memiliki kemampuan pemahaman kode yang terdokumentasi dalam literatur ilmiah.

**DeepSeek Coder 6.7b** [12] memiliki 6,7 miliar parameter dan dilatih pada 2 triliun token dengan komposisi 87% kode sumber dari lebih dari 80 bahasa pemrograman.

**Qwen2.5-Coder 7b** [13] adalah model kode dari Alibaba dengan 7 miliar parameter, dilatih pada 5,5 triliun token kode, dan menunjukkan kemampuan yang kuat pada penyelesaian kode dan perbaikan *bug*.

**CodeLlama 7b** [14] dikembangkan Meta AI berbasis Llama 2. Model ini mendukung konteks hingga 100 ribu token dan tersedia dalam varian *instruct* yang dioptimalkan untuk mengikuti instruksi.

**Mistral 7b** [15] adalah model tujuan umum dari Mistral AI yang menggunakan mekanisme *grouped-query attention* (GQA) dan *sliding window attention* (SWA), membuatnya efisien secara komputasi.

**Llama 3.1 8b** [16] adalah model 8 miliar parameter dari Meta AI yang dilatih pada lebih dari 15 triliun token multibahasa dengan peningkatan signifikan dalam kemampuan penalaran dibanding pendahulunya.

Li, Dutta, dan Naik [11] sudah menunjukkan bahwa kombinasi LLM dan analisis statis menghasilkan deteksi kerentanan yang lebih baik dari masing-masing yang berjalan sendiri. Perbandingan performa kelima model di atas dalam konteks analisis keamanan kode PHP akan dibahas secara empiris di Bab IV.

### 2.2.11 Metrik Evaluasi Kualitas Keluaran LLM

Evaluasi kualitas keluaran model LLM pada tugas pemrograman memerlukan metrik yang dapat diverifikasi secara otomatis dan objektif. Dua dimensi utama yang digunakan dalam literatur adalah kemampuan mengikuti instruksi format dan kemampuan menghasilkan kode yang valid secara sintaksis.

**Pass@k** adalah metrik standar untuk mengukur kualitas kode yang dihasilkan LLM [14] [13]. Metrik ini mengukur probabilitas bahwa setidaknya satu dari $k$ sampel kode yang dihasilkan model dapat lulus serangkaian pengujian. Pada nilai $k = 1$ dengan *greedy decoding*, pass@1 mengukur persentase percobaan yang menghasilkan solusi yang berhasil melewati pengujian dari total percobaan.

**Instruction-following evaluation** mengukur sejauh mana model mematuhi instruksi format yang diberikan dalam *prompt*, terlepas dari kualitas konten yang dihasilkan. Zhou dkk. [25] memperkenalkan IFEval yang mendefinisikan *prompt-level strict-accuracy* sebagai persentase *prompt* di mana seluruh instruksi yang dapat diverifikasi terpenuhi sekaligus. Pendekatan ini bersifat *all-or-nothing*: respons yang

hanya memenuhi sebagian instruksi tidak dihitung sebagai patuh.

Penelitian ini mengadaptasi kedua konsep tersebut ke dalam lima metrik formal yang disesuaikan dengan konteks migrasi PHP dan analisis keamanan.

**Tingkat Konversi Rector** (*Conversion Rate*, CR) mengukur persentase *file* PHP yang berhasil dikonversi oleh Rector:

$$CR = \frac{f_{success}}{f_{total}} \times 100\% \tag{2-1}$$

di mana $f_{success}$ adalah jumlah *file* yang berhasil dikonversi dan $f_{total}$ adalah total *file* PHP dalam direktori `input/`.

**Format Compliance Rate** (FCR) mengadaptasi konsep *prompt-level strict-accuracy* dari IFEval [25] untuk mengukur persentase respons model yang memenuhi ketiga elemen format *prompt* sekaligus:

$$FCR = \frac{r_{compliant}}{r_{total}} \times 100\% \tag{2-2}$$

di mana $r_{compliant}$ adalah jumlah respons yang patuh format dan $r_{total}$ adalah total percobaan per model. Metrik ini mencerminkan kemampuan model mengikuti instruksi, bukan kualitas kodenya.

**Syntax Validity Rate** (SVR) mengadaptasi konsep pass@1 [14] [13] untuk mengukur persentase blok kode PHP yang diekstraksi dan lolos `php -l`:

$$SVR = \frac{c_{valid}}{c_{extracted}} \times 100\% \tag{2-3}$$

di mana $c_{valid}$ adalah jumlah blok kode yang lolos pemeriksaan sintaks dan $c_{extracted}$ adalah total blok kode yang berhasil diekstraksi dari respons model. Hasil ini adalah metrik primer karena hasilnya konkret dan dapat diverifikasi secara independen.

**Security Finding Delta** ($\Delta F$) mengukur perubahan jumlah temuan keamanan antara *pre-scan* dan *post-scan*:

$$\Delta F = F_{post} - F_{pre} \tag{2-4}$$

Nilai $\Delta F < 0$ menunjukkan pengurangan temuan, $\Delta F = 0$ berarti tidak ada perubahan, dan $\Delta F > 0$ tidak otomatis diinterpretasikan sebagai regresi karena bisa mencerminkan kerentanan laten yang baru terlihat setelah transformasi kode. Selain keempat metrik di atas, penelitian ini juga mencatat **Mean Confidence Score** ($\bar{C}_m$) sebagai metrik sekunder yang mengukur rata-rata skor kepercayaan yang dilaporkan

sendiri oleh model $m$:

$$\bar{C}_m = \frac{1}{n} \sum_{i=1}^{n} c_i \tag{2-5}$$

Untuk mengukur konsistensi antar *run*, digunakan variansi:

$$\sigma_m^2 = \frac{1}{n} \sum_{i=1}^{n} (c_i - \bar{C}_m)^2 \tag{2-6}$$

di mana $n$ adalah jumlah *run* dan $c_i$ adalah skor kepercayaan pada *run* ke-$i$. Perlu dicatat bahwa $\bar{C}_m$ adalah metrik sekunder; model LLM berukuran kecil umumnya tidak terkalibrasi dengan baik dalam menilai *output* mereka sendiri [9, 11], sehingga skor ini tidak bisa dijadikan proksi kualitas kode. FCR dan SVR merupakan dua dimensi yang independen: model dapat mencapai FCR tinggi dengan SVR rendah, atau sebaliknya [25]. Hal ini didukung oleh evaluasi CodeIF yang menemukan bahwa kepatuhan LLM terhadap instruksi (terutama instruksi multi-konstrain) sering kali mengorbankan konsistensi dan validitas kode yang dihasilkan [26]. Implikasi dari pola ini dibahas di Bab IV.

### 2.2.12 Ollama: Menerapkan LLM Secara Lokal

Beberapa alternatif dengan fungsi serupa tersedia, di antaranya LM Studio [27] dan llama.cpp [28]. LM Studio menyediakan antarmuka grafis yang intuitif untuk menjalankan model secara lokal, namun tidak menyediakan API REST bawaan yang dapat dipanggil langsung dari skrip Python tanpa konfigurasi tambahan. llama.cpp adalah *backend* inferensi berperforma tinggi yang ditulis dalam C/C++ tanpa dependensi eksternal [28] dan menjadi dasar teknis Ollama, namun memerlukan kompilasi manual dan tidak memiliki lapisan manajemen model yang terintegrasi.

Ollama dipilih karena tiga alasan. Pertama, API REST-nya yang kompatibel dengan format OpenAI memungkinkan integrasi langsung ke skrip Python tanpa pustaka tambahan. Kedua, manajemen model dilakukan melalui satu perintah `ollama pull` tanpa konfigurasi manual, sehingga eksperimen dapat direproduksi dengan mudah di lingkungan lain. Ketiga, Ollama mendukung kuantisasi Q4_0 secara otomatis yang memungkinkan model berukuran 6–8 miliar parameter berjalan dalam VRAM 8 GB tanpa pengaturan tambahan.

### 2.2.13 Algoritma Ratcliff/Obershelp untuk Pengukuran *Similarity* Kode

Pengukuran kemiripan antara dua versi kode sumber memerlukan algoritma yang sensitif terhadap urutan dan struktur baris, bukan sekadar kehadiran kata kunci. Algoritma Ratcliff/Obershelp [29] dirancang untuk kebutuhan ini. Berbeda dari

*cosine similarity* yang merepresentasikan teks sebagai vektor frekuensi kata dan mengabaikan urutan, atau *edit distance* (Levenshtein) yang bekerja pada level karakter, Ratcliff/Obershelp menghitung rasio kecocokan berbasis blok terpanjang yang cocok secara rekursif antara dua sekuens.

Formula dasarnya adalah:

$$S = \frac{2M}{T} \tag{2-7}$$

di mana $M$ adalah jumlah elemen yang cocok dan $T$ adalah total elemen pada kedua sekuens. Nilai $S$ berada di rentang $0{,}0$ (tidak ada kesamaan) hingga $1{,}0$ (identik). Dalam konteks perbandingan kode sumber, algoritma ini diterapkan pada level baris (*line-level*), sehingga setiap baris kode diperlakukan sebagai satu elemen sekuens.

Python mengimplementasikan algoritma ini melalui `difflib.SequenceMatcher` dari *standard library*, yang menjadikannya dapat digunakan tanpa dependensi eksternal tambahan. Dalam penelitian ini, metrik ini digunakan pada tahap validasi akurasi untuk mengukur kemiripan antara keluaran Rector dan konversi referensi yang disiapkan secara manual, sebagaimana dibahas di Subbab **??**.

### 2.2.14 Evaluasi Berbasis Penilaian Manusia

Skala Likert [30] adalah skala penilaian ordinal yang digunakan untuk mengukur sikap atau penilaian subjektif terhadap suatu objek melalui serangkaian poin bertingkat, biasanya dari sangat tidak setuju hingga sangat setuju. Skala ini banyak digunakan dalam penelitian yang melibatkan penilaian kualitas oleh manusia karena sederhana diterapkan namun tetap memberikan gradasi penilaian yang lebih informatif dibandingkan skala biner.

*Wilcoxon signed-rank test* [31] adalah uji statistik nonparametrik untuk membandingkan dua kondisi berpasangan pada data berskala ordinal. Uji ini dipilih ketika asumsi normalitas data tidak dapat dipenuhi, sebagaimana umum terjadi pada sampel berukuran kecil atau data hasil penilaian skala Likert, sehingga lebih sesuai dibandingkan uji parametrik seperti *paired t-test*.

*Cohen's kappa* [32] adalah ukuran reliabilitas antar-*rater* (*inter-rater reliability*) yang mengukur tingkat kesepakatan antara dua *rater* setelah memperhitungkan kesepakatan yang mungkin terjadi secara kebetulan. Nilai ini penting untuk memastikan bahwa penilaian yang diberikan *rater* cukup konsisten sebelum hasilnya digunakan sebagai dasar penarikan kesimpulan.

# BAB III
# METODOLOGI PENELITIAN

Bab ini menjelaskan langkah-langkah yang diambil dalam penelitian ini untuk mencapai tujuan yang ditetapkan dalam Subbab 1.3. Cakupannya meliputi alat dan bahan yang digunakan, metode penelitian, arsitektur sistem, alur data *pipeline*, desain *prompt* LLM, pemetaan kontrol ISO/IEC 27001:2022, rancangan eksperimen komparasi model, dan metode evaluasi.

## 3.1 Alat dan Bahan Penelitian

### 3.1.1 Perangkat Keras

Seluruh proses pengembangan dan pengujian dalam penelitian ini dilakukan pada satu unit laptop HP Victus 16-r0xxx dengan spesifikasi sebagai berikut:

- Sistem operasi: Windows 11 Home Single Language 64-bit (Build 26200)
- Prosesor: Intel Core i7-13700HX, 24 inti logis, kecepatan dasar 2,1 GHz
- Memori: 16.384 MB RAM
- GPU diskrit: NVIDIA GeForce RTX 4060 dengan VRAM 8 GB
- GPU terintegrasi: Intel UHD Graphics
- Penyimpanan: NVMe SSD 1 TB

Kode sumber FT UGM dan DTI UGM bersifat rahasia dan tidak boleh dikirimkan ke layanan AI eksternal, sehingga seluruh komponen AI dalam *pipeline* ini dijalankan secara lokal melalui Ollama. RTX 4060 dengan VRAM 8 GB memungkinkan kelima model yang dievaluasi (masing-masing 6–8 miliar parameter) dimuat sepenuhnya ke GPU pada kuantisasi Q4_0, sehingga model dapat berjalan tanpa mengirimkan data ke luar lingkungan lokal.

### 3.1.2 Perangkat Lunak

- **Python 3.11+**: Bahasa pemrograman utama untuk membangun *pipeline* yang menghubungkan semua modul. Dipilih karena kemudahannya memanggil perintah eksternal via *subprocess* dan ketersediaan pustaka `requests` untuk berkomunikasi dengan API Ollama.
- **Rector**: Alat transformasi kode berbasis AST untuk PHP, dikonfigurasi secara dinamis sesuai versi target yang dipilih.
- **Semgrep**: Alat analisis statis untuk pemindaian kerentanan. Dijalankan dengan *ruleset*

`p/php` dan `p/owasp-top-ten`.

- **PHPStan**: Alat analisis statis untuk validasi tipe dan kompatibilitas kode PHP, dijalankan pada level 5.
- **Ollama**: *Platform* untuk menjalankan LLM secara lokal, diakses via API REST di `localhost:11434`. Justifikasi pemilihan Ollama dibandingkan alternatif LM Studio dan llama.cpp dibahas pada Subbab 2.2.12. Secara teknis, Ollama memuat bobot model dalam format GGUF ke dalam VRAM GPU menggunakan *backend* llama.cpp. Pada RTX 4060 dengan VRAM 8 GB, kelima model yang diuji menghasilkan rata-rata 20–40 token per detik pada kuantisasi Q4_0. Ollama menyimpan model yang sudah dimuat di VRAM selama sesi aktif sehingga latensi *cold start* hanya terjadi sekali per sesi.
- **PHP 8.4.1**: Interpreter PHP untuk validasi sintaks menggunakan `php -l`.
- **Composer**: Manajer dependensi PHP untuk instalasi Rector dan PHPStan.
- **Visual Studio Code**: Sebagai IDE utama selama pengembangan.

### 3.1.3 Model LLM yang Dievaluasi

Lima model LLM *open source* dievaluasi dalam penelitian ini: DeepSeek Coder 6.7b, Qwen2.5-Coder 7b, CodeLlama 7b, Mistral 7b, dan Llama 3.1 8b. Kelima model dijalankan pada kuantisasi Q4_0 agar dapat dimuat ke dalam VRAM 8 GB. Rincian spesifikasi dan literatur pendukung masing-masing model telah diuraikan pada Subbab 2.2.10.

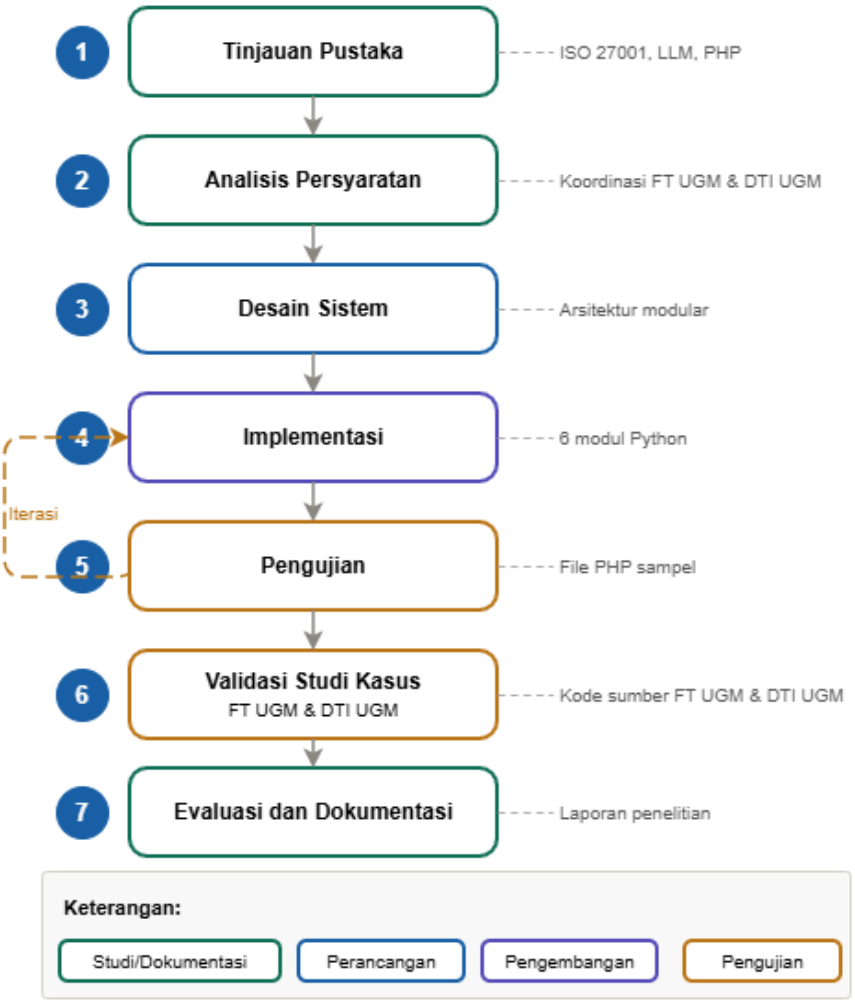
### 3.1.4 Bahan Penelitian

- **Dokumen ISO/IEC 27001:2022**: Acuan utama untuk pemetaan kontrol keamanan.
- **Kode sumber PHP dari FT UGM**: Kumpulan *file* PHP dari aplikasi *Dashboard* TCK berbasis CodeIgniter 3.x dengan PHP 7.4.9. Bersifat rahasia dan hanya digunakan untuk analisis statis tanpa akses ke basis data produksi.
- **Kode sumber PHP dari DTI UGM**: Kumpulan *file* PHP dari aplikasi *Computer-Based Test* (CBT) berbasis CodeIgniter 3.x dengan persyaratan PHP $\geq$ 7.0. Bersifat rahasia dengan perlakuan privasi data yang sama dengan dataset FT UGM.
- **Dokumentasi resmi PHP**: Referensi untuk memverifikasi kebenaran konversi Rector.
- **Registri aturan Semgrep**: Aturan publik `p/php` dan `p/owasp-top-ten` untuk pemindaian keamanan.

## 3.2 Metode Penelitian

Penelitian ini menggunakan metodologi R&D dengan model prototipe sebagaimana diuraikan pada Subbab 2.2.5.

*Pipeline* otomasi yang dirancang dalam penelitian ini merupakan implementasi dari manajemen perangkat lunak dalam skala basis kode nyata: Rector menangani pembaruan versi PHP secara sistematis, sementara komponen AI Engine berbasis LLM merekomendasikan perbaikan kerentanan yang teridentifikasi. Kedua fungsi ini berjalan dalam satu alur terpadu yang dapat diterapkan ulang pada basis kode lain. Gambar 3.1 menunjukkan alur penelitian yang terdiri dari tujuh tahapan.




Gambar 3.1. Alur penelitian dengan model prototipe

1. **Tinjauan pustaka**: Mempelajari permasalahan migrasi PHP lama, persyaratan ISO/IEC 27001:2022, kemampuan model LLM *open source*, dan alat analisis statis yang relevan. Koordinasi awal dengan FT UGM dan DTI UGM dilakukan pada tahap ini.
2. **Analisis persyaratan**: Mendefinisikan persyaratan fungsional dan non-fungsional

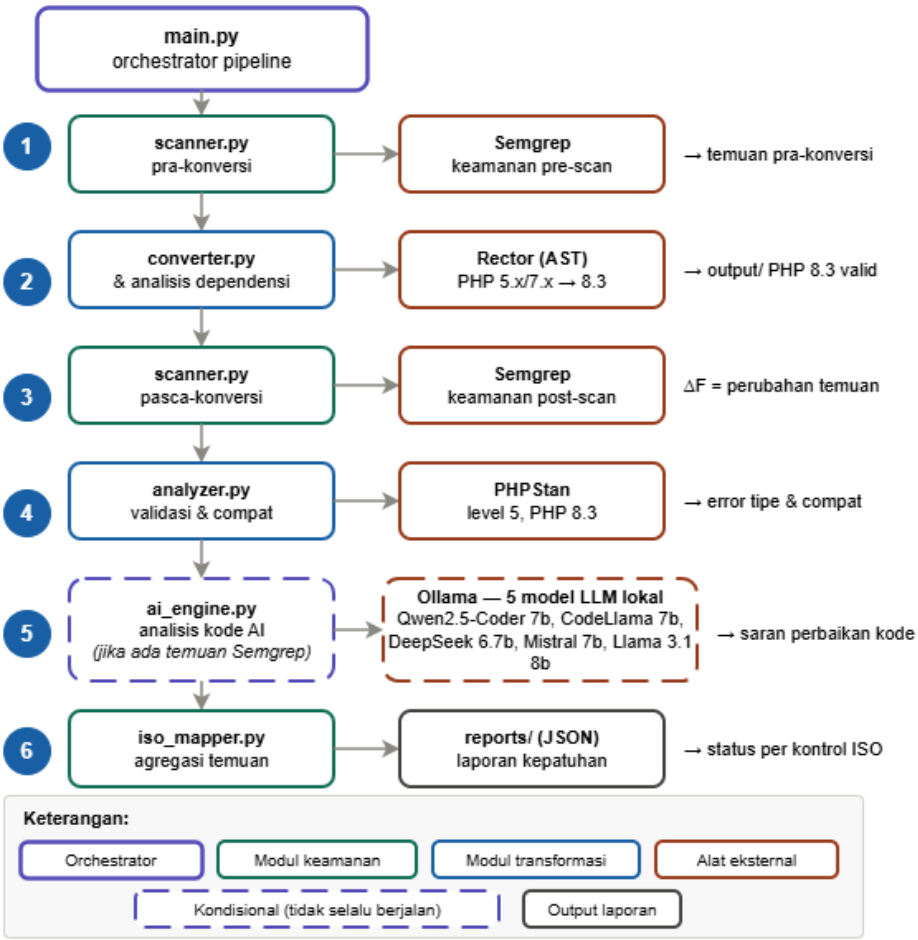
berdasarkan hasil tinjauan pustaka dan koordinasi dengan FT UGM dan DTI UGM, termasuk menentukan kontrol ISO yang dicakup dan persyaratan privasi data.

3. **Desain sistem**: Menentukan struktur enam modul *pipeline*, alur data antar modul, dan cara setiap modul saling berkomunikasi, termasuk merancang desain eksperimen komparasi LLM.
4. **Implementasi**: Membangun enam modul fungsional Python satu per satu, dikoordinasikan oleh `main.py` sebagai orkestrator. Setiap modul diuji secara terpisah sebelum diintegrasikan.
5. **Pengujian**: Menguji prototipe menggunakan *file* PHP buatan sendiri dan potongan kode representatif untuk memverifikasi setiap modul sebelum dijalankan pada dataset studi kasus. Jika ditemukan masalah, proses kembali ke tahap implementasi (iterasi).
6. **Validasi studi kasus**: Pengujian akhir menggunakan kode sumber FT UGM dan DTI UGM setelah prototipe stabil.
7. **Evaluasi dan dokumentasi**: Menganalisis hasil dan menyusun laporan penelitian.

## 3.3 Arsitektur Sistem

Sistem terdiri dari enam modul fungsional Python, di mana masing-masing modul menangani satu langkah spesifik dalam proses konversi dan analisis keamanan. Seluruh modul ini dikoordinasikan oleh `main.py` sebagai orkestrator. Arsitektur modular memungkinkan setiap bagian diuji, diperbaiki, atau diganti secara independen.

Gambar 3.2 menunjukkan hubungan antar modul.




Gambar 3.2. Arsitektur sistem *pipeline* dengan enam modul fungsional dan satu orkestrator

### 3.3.1 Scanner (scanner.py)

Scanner merupakan modul pertama yang dieksekusi dalam *pipeline*. Modul ini menjalankan Semgrep dengan *ruleset* `p/php` dan `p/owasp-top-ten` pada direktori `input/` sebelum konversi (*pre-scan*), dan pada direktori `output/` setelah konversi (*post-scan*) untuk mengukur perubahan postur keamanan melalui $\Delta F$. Fungsi `_classify()` di dalam modul ini juga menugaskan jenis kerentanan, prioritas, dan kontrol ISO awal untuk setiap temuan, sebagaimana dijelaskan pada Subbab 3.6.

- **Input**: direktori berisi *file* PHP (`input/` untuk *pre-scan*, `output/` untuk *post-scan*).
- **Output**: daftar temuan kerentanan terstruktur (`rule_id`, nama *file*, baris, deskripsi, jenis kerentanan, prioritas, kontrol ISO awal) sebagai objek Python *in-memory*, diteruskan ke AI Engine dan ISO Mapper.

### 3.3.2 Converter (converter.py)

Modul ini menggunakan Rector untuk transformasi kode PHP berbasis AST. Argumen CLI `-target-php` menentukan versi target; nilai yang diterima adalah `"7.4"`, `"8.0"`, `"8.1"`, `"8.2"`, atau `"8.3"`. Sebelum Rector dijalankan, seluruh *file* PHP dari `input/` disalin dahulu ke `output/`. Rector kemudian memodifikasi *file* di `output/` langsung tanpa membuat salinan baru. *File* yang gagal dikonversi tetap ada di `output/` sebagai salinan kode asli dan statusnya dicatat `FAILED` di laporan; direktori `input/` tidak dimodifikasi sama sekali. Konfigurasi Rector dibuat sebagai *tempfile* saat *runtime* dan dihapus otomatis setelahnya.

- **Input**: direktori `input/` berisi *file* PHP sumber dan versi target `-target-php`.
- **Output**: direktori `output/` berisi *file* PHP hasil transformasi (atau salinan asli untuk *file* yang gagal/dilewati), beserta status per *file* (`SUCCESS`, `SKIPPED`, atau `FAILED`) yang diteruskan ke laporan akhir.

Tabel 3.1. Pemetaan argumen `-target-php` ke konfigurasi Rector

| Nilai `-target-php` | Chain *LevelSetList* yang diterapkan | Cakupan *rule* kumulatif |
|---|---|---|
| `7.4` | `UP_TO_PHP_53` s.d. `UP_TO_PHP_74` | PHP 5.3, 5.4, 5.5, 5.6, 7.0, 7.1, 7.2, 7.3, 7.4 |
| `8.0` | `UP_TO_PHP_53` s.d. `UP_TO_PHP_80` | Semua *rule* 7.4 + PHP 8.0 |
| `8.1` | `UP_TO_PHP_53` s.d. `UP_TO_PHP_81` | Semua *rule* 8.0 + PHP 8.1 |
| `8.2` | `UP_TO_PHP_53` s.d. `UP_TO_PHP_82` | Semua *rule* 8.1 + PHP 8.2 |
| `8.3` | `UP_TO_PHP_53` s.d. `UP_TO_PHP_83` | Semua *rule* 8.2 + PHP 8.3 |

Target PHP 8.3 digunakan dalam penelitian ini. Selain *level sets* di atas, *pipeline* juga menambahkan `SetList::CODE_QUALITY`, `DEAD_CODE`, `EARLY_RETURN`, dan `TYPE_DECLARATION` untuk semua nilai target. Mengingat *ruleset* yang digunakan bersifat kumulatif, kode PHP 5.x dapat diproses langsung menuju PHP 8.3 dalam satu langkah. Jenis-jenis transformasi yang ditangani Rector dan keterbatasan Rector terhadap pola semantik seperti `mysql_*()` telah diuraikan pada Subbab 2.2.7.

### 3.3.3 Analyzer (analyzer.py)

Setelah konversi, Analyzer menjalankan PHPStan pada direktori `output/` untuk memverifikasi kompatibilitas kode dengan versi PHP target. Level 5 dipilih sebagai keseimbangan antara keketatan dan jumlah *false positive* yang wajar untuk kode warisan. Setiap temuan PHPStan dikategorikan sebagai ERROR atau WARNING, lalu dikaitkan dengan kontrol ISO/IEC 27001:2022 yang relevan.

- **Input**: direktori `output/` hasil konversi Rector.
- **Output**: daftar temuan PHPStan (kategori `ERROR`/`WARNING`) yang masing-masing dikaitkan dengan kontrol ISO/IEC 27001:2022, diteruskan ke ISO Mapper.

### 3.3.4 AI Engine (ai_engine.py)

Modul ini menerima temuan dari Semgrep *pre-scan* sebagai masukan, bukan dari PHPStan dan bukan dari *post-scan*. Temuan dengan prioritas 1–5 dikirim ke AI Engine: SQL Injection (prioritas 1), XSS (prioritas 2), fungsi usang (prioritas 3), *hardcoded credentials* (prioritas 4), serta *path traversal* dan SSRF (prioritas 5). Temuan yang tidak terklasifikasi (prioritas 99) tidak diproses. Prioritas ditentukan berdasarkan potensi dampak langsung terhadap kerahasiaan dan integritas sistem serta frekuensi kemunculannya pada aplikasi PHP warisan menurut OWASP Top 10.

Untuk setiap temuan yang lolos filter, modul mengirimkan konteks dan potongan kode ke model LLM melalui endpoint `/api/chat` Ollama di `localhost:11434`. Jika model tidak menghasilkan blok kode PHP sama sekali, respons tetap dicatat menggunakan *placeholder* `// Manual review required` sebagai penanda untuk tinjauan lanjutan. Jika Ollama tidak tersedia, *pipeline* tetap berjalan tanpa komponen AI. Desain *prompt* yang digunakan dibahas dalam Subbab 3.5.

- **Input**: temuan Semgrep *pre-scan* dengan prioritas 1–5 beserta potongan kode sumber terkait (maksimum 2.000 karakter pertama).
- **Output**: penjelasan kerentanan dalam bahasa Inggris, saran kode perbaikan PHP 8.x (atau *placeholder* `// Manual review required` jika tidak ada blok kode yang berhasil diekstraksi), dan skor kepercayaan `CONFIDENCE`, diteruskan ke ISO Mapper dan laporan akhir.

### 3.3.5 ISO Mapper (iso_mapper.py)

Modul ini menerima semua hasil dari Scanner, Analyzer, dan AI Engine sebagai objek Python *in-memory* langsung dari `main.py`, lalu memetakannya ke kontrol ISO/IEC 27001:2022 secara berbasis aturan. Tidak ada pembacaan *file* JSON. Pemetaan berbasis aturan dipilih agar hasilnya deterministik dan dapat diaudit. Detail pemetaan

dibahas dalam Subbab 3.6.

- **Input**: objek Python *in-memory* dari Scanner (temuan Semgrep *pre-* dan *post-scan*), Analyzer (temuan PHPStan), dan AI Engine (saran perbaikan).
- **Output**: status kepatuhan (`COMPLIANT`, `PARTIAL`, atau `NON_COMPLIANT`) untuk masing-masing dari enam kontrol ISO/IEC 27001:2022 yang dipetakan, diteruskan ke `main.py`.

### 3.3.6 Main (main.py)

Modul `main.py` bertindak sebagai orkestrator yang mengontrol seluruh *pipeline*. Menerima direktori *input* dan versi PHP target sebagai argumen CLI, menjalankan keenam modul secara berurutan, dan menghasilkan laporan JSON di direktori `reports/`. Ringkasan hasil ditampilkan di konsol menggunakan pustaka Rich.

- **Input**: direktori *input* dan versi PHP target sebagai argumen CLI.
- **Output**: laporan JSON terstruktur di direktori `reports/` yang menggabungkan hasil seluruh modul, serta ringkasan yang ditampilkan di konsol.

Contoh kerangka laporan JSON yang dihasilkan:

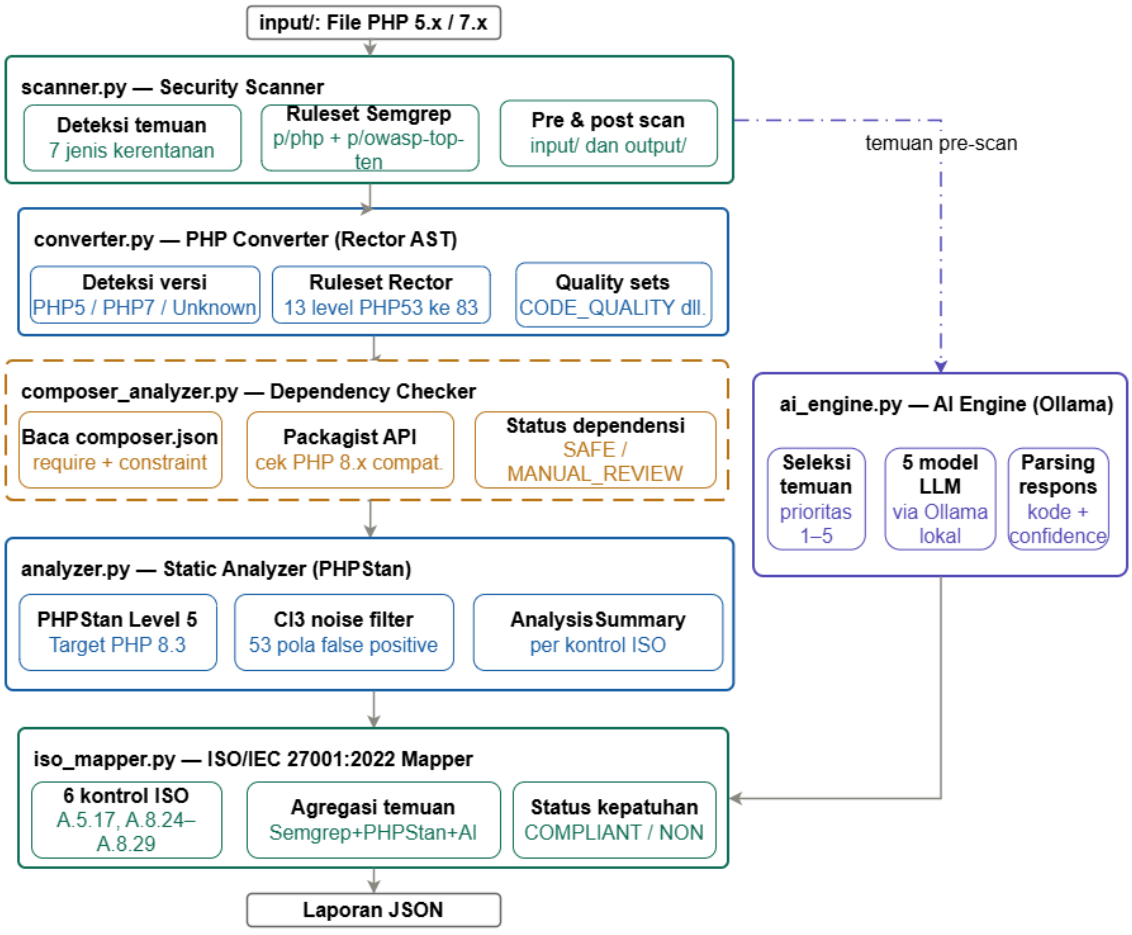
```
{
  "overall_status": "NON_COMPLIANT",
  "duration_seconds": 228.4,
  "conversion": {
    "total_files": 245,
    "converted": 239,
    "conversion_rate": 97.6
  },
  "iso_controls": {
    "A.5.17": {"status": "COMPLIANT", "findings": 0},
    "A.8.28": {"status": "NON_COMPLIANT", "findings": 2790}
  },
  "semgrep": {
    "pre_scan_findings": 11,
    "post_scan_findings": 11,
    "delta_f": 0
  }
}
```

### 3.3.7 Composer Analyzer (composer_analyzer.py)

Modul ini membaca berkas `composer.json` pada direktori *root* repositori dan memeriksa kompatibilitas setiap dependensi terhadap PHP 8.x menggunakan data dari Packagist. Jika berkas `composer.json` tidak ditemukan, modul dilewati secara otomatis tanpa menghentikan *pipeline*. Setiap dependensi diklasifikasikan sebagai `SAFE` (versi yang dikonfigurasi mendukung PHP 8.x secara deklaratif) atau `MANUAL REVIEW` (kompatibilitas tidak dapat diverifikasi secara otomatis).

- **Input**: berkas `composer.json` pada direktori *root* repositori (jika tersedia).
- **Output**: klasifikasi setiap dependensi (`SAFE` atau `MANUAL REVIEW`) terhadap kompatibilitas PHP 8.x, diteruskan ke laporan akhir.

## 3.4 Alur Data Pipeline




Gambar 3.3. Alur data *pipeline*: tahap masukan: Scanner, Converter, dan Composer Analyzer

*File* PHP ditempatkan di `input/`. Scanner menjalankan Semgrep pada `input/` untuk menetapkan *baseline* keamanan kode asli (*pre-scan*). Converter menyalin semua *file* ke `output/` lalu menjalankan Rector dengan versi target yang ditentukan; `input/`

tidak dimodifikasi sama sekali. Setelah konversi, Scanner menjalankan Semgrep kembali pada `output/` (*post-scan*) untuk mengukur perubahan postur keamanan melalui nilai $\Delta F$. Composer Analyzer memeriksa kompatibilitas dependensi `composer.json` terhadap PHP 8.x, atau dilewati otomatis jika berkas tidak ditemukan.

Selanjutnya, Analyzer menjalankan PHPStan level 5 pada `output/` untuk memverifikasi kompatibilitas kode dengan versi PHP target. Sementara itu, AI Engine mengambil temuan Semgrep prioritas 1–5 dan mengirimkannya ke model LLM untuk analisis dan saran perbaikan; modul ini dilewati otomatis jika tidak ada temuan. Pada tahap akhir, ISO Mapper mengumpulkan semua temuan dari Scanner, Analyzer, dan AI Engine lalu memetakannya ke enam kontrol ISO/IEC 27001:2022. Sebagai penutup, `main.py` menggabungkan seluruh data tersebut menjadi laporan JSON dan menampilkan ringkasannya di konsol.

## 3.5 Desain Prompt LLM

Desain *prompt* adalah salah satu keputusan metodologis yang paling berpengaruh terhadap kualitas respons model. *Prompt* dirancang untuk memenuhi tiga tujuan: menghasilkan kode perbaikan yang valid secara sintaksis, menghasilkan skor kepercayaan yang dapat dibaca otomatis oleh *pipeline*, dan dapat dijalankan secara *zero-shot* tanpa contoh mengingat tidak tersedianya *dataset* berlabel.

*Prompt* yang dijelaskan pada subbab ini adalah *prompt* standar Kondisi A yang digunakan secara identik untuk semua model.

### 3.5.1 Struktur Prompt

*Prompt* terdiri dari dua bagian yang dikirim melalui endpoint `/api/chat`: *system message* dan *user message*.

*System message*:

```
1 You are a PHP security expert and migration specialist.
2 You analyze PHP code for security vulnerabilities and provide
3 secure PHP 8.x replacements.
4 Always follow output format instructions exactly as given.
```

*User message* untuk setiap temuan:

```
TASK: Analyze the following PHP code for a [vulnerability_type]
vulnerability and provide a secure PHP 8.x replacement.
ISO/IEC 27001:2022 Controls: [relevant_controls]
Context: File: [filename], line [line_number],
         Semgrep rule: [rule_id], Finding: [description]
VULNERABLE CODE:
```php
[code_snippet]
```
Respond in English:
1. Explain the vulnerability and its security risk.
2. Provide the corrected PHP 8.x code in a ```php code block.
After your explanation and code fix, write exactly this on the
last line:
CONFIDENCE: [number between 0 and 100]
Example last line: CONFIDENCE: 85
```

Potongan kode yang disisipkan pada *placeholder* `[code_snippet]` dibatasi pada 2.000 karakter pertama dari *file* sumber. Pembatasan ini diterapkan untuk menjaga agar ukuran *prompt* tetap dalam batas *context window* default Ollama dan menghindari pemotongan respons akibat token yang melebihi kapasitas model. Konsekuensinya, untuk *file* PHP yang panjang, model hanya menerima bagian awal kode dan tidak selalu memiliki konteks lengkap untuk menganalisis kerentanan secara menyeluruh. Keterbatasan ini dibahas lebih lanjut dalam Subbab 4.1.12.3. Sebagai ilustrasi, berikut contoh pengisian *placeholder* di atas untuk temuan SQL Injection pada `Output.php` baris 768 (*rule*: `tainted-sql-string`, kontrol A.8.26 dan A.8.28):

````
TASK: Analyze the following PHP code for a SQL Injection
vulnerability and provide a secure PHP 8.x replacement.
ISO/IEC 27001:2022 Controls: A.8.26, A.8.28
Context: File: Output.php, line 768,
         Semgrep rule: tainted-sql-string,
         Finding: $_SERVER['QUERY_STRING'] used directly in
                  cache path construction without sanitization
VULNERABLE CODE:
```php
$uri .= '?'.$_SERVER['QUERY_STRING'];
$cache_path .= md5(
    $CI->config->item('base_url')
    .$CI->config->item('index_page')
    .ltrim($uri, '/'));

if ( ! @unlink($cache_path)) {
    log_message('error',
        'Unable to delete cache file for '.$uri);
    return FALSE;
}
```
Respond in English:
1. Explain the vulnerability and its security risk.
2. Provide the corrected PHP 8.x code in a ```php code block.
After your explanation and code fix, write exactly this on the
last line:
CONFIDENCE: [number between 0 and 100]
Example last line: CONFIDENCE: 85
````

Blok kode PHP diekstraksi dari respons model menggunakan pencocokan pola *markdown* "`php ... "`. Jika pola ini tidak ditemukan, *pipeline* mencoba mengekstraksi blok kode generik "` ... "` sebagai *fallback*. Kode hasil ekstraksi kemudian divalidasi menggunakan `php -l` untuk memverifikasi kebenaran sintaksisnya sebelum disertakan dalam laporan. Jika tidak ada blok kode yang berhasil diekstraksi sama sekali, respons tetap dicatat dengan *placeholder* `// Manual review required`.

Contoh keluaran yang diharapkan dari model untuk temuan di atas:

```
### Explanation of Vulnerability
The code uses $_SERVER['QUERY_STRING'] directly in cache
path construction without sanitization, allowing path
traversal or cache poisoning attacks.

Here's the corrected PHP 8.x code:
```php
$uri .= '?' . urlencode($_SERVER['QUERY_STRING']);
$cache_path .= md5(
    $CI->config->item('base_url')
    . $CI->config->item('index_page')
    . ltrim($uri, '/'));

if (!@unlink($cache_path)) {
    log_message('error',
        'Unable to delete cache file for '
        . urlencode($uri));
    return FALSE;
}
```
CONFIDENCE: 78
```

Tiga elemen yang harus hadir sekaligus agar respons dihitung patuh format: penjelasan kerentanan, blok kode PHP yang valid, dan baris `CONFIDENCE:` di akhir.

Teks lengkap *prompt* Kondisi B untuk masing-masing model disertakan pada Lampiran 5.2.

### 3.5.2 Keputusan Desain Prompt

**Penyertaan kontrol ISO/IEC 27001:2022:** Kontrol ISO yang relevan disertakan di setiap *prompt*. Informasi ini bertujuan untuk mendorong model menghasilkan saran perbaikan yang mempertimbangkan kepatuhan standar, bukan sekadar memperbaiki sintaks.

**Instruksi format di akhir pesan:** Format `CONFIDENCE: [0-100]` diletakkan di bagian akhir dengan instruksi eksplisit termasuk contoh konkret. Model kecil cenderung lebih patuh pada instruksi yang dekat dengan akhir pesan.

**Parsing skor berlapis:** Karena tidak semua model mengikuti format yang diminta, `ai_engine.py` menggunakan enam pola *regex* berurutan untuk mengekstraksi skor: (0) tag XML `<confidence>`, (1) bilangan bulat dekat kata

"confidence", (2) desimal 0.x dekat "confidence", (3) persentase dekat "confidence", (4) desimal *bare* di mana saja, dan (5) frasa bahasa alami seperti "not sure" yang dipetakan ke nilai tetap. Jika seluruh pola gagal diekstraksi, sistem akan mengembalikan nilai *default* 0,0.

**Temperature 0,1:** Dipilih untuk meminimalkan variasi acak antar percobaan. Setiap model dijalankan tiga kali per temuan dengan *prompt* identik, sehingga perbedaan hasil antar *run* mencerminkan variabilitas model yang sebenarnya. Ollama tidak menyediakan pengaturan *seed* inferensi yang konsisten untuk seluruh model yang diuji, sehingga variasi kecil antar *run* tetap mungkin terjadi meskipun *temperature* dibuat rendah (0,1).

**Zero-shot tanpa contoh:** Tidak ada contoh perbaikan kode disertakan karena penelitian ingin mengevaluasi kemampuan bawaan model dalam kondisi yang mencerminkan penggunaan nyata. Hal ini juga memastikan bahwa seluruh model dibandingkan pada kondisi yang setara.

**Skor kepercayaan sebagai metrik sekunder:** Skor ini adalah penilaian mandiri dari model dan tidak bisa diverifikasi secara independen. Dalam analisis Bab IV, skor kepercayaan adalah metrik sekunder; metrik primer adalah validitas sintaks kode yang dihasilkan, diukur dengan `php -l`.

### 3.6 Desain Pemetaan ISO/IEC 27001:2022

Pemetaan temuan ke kontrol ISO/IEC 27001:2022 dilakukan secara berbasis aturan (*rule-based*) di dalam `iso_mapper.py`. Data diterima langsung sebagai objek Python *in-memory* dari `main.py`, tidak dibaca dari *file* JSON. Pendekatan berbasis aturan ini dipilih agar hasilnya deterministik dan bisa diaudit, berbeda dari pendekatan berbasis AI yang hasilnya bisa berubah antar inferensi. Ada enam kontrol yang dipetakan, sebagaimana ditunjukkan pada Tabel 3.2.

Status akhir setiap kontrol ditentukan berdasarkan ada tidaknya temuan terbuka setelah *pipeline* selesai. `COMPLIANT` berarti tidak ada temuan yang dipetakan ke kontrol tersebut. `NON_COMPLIANT` berarti ada temuan Semgrep aktif. `PARTIAL` berarti hanya ada temuan PHPStan tanpa temuan Semgrep. Status COMPLIANT pada A.5.17 dan A.8.24 tidak menjamin bahwa kode telah terbebas dari kerentanan secara mutlak; status tersebut sekadar menunjukkan bahwa *ruleset* yang digunakan tidak menghasilkan temuan yang dipetakan ke kedua kontrol itu pada dataset ini.

Tabel 3.2. Pemetaan sumber temuan ke kontrol ISO/IEC 27001:2022 Annex A

| **Kontrol ISO** | **Judul** | **Sumber temuan yang dipetakan** |
|---|---|---|
| A.5.17 | *Authentication Information* | Semgrep: *hardcoded credentials* |
| A.8.24 | *Use of Cryptography* | Semgrep: `md5()`/`sha1()` untuk kata sandi |
| A.8.25 | *Secure Development Lifecycle* | Semgrep (semua) + PHPStan ERROR/WARNING |
| A.8.26 | *Application Security Requirements* | Semgrep: SQL Injection, XSS |
| A.8.28 | *Secure Coding* | Semgrep (semua) + PHPStan ERROR |
| A.8.29 | *Security Testing in Development* | Semgrep: *Path Traversal*/SSRF + PHPStan ERROR/WARNING |

### 3.6.1 Mekanisme Klasifikasi Temuan

Pemetaan dari hasil temuan Semgrep ke kontrol ISO dilakukan dengan fungsi `_classify()` di `scanner.py`. Fungsi ini memeriksa `rule_id` dan `message` dari setiap temuan Semgrep menggunakan pencocokan *substring* (bukan ekspresi reguler) terhadap daftar kata kunci yang telah ditetapkan. Hasilnya adalah penugasan `vuln_type`, daftar `iso_controls`, dan nilai prioritas. Tabel 3.3 menunjukkan pemetaan kata kunci ke jenis kerentanan dan kontrol ISO yang ditugaskan.

Pendekatan *keyword matching* ini dipilih dibandingkan pendekatan berbasis AI karena tiga alasan. Pertama, hasilnya deterministik: temuan yang sama selalu menghasilkan pemetaan yang sama tanpa variasi antar inferensi. Kedua, pendekatan ini sepenuhnya dapat diaudit karena seluruh logika pemetaannya tersimpan dalam bentuk kode Python yang mudah diperiksa dan diverifikasi. Ketiga, tidak memerlukan koneksi eksternal atau model tambahan, sehingga konsisten dengan persyaratan privasi data yang ditetapkan dalam Subbab 1.5.

Tabel 3.3. Pemetaan kata kunci `rule_id` Semgrep ke jenis kerentanan dan kontrol ISO/IEC 27001:2022

| **Kata Kunci (`rule_id`/`message`)** | **Jenis Kerentanan** | **Kontrol ISO** | **Prioritas** |
|---|---|---|---|
| `sql`, `sqli`, `injection` | SQL Injection | A.8.26, A.8.28 | 1 |
| `xss`, `cross-site` | XSS | A.8.26, A.8.28 | 2 |
| `deprecated`, `mysql_` | Fungsi Usang | A.8.25, A.8.28 | 3 |
| `hardcoded`, `secret`, `password` | Kredensial Hardcoded | A.5.17, A.8.28 | 4 |
| `path`, `traversal` | Path Traversal | A.8.28, A.8.29 | 5 |
| `ssrf`, `server-side-request-forgery` | SSRF | A.8.28, A.8.29 | 5 |
| `md5`, `sha1`, `weak-crypto`, `weak-hash` | Kriptografi Lemah | A.8.24, A.8.28 | 6 |

## 3.7 Rancangan Eksperimen Komparasi Model LLM

Eksperimen komparasi dirancang dengan kondisi yang dikontrol ketat dan dilakukan terpisah dari pengujian *pipeline* utama menggunakan skrip evaluasi khusus. Pemisahan ini dilakukan untuk menjaga validitas pengujian; performa Rector dan Semgrep bersifat deterministik, sedangkan performa LLM bersifat stokastik. Jika perbandingan model dijalankan sebagai bagian dari eksekusi *pipeline* penuh, variasi pada tahap-tahap sebelumnya (misalnya jumlah dan urutan temuan yang berbeda antar dataset atau antar versi target) akan menjadi perancu (*confounding variable*) yang mengaburkan perbandingan antar model. Dengan menetapkan 11 temuan Semgrep *pre-scan* dataset FT UGM yang identik sebagai *input* tetap untuk seluruh model dan kondisi eksperimen, perbedaan hasil yang terukur dapat diatribusikan pada perbedaan kemampuan model, bukan pada variasi rangkaian *pipeline* di tahap lain.

### 3.7.1 Baseline Pembanding

Penelitian ini tidak memiliki *baseline* eksternal dalam arti tidak ada alat atau penelitian terdahulu yang menjalankan tugas yang identik – menerjemahkan temuan Semgrep pada kode PHP menjadi penjelasan dan saran remediasi menggunakan LLM lokal – sehingga evaluasi dilakukan menggunakan mekanisme pembanding internal melalui dua kondisi. Pertama, **Kondisi A** berfungsi sebagai *baseline zero-shot* yang sama untuk kelima model, sehingga Kondisi B dan Kondisi C dapat diukur sebagai peningkatan relatif terhadap kondisi awal yang setara, bukan terhadap titik nol yang

sewenang-wenang. Kedua, perbandingan lintas-model itu sendiri berfungsi sebagai *baseline* relatif: pertanyaan yang dijawab bukan "apakah pendekatan ini efektif secara absolut", melainkan "model mana, di antara kandidat yang memenuhi batasan privasi data dan perangkat keras pada Subbab 1.5, yang paling sesuai untuk peran ini". Ketiadaan *baseline* eksternal ini diakui sebagai keterbatasan penelitian dan dibahas lebih lanjut pada Bab V.

Eksperimen ini mencakup tiga kondisi yang dirancang secara bertahap. **Kondisi A** adalah *baseline*: semua model menerima *prompt* yang identik secara *zero-shot* tanpa optimasi apa pun. **Kondisi B** mengoptimasi *prompt* per model berdasarkan pola kegagalan yang terobservasi pada Kondisi A, untuk mengetahui sejauh mana performa model bisa ditingkatkan melalui penyesuaian *prompt*. **Kondisi C** mengeksplorasi variasi parameter *temperature* dan *context window* pada model terbaik hasil Kondisi A dan B.

### 3.7.2 Kondisi Eksperimen

- **Input**: 11 temuan Semgrep dari *pre-scan* dataset FT UGM
- **Jumlah percobaan**: 3 *run* per temuan per model (33 percobaan per model, 165 total)

  **Temperature**: 0,1 untuk Kondisi A dan B guna meminimalkan variasi acak, serta nilai yang divariasikan secara spesifik pada tahap pengujian Kondisi C.

  **Prompt**: Identik untuk semua model pada Kondisi A (*zero-shot*); dioptimasi per model pada Kondisi B; dan dimodifikasi parameter konteksnya pada Kondisi C.
- **Model**: dijalankan lokal melalui Ollama tanpa modifikasi

### 3.7.3 Metrik Evaluasi

Definisi formal seluruh metrik yang digunakan dalam penelitian ini beserta landasannya dalam literatur telah diuraikan dalam Subbab 2.2.11. Secara ringkas, lima metrik yang diterapkan adalah: Conversion Rate (CR) untuk mengukur keberhasilan konversi Rector, Format Compliance Rate (FCR) untuk mengukur kepatuhan format respons model, Syntax Validity Rate (SVR) sebagai metrik primer kualitas kode yang dihasilkan, Security Finding Delta ($\Delta F$) untuk mengukur perubahan postur keamanan pasca-konversi, dan Mean Confidence Score ($\bar{C}_m$) sebagai metrik sekunder konsistensi model.

Dalam eksperimen ini, setiap model dijalankan sebanyak 33 percobaan (3 *run* per temuan, 11 temuan), sehingga $r_{total} = 33$ per model untuk perhitungan FCR. SVR ditetapkan sebagai metrik primer karena hasilnya dapat diverifikasi secara independen menggunakan `php -l`, sedangkan $\bar{C}_m$ bersifat sekunder karena merupakan penilaian mandiri model yang tidak dapat diverifikasi dari luar.

### 3.7.4 Interpretasi *Trade-off* Antar Metrik

FCR dan SVR merupakan dua dimensi yang independen dan perlu diinterpretasikan bersama, sebagaimana dijelaskan dalam Subbab 2.2.11. Model dengan FCR tinggi namun SVR rendah menunjukkan kemampuan mengikuti instruksi format tetapi kualitas kode yang buruk. Sebaliknya, model dengan FCR rendah namun SVR tinggi mengabaikan instruksi format tetapi menghasilkan kode berkualitas baik. Kedua kondisi ini punya implikasi praktis yang berbeda dan dibahas secara empiris di Bab IV.

## 3.8 Rancangan Eksperimen Validasi Efektivitas *AI Engine*

Eksperimen komparasi model pada Subbab sebelumnya menentukan model LLM terbaik untuk menjalankan peran *AI Engine*. Eksperimen ini dirancang untuk menjawab Rumusan Masalah ke-4, yaitu memvalidasi nilai tambah yang diberikan oleh AI Engine dibandingkan dengan sistem tanpa komponen AI.

### 3.8.1 Definisi Kondisi B0 dan T1

Dua kondisi keluaran *pipeline* dibandingkan:

- **B0** (*baseline* tanpa AI): keluaran *pipeline* tanpa komponen *AI Engine* aktif. Setiap temuan hanya mengandung informasi yang dihasilkan Semgrep secara langsung: `rule_id`, nama *file*, nomor baris, tingkat keparahan (*severity*), potongan kode, dan kontrol ISO yang dipetakan secara berbasis aturan.
- **T1** (*treatment* dengan AI): keluaran *pipeline* lengkap dengan *AI Engine* aktif menggunakan model Qwen2.5-Coder 7b. Setiap temuan dilengkapi dengan penjelasan kerentanan dalam bahasa natural dan saran kode remediasi PHP 8.x yang dihasilkan model.

### 3.8.2 Dataset

Dataset yang digunakan adalah 11 temuan Semgrep *pre-scan* dari dataset FT UGM (FT-01 hingga FT-11): 1 kasus SQL Injection dan 10 kasus SSRF yang berasal dari `Output.php`. Dataset DTI UGM dikecualikan karena karakteristik temuannya identik dengan FT UGM (sumber *file* dan jenis kerentanan yang sama), sehingga tidak menambah keberagaman data untuk eksperimen ini.

### 3.8.3 Desain *Blind Rating*

Untuk menghindari bias penilai terhadap keluaran yang diketahui berasal dari AI, penilaian dilakukan secara buta (*blind*). Setiap temuan disajikan dalam dua paket tanpa label asal:

- **Paket A** dan **Paket B** untuk setiap temuan, di mana salah satu paket berisi keluaran B0 dan satu lagi berisi keluaran T1. Penugasan A/B diacak menggunakan *seed* = 42.
- Dokumen penilaian tidak mengandung istilah B0, T1, *AI Engine*, nama model, atau informasi lain yang dapat mengungkap asal paket kepada penilai.
- Paket yang berisi keluaran T1 memiliki bagian "Catatan Tambahan" berisi penjelasan dan saran remediasi; penilai diberitahu bahwa *beberapa* paket mungkin memiliki bagian ini.

Penilaian dilakukan oleh dua *rater* independen dengan latar belakang keamanan siber yang tidak familiar dengan *pipeline* penelitian ini, untuk meminimalkan bias konfirmasi.

### 3.8.4 Instrumen Penilaian

Setiap paket dinilai pada empat dimensi menggunakan skala Likert 7 poin [30] sebagaimana diperkenalkan pada Subbab 2.2.14:

1. **Akurasi**: sejauh mana identifikasi kerentanan yang disajikan tepat dan tidak menyesatkan.
2. **Kejelasan**: sejauh mana informasi yang disajikan mudah dipahami oleh pengembang.
3. **Actionability**: sejauh mana informasi yang disajikan dapat langsung ditindaklanjuti untuk perbaikan kode.
4. **Ketepatan pemetaan ISO**: sejauh mana kontrol ISO/IEC 27001:2022 yang disertakan relevan dengan temuan.

### 3.8.5 Metode Analisis Statistik

Analisis statistik menggunakan *Wilcoxon signed-rank test* [31] untuk membandingkan skor B0 dan T1 pada pasangan temuan yang sama, sebagaimana diperkenalkan pada Subbab 2.2.14. Uji ini dipilih karena data berskala ordinal (Likert 7 poin) dan ukuran sampel kecil tidak memenuhi asumsi normalitas untuk uji parametrik.

Uji Wilcoxon diterapkan pada 10 temuan berpasangan (FT-02 hingga FT-11, $n = 10$). Temuan FT-01 (SQL Injection) dianalisis secara deskriptif saja karena $n = 1$ tidak mencukupi untuk uji nonparametrik. Konsistensi antar-*rater* diukur menggunakan *Cohen's kappa* [32] sebelum hasil penilaian digabungkan untuk analisis utama.

## 3.9 Metode Pengujian dan Evaluasi

Pengujian dilakukan dengan pendekatan berbasis keluaran (*output-oriented evaluation*), dengan fokus pada hasil setiap modul dibandingkan dengan detail implementasi internalnya. Terdapat lima metrik utama:

1. **Tingkat konversi Rector**: Persentase *file* PHP yang berhasil dikonversi tanpa kesalahan fatal Rector, dihitung per *file* dan secara keseluruhan.
2. **Validitas sintaks pasca-konversi**: Persentase *file* hasil konversi yang lolos `php -l`. Diukur secara objektif. Validitas sintaks tidak menjamin kebenaran logika program atau keamanan hasil transformasi, tetapi dipilih sebagai metrik primer karena dapat diverifikasi secara objektif dan otomatis.
3. **Perbedaan temuan keamanan**: Selisih jumlah temuan Semgrep antara *pre-scan* dan *post-scan*. Temuan baru setelah konversi tidak otomatis diinterpretasikan sebagai regresi keamanan, tetapi hal tersebut dapat mencerminkan kerentanan laten yang sebelumnya tersembunyi.
4. **Performa model LLM**: Diukur menggunakan *format compliance rate* (sekunder) dan *syntax validity rate* (primer) sebagaimana dijelaskan dalam Subbab 3.7.3.
5. **Status kepatuhan ISO/IEC 27001:2022**: Status per kontrol (`COMPLIANT`, `PARTIAL`, atau `NON_COMPLIANT`) berdasarkan logika pemetaan dalam Subbab 3.6.

Untuk hasil PHPStan, jumlah kesalahan yang tinggi tidak secara langsung diinterpretasikan sebagai kegagalan *pipeline*. Kesalahan PHPStan pada kode warisan yang baru dimigrasi sebagian besar mencerminkan *technical debt* yang sudah ada sebelum migrasi, bukan akibat dari proses konversi. Pembedaan ini dibahas lebih rinci di Bab IV.

# BAB IV
# HASIL DAN DISKUSI

Bab ini menyajikan hasil implementasi *pipeline* pada dua studi kasus: kode PHP milik FT UGM (Dashboard TCK) dan kode PHP milik DTI UGM (Sistem CBT), beserta analisis perbandingan keduanya. Setiap studi kasus diawali dengan deskripsi dataset yang digunakan, diikuti hasil konversi Rector, pemindaian Semgrep, validasi PHPStan, dan pemetaan kepatuhan ISO/IEC 27001:2022. Studi kasus pertama juga mencakup eksperimen komparasi lima model LLM pada tiga kondisi serta analisis dependensi *composer*. Bagian penutup menyajikan perbandingan hasil kedua studi kasus secara menyeluruh.

## 4.1 Studi Kasus 1: FT UGM (Dashboard TCK)

### 4.1.1 Deskripsi Dataset

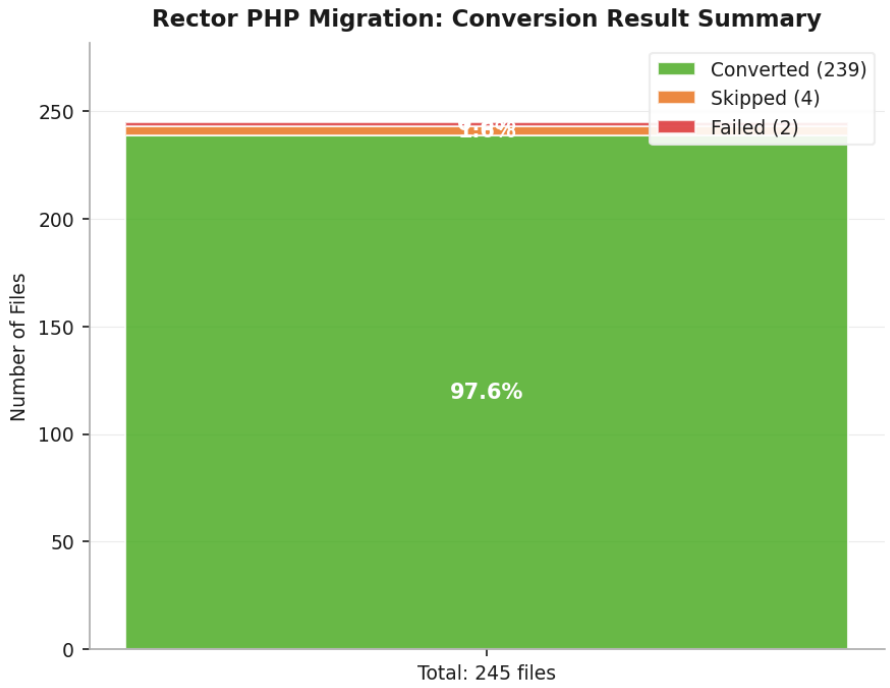
Studi kasus pertama menggunakan 245 *file* PHP dari satu aplikasi web internal berbasis CodeIgniter 3.x yang dikelola oleh FT UGM. Struktur direktorinya mengikuti konvensi standar CodeIgniter 3.x: `application/` untuk kode aplikasi (kontroler, model, *view*, *helper*, konfigurasi, layanan) dan `system/` untuk kode *framework*. Kode diperoleh melalui koordinasi langsung dengan FT UGM dan hanya digunakan untuk analisis statis dalam lingkungan lokal, tanpa akses ke basis data produksi dan tanpa eksekusi kode.

Hanya *file* `.php` yang diproses; *file* lain seperti `.html`, `.js`, dan `.css` tidak disalin ke `output/` dan tidak dianalisis. Dataset mencerminkan karakteristik kode warisan yang umum: pencampuran logika bisnis, akses basis data, dan presentasi dalam *file* yang sama, serta beberapa pola kode PHP 5.x yang masih ada meski aplikasi secara keseluruhan berjalan di atas PHP 7.x.

Untuk eksperimen komparasi LLM, digunakan 11 temuan dari Semgrep *pre-scan*: 1 SQL Injection dan 10 SSRF (*Server-Side Request Forgery*). Hasil ini adalah seluruh populasi temuan *pre-scan*, bukan subset yang dipilih secara subjektif.

### 4.1.2 Hasil Konversi Rector

Rector dijalankan terhadap 245 *file* PHP dari dataset FT UGM dengan target versi PHP 8.3. Dari 245 *file* tersebut, 239 *file* berhasil dikonversi, 4 *file* dilewati (*skipped*), dan 2 *file* gagal diproses. Tingkat konversi keseluruhan ($CR = 97{,}6\%$), sebagaimana ditunjukkan pada Gambar 4.4.




Gambar 4.4. Ringkasan hasil konversi Rector terhadap 245 *file* PHP dataset FT UGM

Karena dataset mengandung campuran kode PHP 5.x dan PHP 7.x, Rector menerapkan rantai *ruleset* kumulatif dari `UP_TO_PHP_53` hingga `UP_TO_PHP_83` secara otomatis dalam satu proses. Hal ini membuktikan bahwa *pipeline* dapat menangani kode dari berbagai era PHP sekaligus tanpa konfigurasi tambahan.

#### 4.1.2.1 Contoh Transformasi Rector

Berikut adalah tiga contoh transformasi yang dilakukan Rector pada dataset FT UGM:

**Transformasi 1: Array sintaks pendek (`ArrayShortSyntaxRector`)**

Pada `system/core/Security.php`, deklarasi *array* menggunakan sintaks lama diubah ke sintaks pendek PHP 5.4:

```
// Sebelum (PHP 7.x)
public $filename_bad_chars = array(
    '../', '<!--', '-->', '<', '>',
    "'", '"', '&', '$', '#',
);
protected $_never_allowed_str = array(
    'document.cookie' => '',
    'document.write'  => '',
);

// Sesudah (PHP 8.3)
public $filename_bad_chars = [
    '../', '<!--', '-->', '<', '>',
    "'", '"', '&', '$', '#',
];
protected $_never_allowed_str = [
    'document.cookie' => '',
    'document.write'  => '',
];
```

**Transformasi 2: Operator *null coalescing* (`NullCoalescingRector`)**

Pada `system/helpers/url_helper.php`, pola pengecekan tiga bagian `isset()` diubah ke operator `??` yang lebih ringkas:

```
// Sebelum (PHP 7.x)
$atts[$key] = isset($attributes[$key])
    ? $attributes[$key] : $val;

// Sesudah (PHP 8.3)
$atts[$key] = $attributes[$key] ?? $val;
```

**Transformasi 3: Fungsi `str_contains()` (`StrContainsRector`)**

Pada `system/helpers/url_helper.php`, pola pencarian substring dengan `strpos()` diubah ke fungsi `str_contains()` yang dikenalkan di PHP 8.0. Rector juga menambahkan *cast* `(string)` karena hasil dari `$_SERVER['SERVER_SOFTWARE']` bertipe `mixed` di PHP 8.x:

```
// Sebelum (PHP 7.x)
if ($method === 'auto'
    && isset($_SERVER['SERVER_SOFTWARE'])
    && strpos($_SERVER['SERVER_SOFTWARE'],
        'Microsoft-IIS') !== FALSE)

// Sesudah (PHP 8.3)
if ($method === 'auto'
    && isset($_SERVER['SERVER_SOFTWARE'])
    && str_contains(
        (string) $_SERVER['SERVER_SOFTWARE'],
        'Microsoft-IIS'))
```

Ketiga transformasi ini mencerminkan kategori perubahan yang paling umum dilakukan Rector: modernisasi sintaks, penggunaan API PHP yang lebih baru, dan penambahan informasi tipe yang dibutuhkan PHP 8.x.

#### 4.1.2.2 File yang Gagal Dikonversi

Kedua *file* yang gagal dikonversi mengalami *internal error* yang sama, yaitu pada fungsi `FiberNodeScopeResolver::callNodeCallback()` yang menerima argumen bertipe `null`, padahal seharusnya bertipe `PhpParser\Node` . Permasalahan ini merupakan *bug* pada versi Rector yang digunakan, bukan berasal dari kode sumber FT UGM.

*File* pertama, yaitu `captcha_helper.php`, memiliki pola percabangan yang mengembalikan tipe data berbeda, yaitu `GdImage` dan `resource`, bergantung pada hasil evaluasi `function_exists()`. Rector menggunakan *Fiber*, sebuah mekanisme konkurensi ringan yang diperkenalkan di PHP 8.1, sebagai unit eksekusi untuk menganalisis setiap *node* AST secara terpisah selama proses transformasi [2]. Dalam kondisi ini, PHPStan *Fiber* yang digunakan oleh Rector kehilangan referensi terhadap *node* AST ketika menganalisis percabangan dengan perbedaan tipe tersebut.

*File* kedua, yaitu `form_helper.php`, mengandung lebih dari 15 definisi fungsi kondisional yang dibungkus dalam konstruksi `if (!function_exists(...))` pada ruang lingkup *file* yang sama. Banyaknya *fiber* analisis yang berjalan secara paralel untuk setiap fungsi menyebabkan potensi kondisi *race*, di mana salah satu *callback*

menerima *node* yang telah selesai diproses oleh *fiber* lain.

Kedua *file* tersebut tetap disertakan dalam direktori `output/` sebagai salinan kode asli, sehingga proses konversi tetap berjalan tanpa kehilangan data.

Setelah proses konversi, seluruh *file* pada direktori `output/` diuji validitas sintaksisnya menggunakan perintah `php -l`. Hasil pengujian menunjukkan tidak adanya *parse error*, sehingga tingkat validitas sintaks pasca-konversi mencapai 100%.

### 4.1.3 Hasil Pemindaian Keamanan Semgrep

#### 4.1.3.1 Pre-Scan: Baseline Keamanan Kode Asli

Pemindaian Semgrep terhadap 241 *file* PHP pada direktori `input/` (dari total 245 *file*; 4 *file* tidak disertakan karena tidak mengandung kode PHP yang dapat dianalisis oleh *ruleset* Semgrep) menghasilkan 11 temuan, yang terdiri atas 1 kerentanan SQL Injection dan 10 kerentanan SSRF (*Server-Side Request Forgery*). Seluruh temuan tersebut terlokalisasi pada satu *file*, yaitu `Output.php`.

Kerentanan SQL Injection teridentifikasi pada baris 768, di mana data dari variabel `$_SERVER['QUERY_STRING']` digunakan secara langsung dalam konstruksi *cache path* tanpa melalui proses sanitasi yang sesuai. Selain itu, terdapat 10 temuan SSRF yang muncul pada beberapa bagian dalam `Output.php`. Temuan ini disebabkan oleh penggunaan nama *file* yang berasal dari *input* pengguna dalam operasi *file system*, sehingga membuka peluang akses terhadap sumber daya yang tidak diinginkan.

Terpusatnya seluruh temuan pada satu *file* mengindikasikan pola yang umum pada kode warisan, di mana modul yang menangani *output* dan *cache* cenderung menjadi titik kerentanan akibat tingginya interaksi dengan data eksternal yang tidak divalidasi secara konsisten.

#### 4.1.3.2 Post-Scan: Perbandingan Setelah Konversi

Pemindaian Semgrep terhadap direktori `output/` setelah proses konversi oleh Rector menghasilkan jumlah temuan yang identik dengan tahap *pre-scan*, yaitu 11 temuan. Nilai perubahan temuan keamanan ($\Delta F = F_{post} - F_{pre} = 0$) menunjukkan tidak adanya perbedaan. Distribusi jenis kerentanan juga tetap sama, yaitu 1 SQL Injection dan 10 SSRF, seluruhnya berada pada *file* yang sama.

Hasil ini mengindikasikan dua hal: (1) proses konversi Rector tidak memperkenalkan kerentanan baru, dan (2) kerentanan yang ada tidak dapat diatasi melalui transformasi sintaksis berbasis AST karena memerlukan perbaikan tingkat logika (semantik).

Kondisi ini terlihat pada `system/core/Output.php` baris 677. Pada bagian tersebut, Rector mengubah operator logika `OR` menjadi `||` yang secara semantik identik, tetapi operasi `fopen($filepath)` di mana `$filepath` berasal dari data pengguna tetap tidak berubah:

```
// Sebelum konversi (Output.php baris 677)
if ( ! file_exists($filepath)
    OR ! $fp = @fopen($filepath, 'rb'))

// Sesudah konversi (Output.php baris 650)
if ( ! file_exists($filepath)
    || ! $fp = @fopen($filepath, 'rb'))
```

Semgrep tetap menandai baris ini sebagai SSRF pada *post-scan* karena `$filepath` yang dibangun dari `$uri` (data pengguna) masih digunakan sebagai argumen `fopen()` tanpa sanitasi. Perubahan `OR` $\rightarrow$ `||` merupakan transformasi sintaksis murni yang tidak memengaruhi aliran data (*taint flow*) pada kode tersebut. Contoh ini mengilustrasikan secara langsung mengapa $\Delta F = 0$: domain kerja Rector (sintaksis) dan domain deteksi Semgrep (logika keamanan) tidak beririsan.

### 4.1.4 Hasil Validasi PHPStan

PHPStan level 5 dijalankan terhadap kode hasil konversi di direktori `output/` dengan target PHP 8.3. Hasilnya adalah 2.790 temuan yang terdiri dari 2.469 ERROR dan 321 WARNING, tersebar di 179 dari 241 *file* yang dianalisis. Di antara temuan tersebut, 354 di antaranya merupakan *noise* dari arsitektur dinamis CodeIgniter 3 yang tidak dapat dianalisis secara statis oleh PHPStan, meliputi *magic property* dan fungsi *helper* yang diinjeksi saat *runtime*.

Sumber teknis dari *noise* ini berakar pada mekanisme *dependency injection* berbasis *runtime* milik CodeIgniter 3. Berkas `system/core/Controller.php` mendaftarkan seluruh *library* yang dimuat dengan cara berikut:

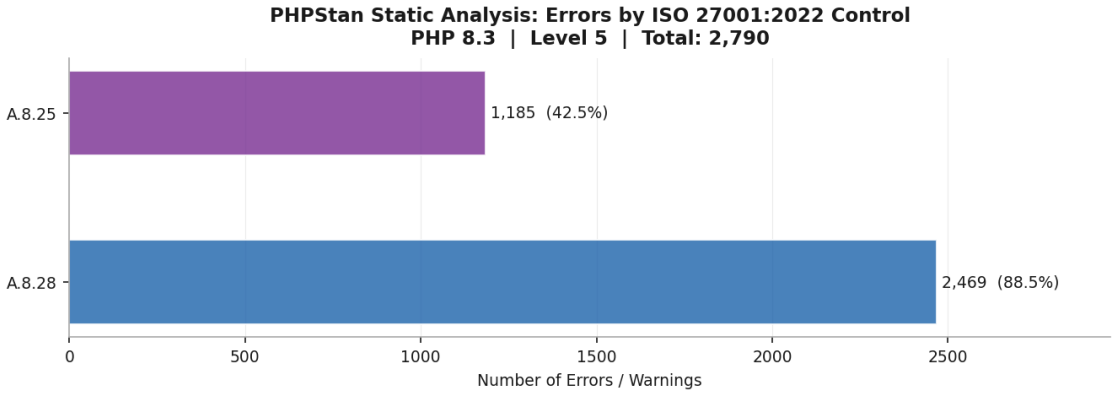
```
// system/core/Controller.php baris 80-83
foreach (is_loaded() as $var => $class)
{
    $this->$var =& load_class($class);
}
```

Akibatnya, properti seperti `$this->db`, `$this->session`, dan `$this->input` tidak pernah dideklarasikan secara eksplisit dalam definisi kelas — keberadaannya hanya terlihat pada saat *runtime*. PHPStan, yang bekerja secara statis, melaporkan setiap akses ke properti tersebut sebagai `"Access to an undefined property"`. Modul

`analyzer.py` mengatasi masalah ini dengan mencocokkan setiap pesan PHPStan terhadap daftar pola `_CI3_NOISE_SUBSTRINGS` yang telah dikurasi, mencakup *substring* seperti `":: $db."`, `":: $lang."`, dan `"Function base_url not found"`. Pada SK1, mekanisme ini menyaring 354 pesan *noise* dari total keluaran PHPStan.

Sebagian besar *error* yang ditemukan PHPStan berasal dari tiga kategori utama. Pertama, kelas dari *library* pihak ketiga yang tidak ditemukan, seperti `PhpOffice\PhpSpreadsheet` dan `Dotenv\Dotenv`: PHPStan tidak dapat menemukan kelas-kelas ini karena hanya kode sumber yang dianalisis, bukan dependensi lengkap aplikasi. Kedua, penggunaan nilai kembalian fungsi `void` sebagai nilai, misalnya `Result of method sendResponse() (void) is used`, yang merupakan pola umum pada kode warisan yang ditulis tanpa deklarasi tipe eksplisit. Ketiga, ketidakkonsistenan tipe seperti `Cannot access property $role on array`, yang mencerminkan penggunaan *array* sebagai pengganti objek bertipe tanpa definisi yang jelas.

Temuan-temuan ini merupakan manifestasi dari *technical debt* (akumulasi masalah kualitas kode yang penanganannya ditunda) yang telah ada sebelum migrasi, bukan kesalahan yang ditimbulkan oleh proses konversi. PHP 8.3 memiliki sistem tipe yang jauh lebih ketat dibandingkan PHP 7.x, sehingga masalah yang sebelumnya tidak terdeteksi karena PHP 7.x lebih permisif kini menjadi terlihat oleh PHPStan. Gambar 4.5 menunjukkan distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 yang dipetakan.




Gambar 4.5. Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A

Tingginya jumlah temuan PHPStan mengonfirmasi fungsi komponen validasi dalam *pipeline* ini. Sejumlah 2.790 potensi masalah tipe dan kompatibilitas ini sebelumnya tidak terdeteksi pada lingkungan PHP 7.x, sesuai dengan temuan Strobl et al. [8] tentang pentingnya jaminan kualitas otomatis dalam transformasi kode skala

besar.

### 4.1.5 Status Kepatuhan ISO/IEC 27001:2022

ISO/IEC 27001:2022 Annex A mendefinisikan 93 kontrol keamanan yang tersebar di empat domain: *Organizational* (37), *People* (8), *Physical* (14), dan *Technological* (34). Mayoritas kontrol bersifat manajerial, prosedural, atau terkait keamanan fisik yang tidak dapat diverifikasi melalui analisis artefak kode sumber. Sebagai contoh, A.6.3 (*Security Awareness*) mengatur pelatihan kesadaran keamanan pegawai dan A.7.1 (*Physical Security Perimeters*) mengatur batas fisik area aman — keduanya tidak memiliki representasi yang dapat diukur dari kode PHP.

Dari 93 kontrol tersebut, *pipeline* memetakan temuan ke enam kontrol yang secara teknis dapat diverifikasi melalui keluaran Semgrep dan PHPStan: empat kontrol kluster *Secure Software Development* (A.8.24–A.8.29) yang secara eksplisit mengatur praktik pengembangan kode aman, serta A.5.17 (*Authentication Information*) yang dipetakan ke deteksi *hardcoded credentials*. Pemilihan ini bukan berarti kontrol lain tidak penting, melainkan bahwa cakupan *pipeline* ini terbatas pada analisis statis kode sumber dan kontrol yang dipilih adalah satu-satunya yang dapat dipetakan secara deterministik ke keluaran alat yang tersedia. Tabel 4.1 menunjukkan status kepatuhan terhadap keenam kontrol tersebut.

Tabel 4.1. Status kepatuhan ISO/IEC 27001:2022 hasil *pipeline* pada dataset FT UGM

| **Kontrol** | **Judul** | **Dasar Penilaian** | **Status** |
|---|---|---|---|
| A.5.17 | *Authentication Information* | Tidak ada temuan *hardcoded credentials* | `COMPLIANT` |
| A.8.24 | *Use of Cryptography* | Tidak ada temuan penggunaan `md5()`/`sha1()` untuk kata sandi | `COMPLIANT` |
| A.8.25 | *Secure Development Lifecycle* | 1.185 temuan PHPStan kategori *undefined/not found* yang dipetakan ke kontrol ini | `NON_COMPLIANT` |
| A.8.26 | *Application Security Requirements* | 1 temuan SQL Injection dari Semgrep | `NON_COMPLIANT` |
| A.8.28 | *Secure Coding* | Temuan Semgrep aktif dan 2.469 ERROR PHPStan | `NON_COMPLIANT` |

*(bersambung ke halaman berikutnya)*

*(lanjutan Tabel 4.1)*

| Kontrol | Judul | Dasar Penilaian | Status |
|---|---|---|---|
| A.8.29 | *Security Testing in Development* | 10 temuan SSRF dari Semgrep masih terbuka | NON_COMPLIANT |

Status keseluruhan adalah NON_COMPLIANT. Angka temuan per kontrol pada tabel ini dapat melebihi total temuan unik (2.790) karena satu temuan PHPStan dapat dipetakan ke lebih dari satu kontrol ISO sekaligus. Hasil ini sejalan dengan kondisi kode warisan yang dianalisis, serta secara objektif merepresentasikan postur keamanan aplikasi sebelum tahap remediasi manual.

Dua kontrol yang berstatus COMPLIANT, yaitu A.5.17 dan A.8.24, perlu dipahami dalam konteks yang tepat. Status ini berarti *ruleset* Semgrep yang digunakan tidak menghasilkan temuan yang dipetakan ke kedua kontrol tersebut, bukan berarti kode bebas dari masalah autentikasi atau kriptografi secara absolut.

Kontrol A.8.29 berstatus NON_COMPLIANT karena terdapat 10 temuan SSRF aktif dari Semgrep yang dipetakan ke kontrol ini dan belum diatasi. Berdasarkan mekanisme pemetaan berbasis aturan yang didefinisikan dalam Subbab 3.6, keberadaan temuan Semgrep aktif pada kontrol mana pun menghasilkan status NON_COMPLIANT, terlepas dari apakah *pipeline* sudah menjalankan proses pengujian keamanan otomatis atau belum. Laporan kepatuhan ini dihasilkan secara otomatis dan dapat langsung digunakan sebagai dokumentasi audit ISO/IEC 27001:2022 [18].

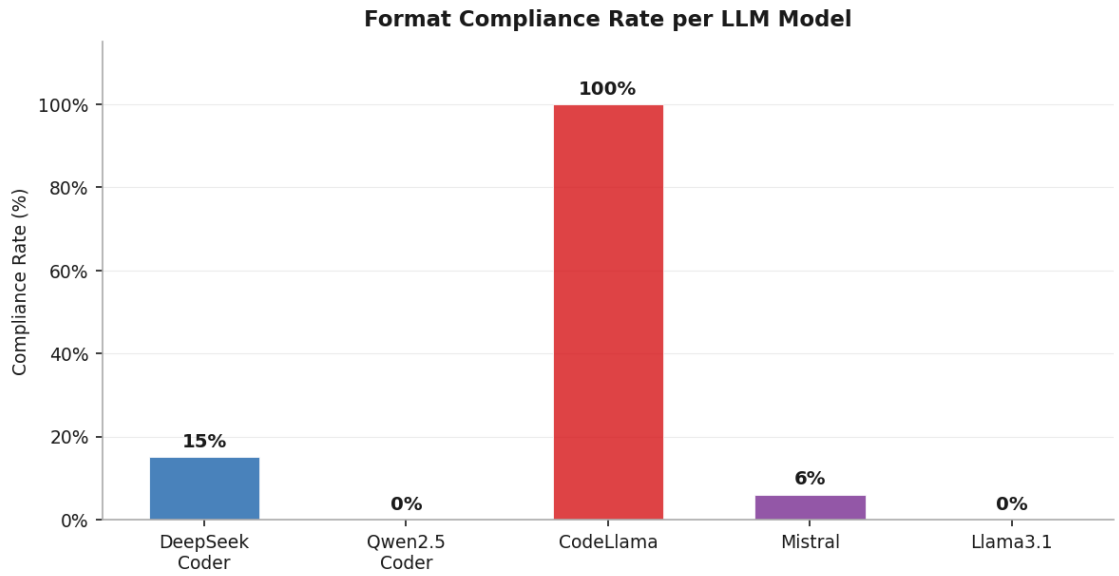
### 4.1.6 Hasil Komparasi Lima Model LLM (Kondisi A)

Eksperimen Kondisi A dijalankan dengan *prompt* yang identik untuk semua model (*zero-shot*), 11 temuan Semgrep *pre-scan* sebagai masukan, 3 *run* per temuan per model, dan *temperature* 0,1. Total percobaan adalah 165 (33 per model). Semua model dijalankan secara lokal melalui Ollama tanpa modifikasi.

#### 4.1.6.1 Format Compliance Rate

Setelah mengidentifikasi kerentanan keamanan melalui Semgrep, *pipeline* memanfaatkan komponen *AI Engine* untuk merumuskan saran perbaikan secara otomatis terhadap setiap temuan. Gambar 4.6 menunjukkan *format compliance rate* untuk masing-masing model.

CodeLlama adalah satu-satunya model yang secara konsisten mengikuti format instruksi *prompt* di semua 33 percobaan (100%). DeepSeek Coder mengikuti format di 5 dari 33 percobaan (15%). Mistral mengikuti format di 2 dari 33 percobaan (6%).

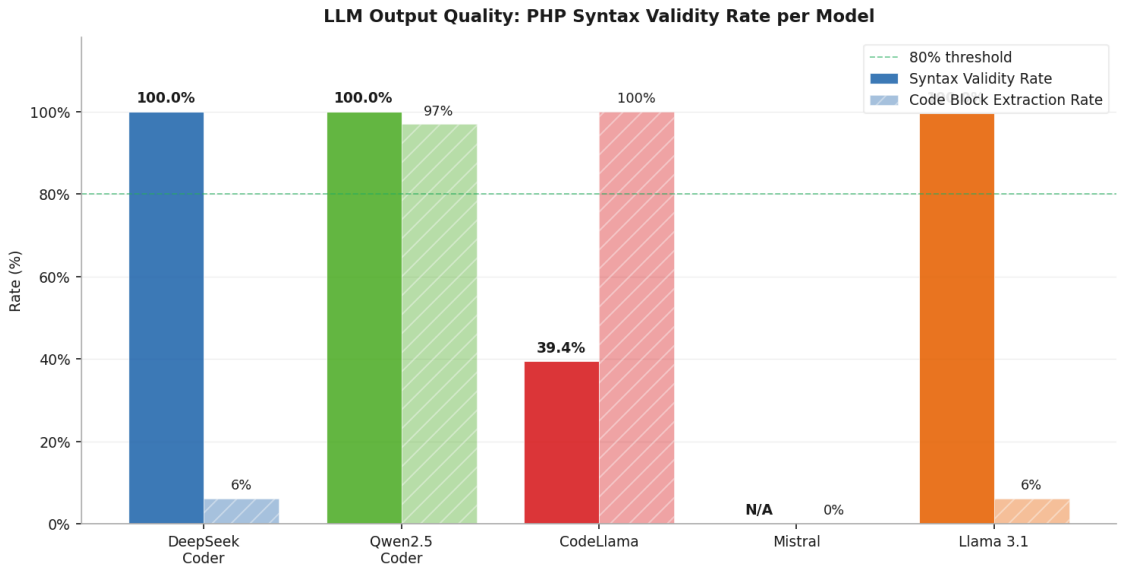



Gambar 4.6. *Format compliance rate* kelima model LLM pada Kondisi A (*zero-shot*, *prompt* identik)

Qwen2.5-Coder dan Llama 3.1 tidak menghasilkan satu pun respons yang memenuhi ketiga elemen format sekaligus.

### 4.1.6.2 Syntax Validity Rate

Gambar 4.7 menunjukkan *syntax validity rate* dan *code block extraction rate* untuk masing-masing model.




Gambar 4.7. *Syntax validity rate* dan *code block extraction rate* kelima model LLM pada Kondisi A

Hasil ini berbanding terbalik dengan metrik kepatuhan format. CodeLlama yang memiliki *format compliance* 100% ternyata hanya menghasilkan kode PHP yang *valid*

secara sintaksis pada 9 dari 33 blok kode yang berhasil diekstraksi ($SVR = 27{,}3\%$). Artinya, 24 dari 33 blok kode CodeLlama mengandung *parse error*. Sebaliknya, Qwen2.5-Coder yang memiliki *format compliance* 0% berhasil mengekstraksi blok kode PHP dari 32 dari 33 percobaan (97%), dan seluruh 32 blok kode tersebut lolos pemeriksaan `php -l` ($SVR = 100\%$). Model ini mengabaikan instruksi format sepenuhnya, tetapi secara konsisten menghasilkan kode yang berkualitas baik.

Mistral berhasil mengekstraksi 1 blok kode dari 33 percobaan, tetapi blok tersebut tidak lolos `php -l` (0% validitas). DeepSeek Coder dan Llama 3.1 masing-masing mengekstraksi 3 dan 1 blok dengan validitas 100%, namun jumlah sampel yang terlalu kecil membuat angka ini tidak representatif secara statistik.

#### 4.1.6.3 Statistik Ringkasan

Tabel 4.2. Statistik lengkap komparasi kelima model LLM pada Kondisi A

| **Model** | **Confidence Score** | | | | **Fmt OK** | **Avg Time (s)** |
|---|---|---|---|---|---|---|
| | **Mean** | **Std Dev** | **Min** | **Max** | | |
| DeepSeek Coder 6.7b | 0,13 | 0,31 | 0,00 | 0,90 | 15% | 4,4 |
| Qwen2.5-Coder 7b | 0,00 | 0,00 | 0,00 | 0,00 | 0% | 10,0 |
| CodeLlama 7b | 0,91 | 0,06 | 0,85 | 1,00 | 100% | 5,0 |
| Mistral 7b | 0,06 | 0,23 | 0,00 | 0,95 | 6% | 7,3 |
| Llama 3.1 8b | 0,00 | 0,00 | 0,00 | 0,00 | 0% | 8,2 |

Statistik berikut dihitung dari 33 inferensi per model (11 temuan $\times$ 3 *run* independen, *temperature* = 0,1).

CodeLlama memiliki *confidence score* rata-rata $\bar{C}_m = 0{,}909$ dengan standar deviasi $\sigma_m = 0{,}054$, mencerminkan konsistensi yang tinggi di rentang sempit $[0{,}85;\ 1{,}00]$. Namun konsistensi ini tidak berkorelasi dengan kualitas kode: 72,7% kode CodeLlama tidak *valid* secara sintaksis, mengkonfirmasi bahwa *confidence score self-reported* bukan ukuran kualitas kode.

DeepSeek Coder mencatat $\bar{C}_m = 0{,}132$ dengan standar deviasi yang jauh lebih besar ($\sigma_m = 0{,}310$) dan rentang $[0{,}00;\ 0{,}90]$. Variansi tinggi ini konsisten dengan *format compliance* 15%: hanya sebagian kecil *run* yang benar-benar menghasilkan baris `CONFIDENCE:`, sementara sisanya menghasilkan nilai 0,0.

Mistral mencatat $\bar{C}_m = 0{,}060$ dengan $\sigma_m = 0{,}230$ dan rentang $[0{,}00;\ 0{,}95]$, pola yang serupa dengan DeepSeek: variansi besar akibat jarangnya respons yang mengikuti format.

Qwen2.5-Coder dan Llama 3.1 keduanya mencatat $\bar{C}_m = 0{,}000$ dengan $\sigma_m =$

0,000. Nilai 0,0 muncul ketika respons tidak mengandung pola apa pun yang dapat di-*parse* oleh keenam pola ekstraksi yang diterapkan secara berurutan — baik baris `CONFIDENCE:`, ekspresi numerik, maupun frasa bahasa alami seperti *"confident"* atau *"not sure"*. Khusus untuk Qwen2.5-Coder, ketiadaan skor ini bukan berarti model tidak mampu, melainkan menunjukkan karakteristik model yang mengabaikan instruksi format sambil tetap mengerjakan tugas utama dengan baik. Waktu inferensi Qwen2.5-Coder (10,0 detik) adalah yang tertinggi dibandingkan model lain, selaras dengan respons yang lebih panjang dan detail yang dihasilkannya.

```
$uri .= '?'.$_SERVER['QUERY_STRING'];
$cache_path .= md5(
    $CI->config->item('base_url')
    .$CI->config->item('index_page')
    .ltrim($uri, '/'));

if ( ! @unlink($cache_path)) {
    log_message('error',
        'Unable to delete cache file for '.$uri);
    return FALSE;
}
```

**Respons CodeLlama:**

````
VULNERABILITY: SQL Injection Vulnerability

The vulnerability is caused by the manual construction
of an SQL string, which allows user-controlled data
to be incorporated into the query. ...

SECURE PHP 8.x REPLACEMENT:
```php
$uri = $_SERVER['QUERY_STRING'];
$cache_path = md5(
    $CI->config->item('base_url')
    .$CI->config->item('index_page')
    .ltrim($uri, '/'));
if ( ! @unlink($cache_path)) {
    log_message('error',
        'Unable to delete cache file for '.$uri);
    return FALSE;
}
% (blok kode tidak ditutup, php -l gagal)
````
````

**Respons Qwen2.5-Coder:**

````
### Explanation of Vulnerability
...
Here's the corrected PHP 8.x code:
```php
$uri .= '?' . urlencode($_SERVER['QUERY_STRING']);
$cache_path .= md5(
    $CI->config->item('base_url')
    . $CI->config->item('index_page')
    . ltrim($uri, '/'));

if (!@unlink($cache_path)) {
    log_message('error',
        'Unable to delete cache file for '
        . urlencode($uri));
    return FALSE;
}
```
````

Perbandingan tersebut menunjukkan perbedaan yang signifikan. CodeLlama mengidentifikasi kerentanan dengan benar tapi solusinya tidak tepat: model hanya mengubah cara penulisan kode tanpa benar-benar mengatasi masalahnya. Kode yang dihasilkan masih menggunakan `$_SERVER['QUERY_STRING']` secara langsung tanpa sanitasi apapun.

Qwen2.5-Coder menunjukkan pemahaman kontekstual yang lebih dalam. Model ini menyadari bahwa meskipun Semgrep menandainya sebagai SQL Injection, kode tersebut sebenarnya membangun *cache path*, bukan kueri SQL. Saran perbaikannya menggunakan `urlencode()` untuk menghindari karakter berbahaya dalam *path*. Secara semantik ini lebih tepat, meski solusi yang ideal tetap perlu ditinjau oleh pengembang yang memahami konteks aplikasi secara menyeluruh.

#### 4.1.6.4 Trade-off dan Implikasinya

Hasil Kondisi A menunjukkan adanya *trade-off* yang jelas antara kepatuhan format dan kualitas kode. Model yang paling patuh pada instruksi format (CodeLlama, 100%) menghasilkan kode dengan tingkat sintaksis *error* tertinggi (72,7% kode tidak valid). Model yang menghasilkan kode paling berkualitas (Qwen2.5-Coder, 97% *extraction rate* dan 100% validitas) mengabaikan instruksi format sepenuhnya.

Temuan ini berimplikasi langsung pada desain sistem berbasis LLM lokal: metrik kualitas kode (*syntax validity rate*) adalah indikator yang jauh lebih relevan dibandingkan

metrik kepatuhan format. Sistem evaluasi yang hanya mengandalkan kepatuhan format akan menghasilkan kesimpulan yang keliru mengenai utilitas praktis suatu model.

*Trade-off* ini juga menunjukkan bahwa kondisi *zero-shot* dengan *prompt* identik mungkin tidak optimal untuk semua model secara bersamaan. Hal ini menjadi motivasi utama untuk Kondisi B, di mana setiap model menerima *prompt* yang dioptimasi berdasarkan karakteristik dan kelemahan yang terobservasi pada Kondisi A.

### 4.1.7 Hasil Komparasi Lima Model LLM (Kondisi B)

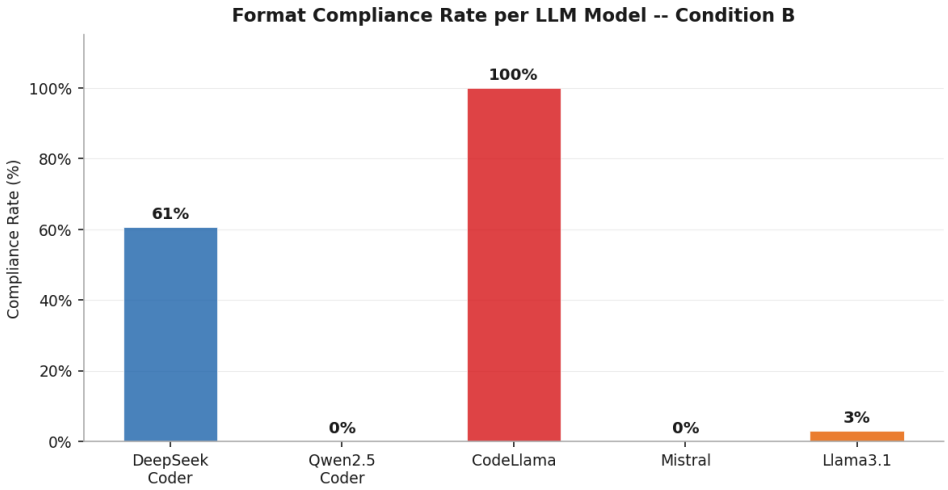
Kondisi B dirancang untuk mengatasi kelemahan spesifik setiap model yang teridentifikasi pada Kondisi A. Setiap model menerima *prompt* yang berbeda melalui metode `_build_optimized_chat_messages()` di `ai_engine.py`, sementara parameter eksperimen lainnya tetap sama: 11 temuan Semgrep yang sama, 3 *run* per temuan per model, dan *temperature* 0,1.

Strategi optimasi per model dirumuskan berdasarkan pola kegagalan Kondisi A. Qwen2.5-Coder menerima *prompt* yang lebih longgar tanpa tuntutan format ketat karena kualitas kodenya sudah baik. DeepSeek Coder menerima instruksi yang lebih pendek dan langsung untuk mengurangi kompleksitas yang diduga menjadi penyebab rendahnya *extraction rate*. Mistral menerima satu contoh *few-shot* konkret untuk menunjukkan format keluaran yang diharapkan. Llama 3.1 menerima instruksi berbasis *role-playing* eksplisit dengan langkah-langkah bernomor. CodeLlama mempertahankan format Kondisi A dengan tambahan instruksi eksplisit agar menghasilkan kode PHP yang *valid* secara sintaksis.

#### 4.1.7.1 Format Compliance Rate

DeepSeek Coder mencatat peningkatan yang paling signifikan, dari 15% pada Kondisi A menjadi 61% pada Kondisi B. Hal ini mengkonfirmasi bahwa model ini sangat sensitif terhadap kompleksitas instruksi: *prompt* yang lebih pendek dan langsung secara substansial meningkatkan kemampuannya mengikuti format. CodeLlama mempertahankan *format compliance* 100% seperti pada Kondisi A. Sebaliknya, Mistral mengalami regresi dari 6% menjadi 0%, dan Llama 3.1 hanya naik tipis dari 0% menjadi 3% (1 dari 33 *run*).

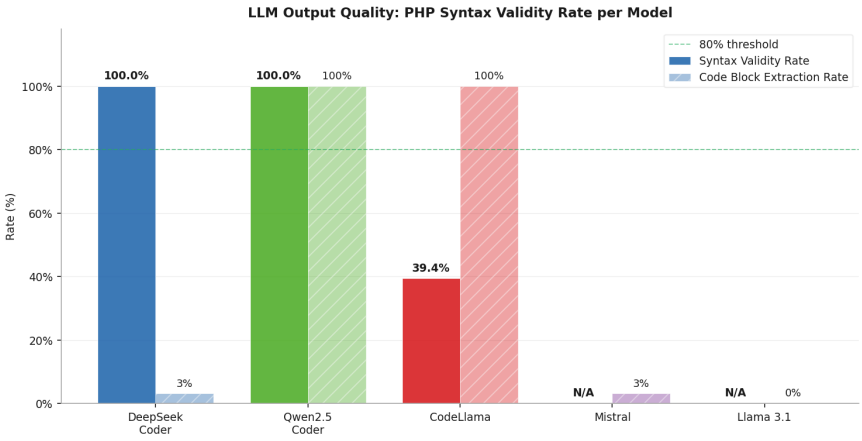
Gambar 4.8 menunjukkan *format compliance rate* untuk masing-masing model pada Kondisi B.




Gambar 4.8. *Format compliance rate* kelima model LLM pada Kondisi B (*prompt* dioptimasi per model)

### 4.1.7.2 Syntax Validity Rate

Gambar 4.9 menunjukkan *syntax validity rate* dan *code block extraction rate* pada Kondisi B.




Gambar 4.9. *Syntax validity rate* dan *code block extraction rate* kelima model LLM pada Kondisi B

Qwen2.5-Coder mempertahankan posisinya sebagai model dengan kualitas kode terbaik: *extraction rate* naik dari 97% menjadi 100% dan *syntax validity* tetap 100%. CodeLlama menunjukkan perbaikan parsial pada *syntax validity*, dari 27,3% menjadi 39,4%, meskipun masih jauh dari ambang batas yang dapat diandalkan. DeepSeek Coder mengalami penurunan *extraction rate* dari 9% menjadi 3% meskipun *format compliance*-nya meningkat, yang menunjukkan adanya *trade-off* antara kepatuhan format dan kemampuan menghasilkan blok kode. Mistral dan Llama 3.1 secara efektif tidak menghasilkan kode yang dapat digunakan pada Kondisi B.

#### 4.1.7.3 Statistik Ringkasan

Tabel 4.3. Statistik lengkap komparasi kelima model LLM pada Kondisi B

| Model | Confidence Score | | | | Fmt OK | Avg Time (s) |
|---|---|---|---|---|---|---|
| | Mean | Std Dev | Min | Max | | |
| DeepSeek Coder 6.7b | 0,59 | 0,48 | 0,00 | 1,00 | 61% | 8,1 |
| Qwen2.5-Coder 7b | 0,00 | 0,00 | 0,00 | 0,00 | 0% | 9,7 |
| CodeLlama 7b | 0,89 | 0,16 | 0,00 | 1,00 | 100% | 5,6 |
| Mistral 7b | 0,00 | 0,00 | 0,00 | 0,00 | 0% | 7,0 |
| Llama 3.1 8b | 0,02 | 0,14 | 0,00 | 0,80 | 3% | 8,3 |

DeepSeek Coder mencatat peningkatan *mean confidence* yang paling mencolok, dari $\bar{C}_m = 0{,}132$ pada Kondisi A menjadi $\bar{C}_m = 0{,}588$ pada Kondisi B. Variansi yang tinggi ($\sigma_m = 0{,}477$) menunjukkan bahwa skor ini tidak stabil: sebagian *run* menghasilkan skor mendekati 1,0 sementara sebagian lainnya tetap di 0,0. Pola ini konsisten dengan *format compliance* 61%: hanya sekitar dua pertiga percobaan yang benar-benar menulis baris `CONFIDENCE:`, sedangkan sisanya tidak sama sekali.

CodeLlama mempertahankan *mean confidence* yang tinggi ($\bar{C}_m = 0{,}894$) dengan standar deviasi yang kecil ($\sigma_m = 0{,}162$), mencerminkan konsistensi yang sama seperti pada Kondisi A. Angka ini mungkin tampak meyakinkan, tetapi perlu diingat bahwa *syntax validity rate* CodeLlama hanya 39,4%: artinya model ini sangat yakin dengan jawabannya meski lebih dari separuh kode yang dihasilkannya tidak lolos `php -l`. Hasil ini kembali mengkonfirmasi bahwa *confidence score self-reported* tidak bisa dijadikan proksi kualitas kode.

Qwen2.5-Coder, Mistral, dan Llama 3.1 mencatat *mean confidence* 0,000 karena tidak satu pun respons mereka yang menulis baris `CONFIDENCE:` dalam format yang dapat di-*parse*. Khusus untuk Qwen2.5-Coder, ketiadaan skor ini justru tidak mencerminkan kualitas yang buruk: model ini tetap menghasilkan kode dengan *syntax validity* 100% di Kondisi B, sama seperti Kondisi A. Ketiadaan skor kepercayaan semata-mata mencerminkan gaya respons model yang mengabaikan instruksi format sambil tetap mengerjakan tugas utamanya dengan baik.

Pola pada Kondisi A kembali terlihat: tidak ada korelasi yang dapat diandalkan antara tingginya *confidence score* dan kualitas kode yang dihasilkan. Hasil ini memperkuat argumen bahwa untuk sistem berbasis LLM lokal berukuran kecil hingga menengah, *syntax validity rate* tetap menjadi metrik evaluasi yang lebih sahih dibandingkan *confidence score self-reported*.

Tabel 4.4. Perbandingan metrik utama Kondisi A vs Kondisi B untuk kelima model LLM

| Model | Format Compliance | | Extraction Rate | | Syntax Validity | |
|---|---|---|---|---|---|---|
| | Kond. A | Kond. B | Kond. A | Kond. B | Kond. A | Kond. B |
| DeepSeek Coder 6.7b | 15% | **61%** | 9% | 3% | 100% | 100% |
| Qwen2.5-Coder 7b | 0% | 0% | 97% | **100%** | 100% | 100% |
| CodeLlama 7b | 100% | 100% | 100% | 100% | 27% | **39%** |
| Mistral 7b | 6% | 0% | 3% | 3% | 0% | 0% |
| Llama 3.1 8b | 0% | 3% | 3% | 0% | 100% | 0% |

### 4.1.8 Perbandingan Kondisi A dan Kondisi B

Terdapat tiga pola utama dari perbandingan tersebut:

**Pertama**, optimasi *prompt* hanya bekerja signifikan pada model yang memiliki kapabilitas *instruction-following* dasar yang memadai. DeepSeek Coder adalah satu-satunya model yang menunjukkan peningkatan substansial pada *format compliance* (dari 15% menjadi 61%), mengkonfirmasi bahwa model ini peka terhadap kompleksitas instruksi: bukan tidak mampu mengikuti format, tapi terbebani oleh instruksi yang terlalu berlapis pada Kondisi A.

**Kedua**, penerapan strategi *few-shot* pada Mistral justru memberikan hasil yang kontradiktif: *format compliance* turun dari 6% menjadi 0%. Fenomena ini, di mana contoh *few-shot* justru mengalihkan model dari mengerjakan tugas aslinya, konsisten dengan temuan dalam literatur yang menunjukkan bahwa teknik *prompt engineering* yang efektif untuk model besar tidak selalu dapat diterapkan langsung pada model berukuran kecil hingga menengah [9].

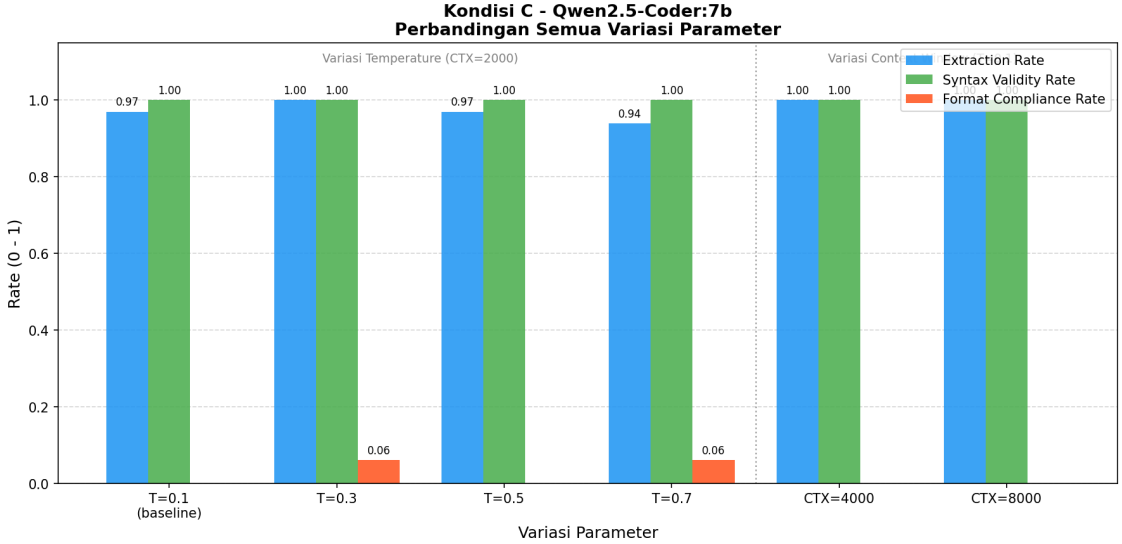
**Ketiga**, keterbatasan CodeLlama dalam menghasilkan kode PHP yang *valid* secara sintaksis bersifat struktural, bukan instruksional. Meskipun instruksi eksplisit untuk memverifikasi sintaks sebelum menulis kode memberikan perbaikan parsial (dari 27% menjadi 39%), sebagian besar kode yang dihasilkan masih mengandung *parse error*. Fakta ini menunjukkan bahwa masalah ini berakar pada representasi internal model terhadap sintaks PHP 8.x, bukan pada cara instruksi diformulasikan.

Hasil Kondisi B semakin mempertegas kesimpulan dari Kondisi A: dalam konteks analisis keamanan kode PHP menggunakan LLM lokal berukuran kecil hingga menengah, Qwen2.5-Coder secara konsisten menghasilkan kode dengan kualitas terbaik pada kedua kondisi tersebut, terlepas dari format *prompt* yang digunakan.

### 4.1.9 Optimasi Parameter Qwen2.5-Coder (Kondisi C)

Setelah Kondisi A dan B mengidentifikasi Qwen2.5-Coder sebagai model dengan kualitas kode terbaik, Kondisi C mengevaluasi apakah performa model ini dapat ditingkatkan lebih lanjut melalui variasi dua parameter inferensi: *temperature* dan *context window*. Eksperimen ini menggunakan input, jumlah *run*, dan *prompt* yang identik dengan Kondisi A, hanya parameter yang diuji yang berbeda.

Gambar 4.10 menunjukkan perbandingan ketiga metrik utama di semua variasi parameter.




Gambar 4.10. Perbandingan metrik Qwen2.5-Coder pada variasi *temperature* dan *context window* (Kondisi C)

#### 4.1.9.1 Pengaruh Temperature

Nilai *temperature* T=0,1 tidak diuji ulang pada Kondisi C karena nilai tersebut telah digunakan sebagai nilai dasar pada Kondisi A; Kondisi C hanya menguji nilai T=0,3 hingga T=0,7 untuk mengamati pengaruh peningkatan *temperature* terhadap kinerja model.

Pada variasi *temperature*, *syntax validity rate* bertahan di 100% di semua nilai yang diuji (0,3 hingga 0,7). Data ini menunjukkan bahwa kemampuan Qwen2.5-Coder menghasilkan kode PHP yang *valid* secara sintaksis tidak terdegradasi meski *temperature* dinaikkan secara signifikan.

*Extraction rate* justru sedikit menurun seiring naiknya *temperature*: dari 100% di T=0,3 menjadi 94% di T=0,7. Hasil ini konsisten dengan ekspektasi teoretis bahwa *temperature* tinggi meningkatkan variabilitas respons, sehingga sebagian kecil respons tidak lagi mengandung blok kode yang dapat diekstraksi.

*Format compliance* memperlihatkan pola yang tidak monoton: muncul di T=0,3 (6%) dan T=0,7 (6%), tetapi kembali ke 0% di T=0,5. Kemunculan *format compliance* yang tidak konsisten ini mengindikasikan bahwa kepatuhan format pada Qwen2.5-Coder lebih dipengaruhi oleh variabilitas stokastik daripada perubahan *temperature* secara sistematis.

#### 4.1.9.2 Pengaruh Context Window

Pada variasi *context window*, baik CTX=4000 maupun CTX=8000 menghasilkan *extraction rate* dan *syntax validity rate* 100%, sedikit di atas baseline CTX=2000 yang memiliki *extraction rate* 97%. *Format compliance* tetap 0% di kedua variasi.

Peningkatan *extraction rate* dari 97% ke 100% pada konteks yang lebih besar mengindikasikan bahwa dengan informasi kode yang lebih lengkap, model lebih konsisten menghasilkan blok kode dalam setiap respons. Namun peningkatan ini marginal dan tidak mengubah karakteristik utama model.

#### 4.1.9.3 Implikasi Kondisi C

Kondisi C menunjukkan bahwa karakteristik Qwen2.5-Coder lebih ditentukan oleh arsitektur modelnya daripada konfigurasi parameter. Kualitas kode yang dihasilkan sangat stabil di semua variasi parameter yang diuji. Dalam penerapan nyata, hal ini berarti operator tidak perlu melakukan penyesuaian parameter yang signifikan untuk mendapatkan keluaran berkualitas dari model ini. Konfigurasi *default* (T=0,1, CTX=2000) sudah memberikan hasil yang hampir optimal, sementara peningkatan *context window* ke 4000 atau 8000 karakter dapat memberikan sedikit keuntungan tambahan pada kode yang lebih panjang.

### 4.1.10 Analisis Dependensi Composer

Modul `composer_analyzer.py` membaca berkas `composer.json` pada direktori *root* repositori FT UGM dan memeriksa kompatibilitas setiap dependensi terhadap PHP 8.x menggunakan data dari Packagist. Terdapat 5 dependensi yang teridentifikasi, dengan hasil sebagaimana ditunjukkan pada Tabel 4.5.

Tiga dari lima dependensi (`phpdotenv`, `phpspreadsheet`, `firebase/php-jwt`) berstatus `SAFE`, artinya versi yang dikonfigurasi di `composer.json` sudah deklaratif mendukung PHP 8.x. Dua dependensi lainnya berstatus `MANUAL REVIEW`: `ngekoding/google-login` merupakan paket dengan aktivitas pemeliharaan rendah sehingga kompatibilitas PHP 8.x-nya tidak dapat diverifikasi secara otomatis, sementara `chriskacerguis/codeigniter-restserver` merupakan ekstensi pihak ketiga untuk CodeIgniter 3.x yang diketahui tidak memiliki dukungan resmi

Tabel 4.5. Status kompatibilitas PHP 8.x dependensi *composer* FT UGM

| Package | Versi | Status |
|---|---|---|
| `vlucas/phpdotenv` | `^5.4` | SAFE |
| `phpoffice/phpspreadsheet` | `^1.12` | SAFE |
| `firebase/php-jwt` | `^6.10` | SAFE |
| `ngekoding/google-login` | `^1.0` | MANUAL REVIEW |
| `chriskacerguis/codeigniter-restserver` | `^3.1` | MANUAL REVIEW |

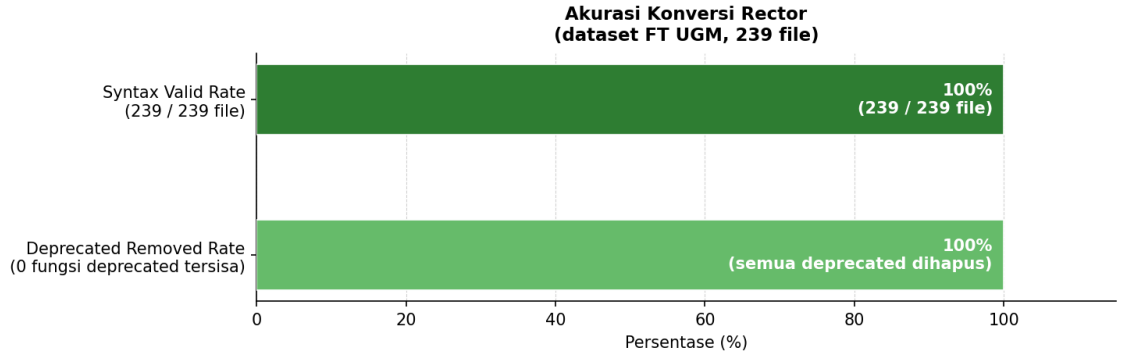
untuk PHP 8.x dan perlu diganti atau diperbarui secara manual sebelum deployment ke lingkungan PHP 8.3.

### 4.1.11 Validasi Keakuratan Hasil Konversi

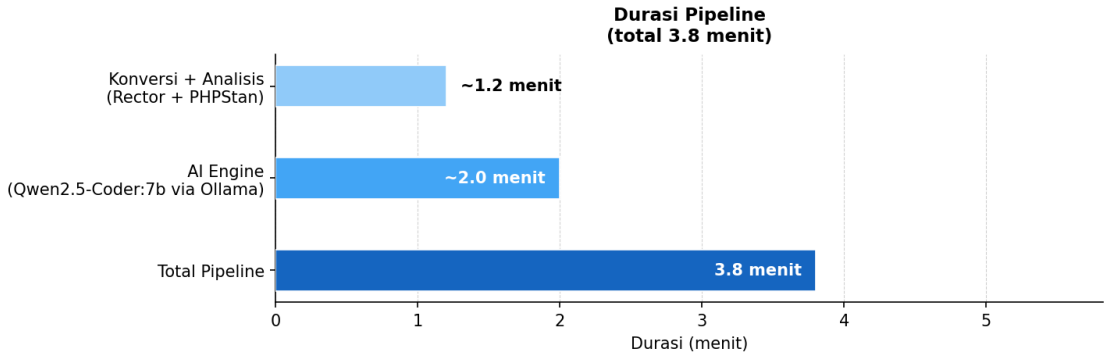
Validasi keakuratan dilakukan melalui dua pendekatan untuk mengukur kualitas hasil konversi Rector pada dataset FT UGM secara objektif. Karena kedua dataset menggunakan CodeIgniter 3 dengan karakteristik serupa, hasil validasi ini merepresentasikan keakuratan konversi pada kedua studi kasus.

#### 4.1.11.1 Metrik Pipeline-Level

Gambar 4.11 menunjukkan dua metrik keakuratan yang diukur langsung dari keluaran *pipeline* terhadap seluruh 241 *file* hasil konversi, sedangkan Gambar 4.12 menunjukkan profil durasi eksekusi tiap tahap.




Gambar 4.11. Akurasi konversi Rector pada dataset FT UGM




Gambar 4.12. Durasi eksekusi *pipeline* pada dataset FT UGM

Pertama, *syntax validity rate* diukur dengan menjalankan `php -l` terhadap seluruh *file* di direktori `output/`. Hasilnya adalah 241 dari 241 *file* lolos pemeriksaan tanpa *parse error* (100%). Hasil ini berarti tidak ada satu pun *file* yang mengalami kerusakan sintaksis akibat proses konversi Rector.

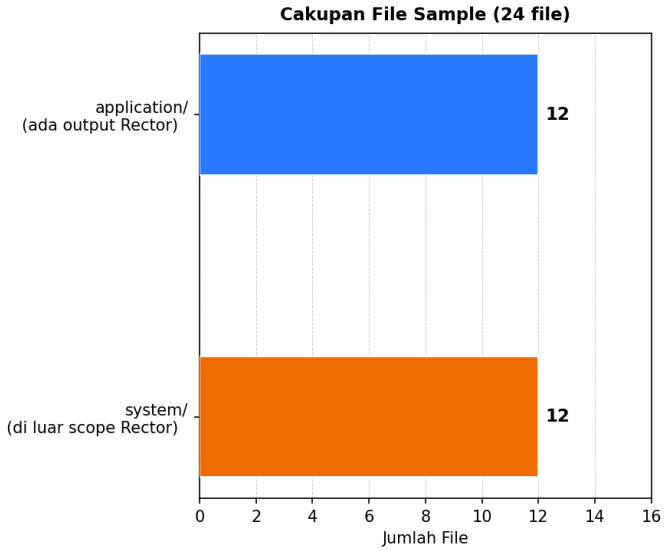
Kedua, *deprecated removed rate* diukur dengan membandingkan temuan Semgrep *pre-scan* dan *post-scan* khusus untuk kategori fungsi *deprecated*. Hasil *post-scan* menunjukkan 0 fungsi *deprecated* tersisa, artinya seluruh pola kode usang yang terdeteksi Semgrep berhasil dihapus atau ditransformasi oleh Rector (100%).

Dari sisi durasi, total eksekusi *pipeline* adalah sekitar 3,8 menit menggunakan Qwen2.5-Coder 7b via Ollama sebagai *AI Engine*. Durasi ini tidak mencakup eksperimen komparasi multi-model (Kondisi A–C) yang merupakan prosedur evaluasi terpisah. Tahap konversi Rector dan analisis PHPStan secara gabungan mendominasi waktu eksekusi.

#### 4.1.11.2 Validasi 24 file

Untuk mengukur seberapa dekat hasil konversi Rector dengan konversi manual yang ideal, dilakukan validasi berbasis *sampling* terhadap 24 *file* dari total *file* valid. Sebanyak 24 *file* dipilih secara acak (seed=42) dari seluruh *file* valid yang lolos pemeriksaan `php -l` menggunakan modul `accuracy_validator.py`, merepresentasikan 24 *file* dari total 241 *file* valid.

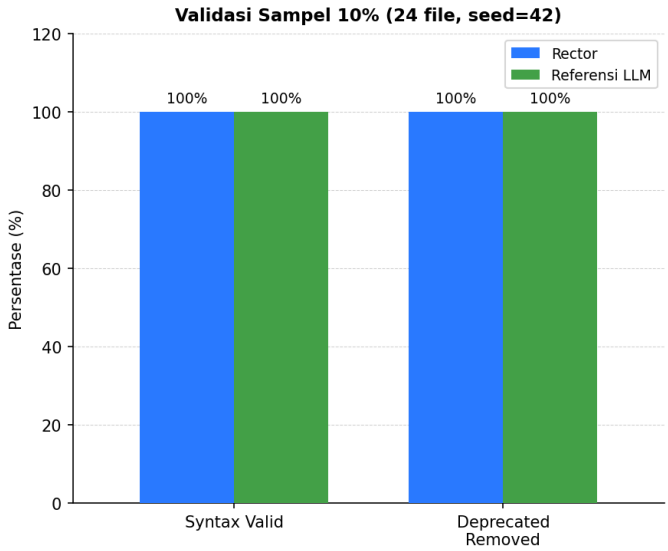
Gambar 4.13 menunjukkan distribusi cakupan 24 *file* sampel tersebut.




Gambar 4.13. Distribusi cakupan *file* sampel validasi 24 file berdasarkan direktori asal

Dari 24 *file* sampel, 12 berasal dari direktori `application/` (kode aplikasi yang diproses Rector) dan 12 berasal dari direktori `system/` (kode *framework* yang juga ikut dikonversi). Karena perbandingan dilakukan antara keluaran Rector dan konversi referensi yang disiapkan secara manual oleh peneliti dengan bantuan model LLM *enterprise* sebagai proksi konversi ideal, hanya 12 *file* dari `application/` yang memiliki padanan keluaran Rector untuk dibandingkan. *File* dari `system/` tetap dicatat dalam sampel untuk transparansi prosedur pengambilan sampel.

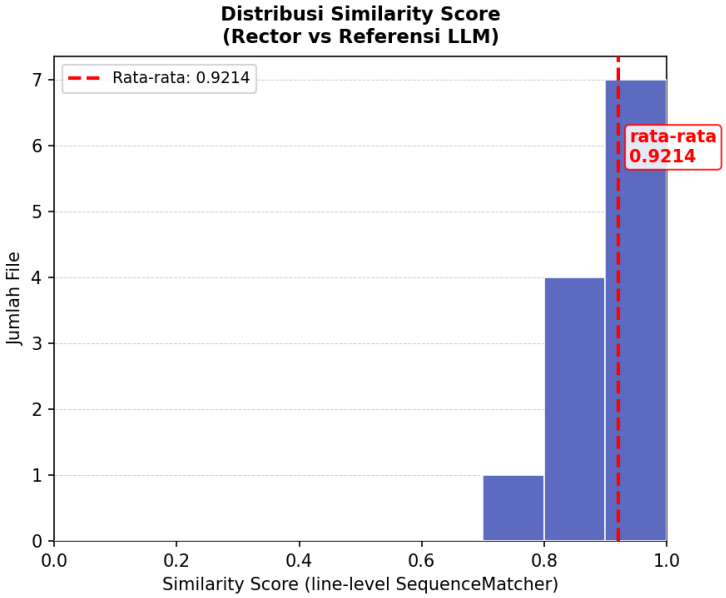
Gambar 4.14 menunjukkan hasil perbandingan antara keluaran Rector dan referensi LLM pada 12 *file* tersebut.




Gambar 4.14. Perbandingan metrik validasi antara keluaran Rector dan referensi LLM pada 12 *file* sampel dari `application/`

Pada kedua metrik biner, keluaran Rector dan referensi LLM mencapai hasil yang identik: *syntax validity* 100% dan *deprecated removed* 100%. Hasil ini mengkonfirmasi bahwa konversi Rector menghasilkan kode yang setara dengan konversi manual dalam hal kebenaran sintaksis dan penghapusan pola usang.

Gambar 4.15 menunjukkan distribusi *similarity score* antara keluaran Rector dan referensi LLM.




Gambar 4.15. Distribusi *similarity score* antara keluaran Rector dan referensi LLM pada 12 *file* sampel (*line-level SequenceMatcher*)

*Similarity score* dihitung menggunakan algoritma Ratcliff/Obershelp melalui mekanisme *line-level* pada `difflib.SequenceMatcher`. Sesuai dengan formula pada Bab 2, yaitu $2M/T$, $M$ adalah jumlah baris yang cocok secara presisi dan $T$ adalah total baris pada kedua teks yang dibandingkan.

Rata-rata *similarity score* yang didapatkan adalah 0,9214, dengan distribusi yang terkonsentrasi di rentang 0,90–1,00 (11 dari 12 *file*). Satu *file* memiliki skor di rentang 0,70–0,80.

Sebagai ilustrasi perhitungan numerik dari formula tersebut, misalkan sebuah *file* hasil konversi Rector memiliki 50 baris dan *file* referensi manual juga memiliki 50 baris, maka total baris yang dievaluasi ($T$) adalah 100. Jika terdapat 46 baris yang identik secara presisi ($M = 46$), maka skor kemiripannya dihitung sebagai:

$$\text{Similarity} = \frac{2 \times 46}{100} = 0{,}92$$

Selisih baris yang tidak cocok (sehingga skor tidak mencapai maksimal 1,0) bukanlah indikasi kesalahan fungsional konversi, melainkan karena sensitivitas metrik

*line-level* terhadap perbedaan gaya penulisan (*formatting*). Sebagai contoh konkret dari perbedaan tersebut, perhatikan perbandingan potongan kode berikut:

```
// Hasil Konversi AST Otomatis (Rector)
$config['base_url'] = '';
$config['index_page'] = 'index.php';

// Referensi Konversi Manual (LLM)
$config['base_url']   = '';
$config['index_page'] = 'index.php';
```

Pada contoh di atas, meskipun logika dan sintaksis keduanya sepenuhnya ekuivalen, penambahan spasi kosong (*whitespace*) yang dilakukan secara manual untuk menyejajarkan tanda sama dengan (=) menyebabkan baris tersebut dievaluasi sebagai baris yang berbeda. Perbedaan kosmetik semacam inilah yang mendasari mengapa skor kemiripan menghasilkan nilai rata-rata di angka 0,9214, alih-alih 1,0.

Skor 0,9214 ini secara objektif mengindikasikan tingkat kesesuaian yang tinggi antara pendekatan otomatis berbasis AST (Rector) dan pendekatan manual berbasis LLM. Perbedaan yang tersisa umumnya berupa variasi kosmetik seperti spasi, urutan atribut, atau gaya penulisan yang sama sekali tidak mempengaruhi kebenaran fungsional kode. Karena kedua dataset menggunakan *framework* CodeIgniter 3.x dengan karakteristik kode yang serupa, hasil validasi ini berlaku pula untuk studi kasus CBT DTI UGM.

### 4.1.12 Diskusi dan Keterbatasan

#### 4.1.12.1 Keterbatasan Dataset

Dataset yang digunakan terdiri dari satu aplikasi berbasis CodeIgniter 3.x milik FT UGM. Dari 245 *file*, 237 diklasifikasikan sebagai *unknown* oleh Rector karena merupakan *file framework* CodeIgniter yang tidak mengandung pola yang perlu ditransformasi. Hal ini membatasi tingkat generalisasi hasil penelitian terhadap jenis aplikasi PHP lainnya, terutama yang tidak menggunakan *framework* atau yang menggunakan *framework* berbeda.

Dari sisi *input* eksperimen LLM, 10 dari 11 temuan adalah SSRF yang berasal dari satu pola kode yang berulang di `Output.php`. Homogenitas *input* ini berarti eksperimen komparasi pada dasarnya menguji respons model terhadap dua jenis kerentanan dengan variasi terbatas. Hasil komparasi perlu diinterpretasikan dalam konteks keterbatasan ini.

#### 4.1.12.2 Keterbatasan Kompatibilitas PHP 8.4 dan PHP 8.5

Eksperimen tambahan dilakukan untuk menguji *pipeline* dengan target PHP 8.5 terhadap dataset CBT DTI UGM. Hasil menunjukkan dua masalah fundamental yang membatasi penerapan otomatis pada versi ini.

Pertama, Rector memperkenalkan *syntax error* pada beberapa berkas inti CodeIgniter 3.x di direktori `system/`, khususnya `CodeIgniter.php`, `Output.php`, dan `Security.php`, karena *ruleset* PHP 8.5 menerapkan transformasi yang tidak kompatibel dengan pola kode *framework* lama. Kedua, akibat berkas `system/` yang rusak pasca-konversi, PHPStan tidak dapat me-*resolve* kelas induk CodeIgniter seperti `CI_Controller` dan `CI_Model`, sehingga menghasilkan ribuan *error* bertipe *unknown class* yang merupakan *false positive* kaskade. Total 6.137 *error* PHPStan yang dihasilkan tidak representatif terhadap kualitas kode aplikasi yang sebenarnya.

Penyebab utama kegagalan ini bukan semata inkompatibilitas CI3 dengan PHP 8.5. Hal ini lebih disebabkan oleh *ruleset* `UP_TO_PHP_85` pada Rector yang masih bersifat eksperimental, sehingga menerapkan transformasi yang terlalu agresif terhadap kode *framework* lama. PHP 8.4 diprediksi lebih *feasible* dibanding PHP 8.5 karena *ruleset*-nya sudah lebih matang dan perubahannya lebih konservatif, namun tetap memerlukan perbaikan manual pada pola *dynamic property* CodeIgniter 3.x yang sudah sepenuhnya dihapus di PHP 8.4. PHP 8.3 tetap merupakan target migrasi otomatis yang paling realistis untuk basis kode CodeIgniter 3.x yang masih aktif digunakan.

Pendekatan yang paling memungkinkan menuju PHP 8.5 adalah pembaruan *framework* ke CodeIgniter 4 atau *framework* modern lainnya yang sudah mendukung PHP 8.x secara *native*, sebuah proses yang melampaui cakupan migrasi versi otomatis dan pada dasarnya merupakan penulisan ulang aplikasi. Oleh karena itu, PHP 8.3 ditetapkan sebagai versi target dalam penelitian ini sebagai pilihan yang dapat dicapai secara otomatis untuk aplikasi berbasis CodeIgniter 3.x.

#### 4.1.12.3 Keterbatasan Teknis Pipeline

*AI Engine* hanya memproses temuan dengan prioritas 1–5 dan membatasi potongan kode yang dikirim ke model pada 2.000 karakter pertama. Untuk *file* PHP yang panjang, konteks yang diterima model terpotong sehingga saran perbaikan mungkin tidak mempertimbangkan keseluruhan logika fungsi. Meskipun Qwen2.5-Coder terbukti mampu membuat inferensi yang tepat dengan konteks terbatas, akurasi ini tidak selalu dapat dijamin untuk setiap kasus.

Selain itu, validasi menggunakan `php -l` hanya memeriksa kebenaran sintaksis, bukan kebenaran semantik. Kode yang lolos `php -l` bisa saja masih mengandung kerentanan yang tidak tertangani atau logika yang tidak ekuivalen dengan kode aslinya. Pengujian fungsional terhadap kode hasil perbaikan AI bukan bagian dari cakupan penelitian ini.

Proses validasi 24 file menggunakan model LLM *enterprise* sebagai referensi konversi ideal. Tahap ini merupakan prosedur evaluasi peneliti yang dilakukan sekali dan tidak melibatkan pengiriman kode sumber secara utuh, sehingga berbeda dari komponen *pipeline* produksi yang sepenuhnya lokal. Penggunaan penilai berbasis model eksternal dipilih untuk menjaga independensi evaluator dan menghindari *bias* penilaian mandiri. Meskipun demikian, pendekatan ini idealnya digantikan dengan penilai lokal agar sepenuhnya konsisten dengan prinsip kerahasiaan data yang diterapkan pada komponen lain dalam *pipeline*.

#### 4.1.12.4 Interpretasi Confidence Score

Tidak ada korelasi yang jelas antara *confidence score* yang dilaporkan model dan kualitas kode yang dihasilkan, baik pada Kondisi A maupun Kondisi B. CodeLlama melaporkan kepercayaan diri rata-rata lebih dari 90% di kedua kondisi, tetapi mayoritas kode yang dihasilkannya tidak *valid* secara sintaksis. Qwen2.5-Coder tidak melaporkan skor sama sekali tetapi menghasilkan kode yang hampir selalu berkualitas baik. Temuan ini konsisten dengan literatur yang menunjukkan bahwa model LLM berukuran kecil umumnya tidak terkalibrasi dengan baik dalam menilai *output* mereka sendiri [9] [11]. Dari ketiga metrik yang diukur $FCR$, *extraction rate*, dan $SVR$, hanya $SVR$ yang secara konsisten mencerminkan kegunaan praktis model dalam konteks ini.

## 4.2 Studi Kasus 2: Sistem CBT DTI UGM

### 4.2.1 Deskripsi Dataset

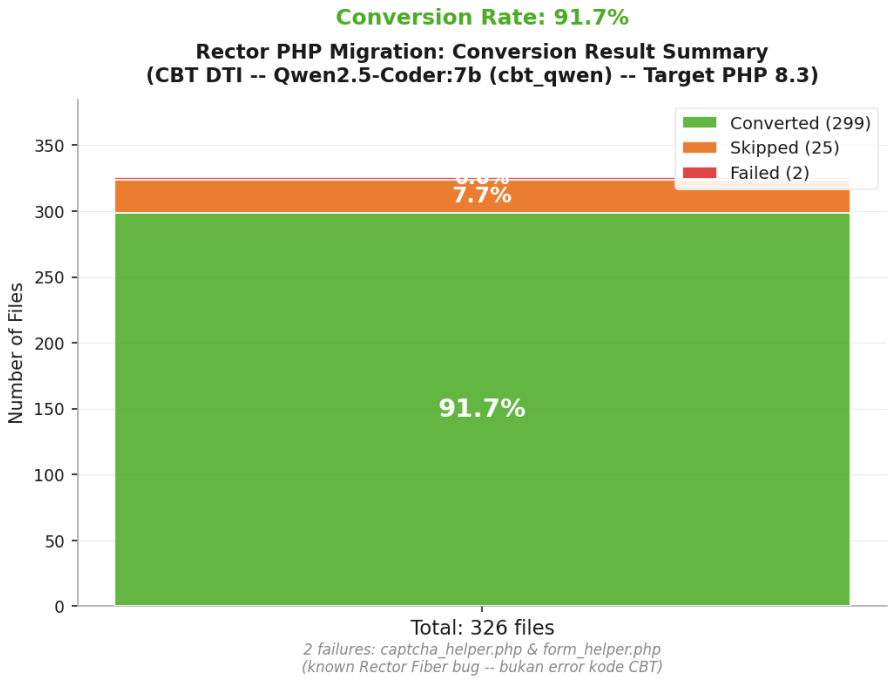
Studi kasus kedua menggunakan kode sumber aplikasi *Computer-Based Test* (CBT) milik Direktorat Teknologi Informasi (DTI) Universitas Gadjah Mada. Aplikasi ini dibangun di atas framework CodeIgniter 3 dengan basis kode PHP 5.x/7.x. *Pipeline* dijalankan terhadap seluruh direktori repositori secara penuh (*full scope*), mencakup direktori `application/` untuk kode aplikasi kustom maupun direktori `system/` untuk kode *framework*, konsisten dengan pendekatan yang diterapkan pada studi kasus FT UGM. Pengecualian hanya diberikan pada subdirektori `libraries/` dan `tcpdf/` yang menyebabkan *memory crash* selama proses analisis. Total *file* PHP yang diproses adalah 326 *file*.

Karena tidak ada berkas `composer.json` pada tingkat aplikasi, langkah analisis dependensi dilewati secara otomatis. Komponen pihak ketiga seperti *library*

TCPDF disertakan langsung sebagai direktori dalam repositori dan telah dieksklusi dari analisis *pipeline*.

### 4.2.2 Hasil Konversi Rector

Rector dijalankan terhadap 326 *file* PHP dengan target versi PHP 8.3. Dari jumlah tersebut, 299 *file* berhasil dikonversi, 25 *file* dilewati (*skipped*), dan 2 *file* gagal diproses. Tingkat konversi keseluruhan ($CR = 91{,}7\%$), sebagaimana ditunjukkan pada Gambar 4.16. Tingkat konversi yang lebih rendah dibandingkan dengan studi kasus FT




Gambar 4.16. Ringkasan hasil konversi Rector terhadap 326 *file* PHP dataset CBT DTI UGM

UGM (91,7% vs 97,6%) disebabkan oleh proporsi *file* yang dilewati yang lebih besar: 25 dari 326 *file* (7,7%) tidak memerlukan transformasi apa pun karena kodenya sudah kompatibel dengan target PHP 8.3 atau tidak mengandung pola yang perlu ditangani oleh *ruleset* Rector. *File* yang dilewati bukan berarti gagal, melainkan sudah memenuhi persyaratan PHP 8.3 tanpa perlu modifikasi.

Seluruh *file* pada direktori `output/` diuji validitas sintaksisnya menggunakan perintah `php -l` dan tidak ditemukan *parse error*, sehingga tingkat validitas sintaks pasca-konversi mencapai 100%.

#### 4.2.2.1 File yang Gagal Dikonversi

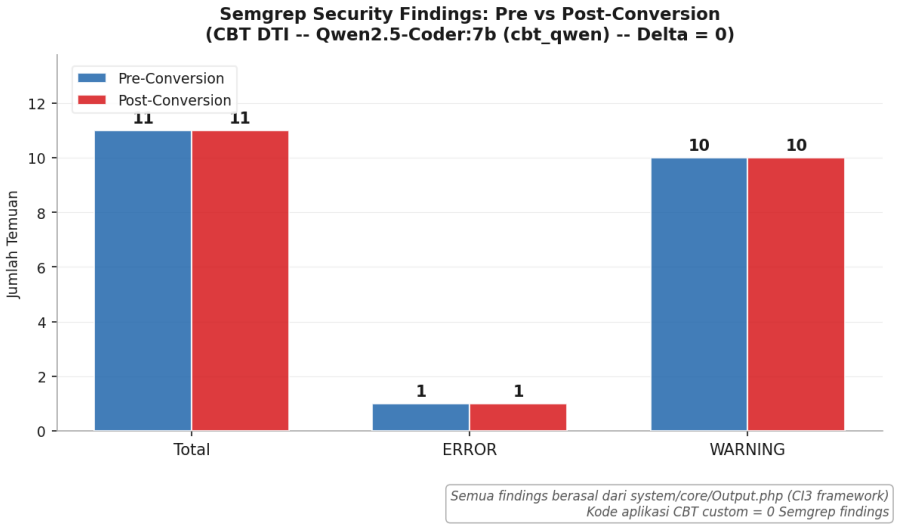
Kedua *file* yang gagal dikonversi adalah `captcha_helper.php` dan `form_helper.php`, mengalami *internal error* yang identik dengan yang

ditemukan pada dataset FT UGM: *bug* `FiberNodeScopeResolver` pada versi Rector yang digunakan, bukan berasal dari kode sumber CBT DTI. Karakteristik kode yang memicu *bug* ini sama persis: `captcha_helper.php` mengandung percabangan dengan perbedaan tipe kembalian, sementara `form_helper.php` memiliki banyak definisi fungsi kondisional yang dibungkus dalam konstruksi `if (!function_exists(...))`. Kedua *file* tetap disalin ke direktori `output/` sebagai salinan kode asli sehingga proses konversi tidak kehilangan data.

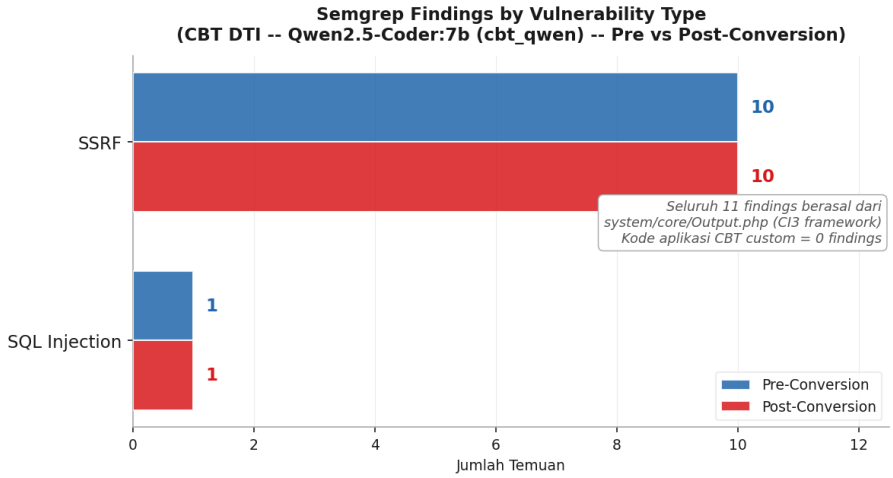
### 4.2.3 Hasil Pemindaian Keamanan Semgrep

#### 4.2.3.1 Pre-Scan: Baseline Keamanan Kode Asli

Pemindaian Semgrep terhadap 326 *file* PHP pada direktori `input/` menghasilkan 11 temuan, yang terdiri atas 1 kerentanan SQL Injection dan 10 kerentanan SSRF (*Server-Side Request Forgery*), sebagaimana ditunjukkan pada Gambar 4.17 dan Gambar 4.18.




Gambar 4.17. Temuan Semgrep *pre-scan* vs *post-scan* berdasarkan tingkat keparahan pada dataset CBT DTI UGM




Gambar 4.18. Temuan Semgrep *pre-scan* vs *post-scan* berdasarkan jenis kerentanan pada dataset CBT DTI UGM

Seluruh 11 temuan tersebut terlokalisasi pada satu *file*, yaitu pada `system/core/Output.php`, yang merupakan komponen inti *framework* CodeIgniter 3. Pola temuan ini identik dengan yang ditemukan pada dataset FT UGM: kerentanan SQL Injection pada baris 768 akibat penggunaan `$_SERVER['QUERY_STRING']` tanpa sanitasi dalam konstruksi *cache path*, serta 10 kerentanan SSRF akibat penggunaan nama *file* dari *input* pengguna dalam operasi *file system*. Kode aplikasi kustom CBT DTI di luar direktori `system/` tidak mengandung temuan Semgrep.

#### 4.2.3.2 *Post-Scan* dan *AI Engine*

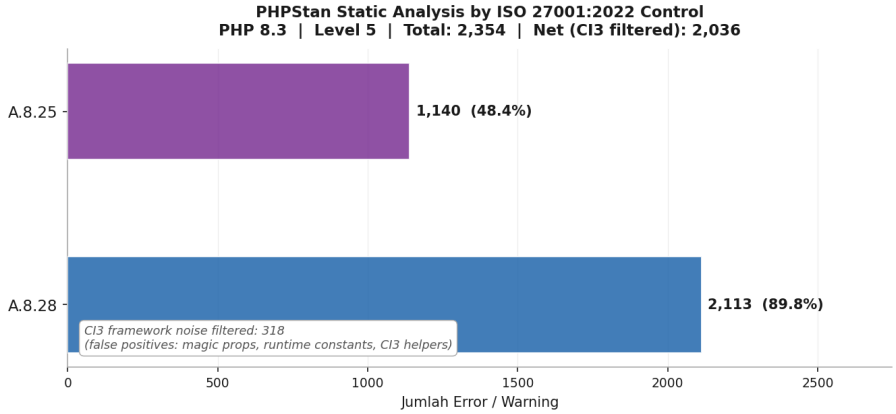
Pemindaian Semgrep terhadap direktori `output/` setelah konversi menghasilkan jumlah temuan yang identik dengan tahap *pre-scan*, yaitu 11 temuan ($\Delta F = F_{post} - F_{pre} = 0$). Distribusi jenis kerentanan juga tetap sama, seluruhnya pada `system/core/Output.php`. Hasil ini menunjukkan bahwa konversi Rector tidak memperkenalkan kerentanan baru maupun menghilangkan kerentanan yang sudah ada, konsisten dengan karakteristik transformasi berbasis AST yang hanya memodifikasi sintaks tanpa mengubah logika program.

Karena terdapat 11 temuan Semgrep, modul *AI Engine* diaktifkan secara otomatis oleh *pipeline*. Model *qwen2.5-coder:7b* melalui Ollama memproses seluruh 11 temuan dan menghasilkan saran perbaikan dengan rata-rata *confidence* 26%. Model dipilih berdasarkan hasil eksperimen komparasi pada studi kasus FT UGM yang mengidentifikasi Qwen2.5-Coder sebagai model dengan kualitas kode terbaik. Satu-satunya pengecualian adalah temuan SQL Injection pada `Output.php` baris 768 yang mendapat *confidence* 0%, karena potongan kode yang dikirim ke model tidak menyertakan konteks yang cukup untuk menganalisis konstruksi internal *framework*. Seluruh saran perbaikan dari *AI Engine* bersifat informatif dan memerlukan tinjauan

manual sebelum diterapkan, karena seluruh temuan berasal dari kode *framework* yang tidak seharusnya dimodifikasi secara langsung.

### 4.2.4 Hasil Validasi PHPStan

PHPStan *level* 5 dijalankan terhadap kode hasil konversi di direktori `output/` dengan target PHP 8.3. Hasilnya adalah 2.354 temuan. Di antara temuan tersebut, 318 merupakan *false positive* dari *framework* CodeIgniter 3 yang timbul karena PHPStan tidak dapat menelusuri *magic property* dan fungsi *helper* yang diinjeksi secara dinamis, konsisten dengan 354 *noise* CI3 yang teridentifikasi pada dataset FT UGM. Distribusi temuan berdasarkan kontrol ISO/IEC 27001:2022 ditunjukkan pada Gambar 4.19.




Gambar 4.19. Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A pada dataset CBT DTI UGM

Sebagaimana pada studi kasus FT UGM, mayoritas temuan PHPStan pada dataset CBT DTI bersumber dari keterbatasan PHPStan dalam memahami arsitektur CodeIgniter 3.x. PHPStan tidak dapat menelusuri dua kategori elemen yang diinjeksi secara dinamis oleh *framework* pada saat *runtime*: (1) *magic property* seperti `$this->db` dan `$this->load` yang berasal dari `CI_Controller` dan `CI_Model`, dan (2) fungsi-fungsi *helper* seperti `base_url()`, `redirect()`, dan `form_open()` yang dimuat secara dinamis. Karena direktori `system/` CodeIgniter dianalisis bersama kode aplikasi, PHPStan menghasilkan sejumlah besar laporan *undefined* untuk elemen-elemen tersebut yang merupakan *false positive* kerangka kerja.

Pola distribusi *error* PHPStan pada CBT DTI menunjukkan konsentrasi tinggi pada kontrol A.8.28 (*Secure Coding*) dengan 2.113 temuan (89,8% dari total), sementara A.8.25 (*Secure Development Lifecycle*) mencatat 1.140 temuan (48,4%). Sebagaimana pada studi kasus pertama, angka PHPStan yang tinggi mencerminkan akumulasi *technical debt* yang sudah ada sebelum migrasi, bukan kesalahan yang ditimbulkan oleh proses konversi.

### 4.2.5 Status Kepatuhan ISO/IEC 27001:2022

Berdasarkan pemetaan yang dilakukan oleh `iso_mapper.py`, status kepatuhan terhadap enam kontrol ISO/IEC 27001:2022 yang dipetakan adalah sebagaimana ditunjukkan pada Tabel 4.6.

Tabel 4.6. Status kepatuhan ISO/IEC 27001:2022 hasil *pipeline* pada dataset CBT DTI UGM

| **Kontrol** | **Judul** | **Dasar Penilaian** | **Status** |
|---|---|---|---|
| A.5.17 | *Authentication Information* | Tidak ada temuan *hardcoded credentials* | `COMPLIANT` |
| A.8.24 | *Use of Cryptography* | Tidak ada temuan penggunaan `md5()`/`sha1()` untuk kata sandi | `COMPLIANT` |
| A.8.25 | *Secure Development Lifecycle* | 1.140 temuan PHPStan (mayoritas *false positive* CI3) | `NON_COMPLIANT` |
| A.8.26 | *Application Security Requirements* | 1 temuan SQL Injection dari Semgrep (`Output.php`, CI3 *framework*) | `NON_COMPLIANT` |
| A.8.28 | *Secure Coding* | 2.113 temuan PHPStan dan temuan Semgrep aktif | `NON_COMPLIANT` |
| A.8.29 | *Security Testing in Development* | 10 temuan SSRF dari Semgrep masih terbuka | `NON_COMPLIANT` |

Status `NON_COMPLIANT` ditemukan pada A.8.26 dan A.8.29 dari `system/core/Output.php` *framework* CodeIgniter 3, bukan dari kode aplikasi kustom CBT DTI. Hasil ini konsisten dengan cakupan analisis *full scope* yang diterapkan pada kedua studi kasus.

### 4.2.6 Analisis Dependensi Composer

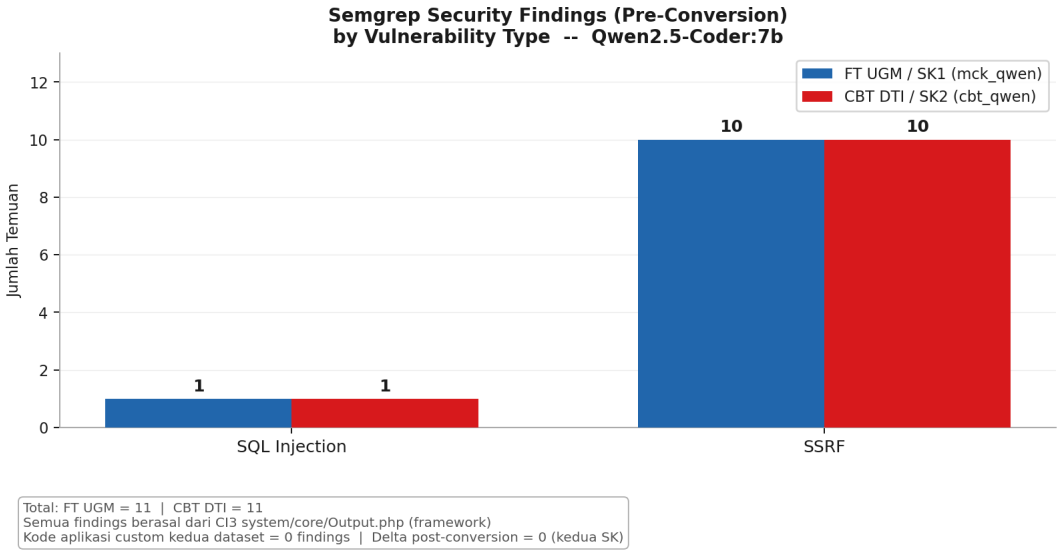
Aplikasi CBT DTI UGM tidak memiliki berkas `composer.json` pada tingkat aplikasi. Komponen pihak ketiga seperti *library* TCPDF disertakan langsung sebagai direktori dalam repositori dan telah dieksklusi dari analisis *pipeline*. Oleh karena itu, modul `composer_analyzer.py` tidak menghasilkan keluaran untuk studi kasus ini dan langkah analisis dependensi dilewati secara otomatis.

## 4.3 Perbandingan Kedua Studi Kasus

Bagian ini membandingkan hasil implementasi *pipeline* pada kedua studi kasus secara berdampingan untuk mengidentifikasi pola yang konsisten maupun perbedaan yang signifikan.

### 4.3.1 Perbandingan Temuan Keamanan Semgrep

Gambar 4.20 menunjukkan perbandingan temuan Semgrep *pre-scan* berdasarkan jenis kerentanan pada kedua dataset.



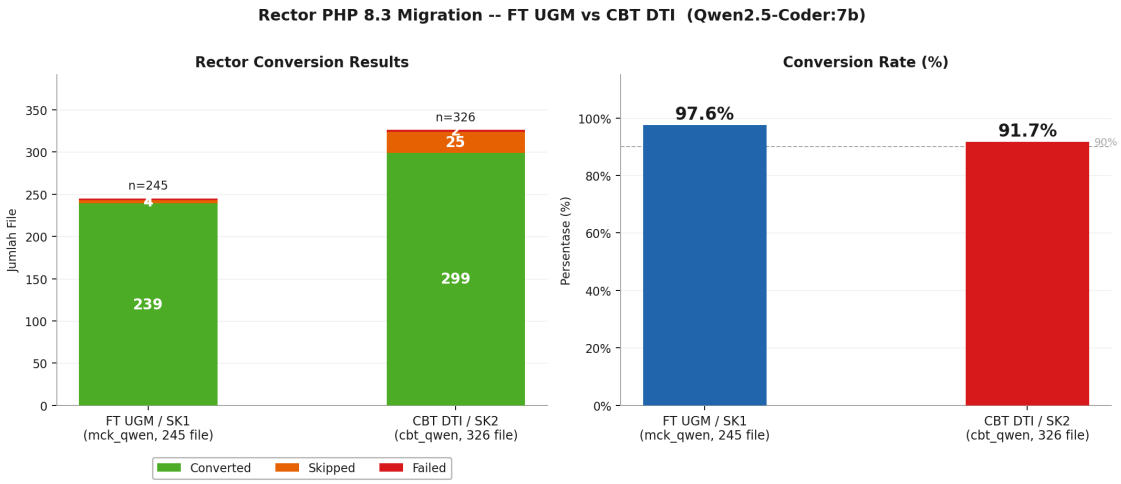

Gambar 4.20. Perbandingan temuan Semgrep *pre-scan* berdasarkan jenis kerentanan pada kedua studi kasus

Dataset FT UGM dan CBT DTI masing-masing menghasilkan 11 temuan Semgrep *pre-scan*, seluruhnya terlokalisasi pada `system/core/Output.php` yang merupakan komponen inti *framework* CodeIgniter 3. Pola temuan identik: 1 SQL Injection dan 10 SSRF pada kedua dataset. Pola ini mengonfirmasi fakta bahwa kedua aplikasi menggunakan versi CodeIgniter 3 yang sama dengan kode *framework* yang tidak dimodifikasi. Kode aplikasi kustom pada kedua dataset tidak menghasilkan temuan Semgrep.

Pada kedua studi kasus, nilai $\Delta F = 0$ menunjukkan konversi Rector tidak memperkenalkan maupun menghilangkan kerentanan. Temuan yang ada memerlukan remediasi pada tingkat kode *framework*, yang berada di luar cakupan transformasi sintaksis otomatis.

### 4.3.2 Perbandingan Tingkat Konversi Rector

Gambar 4.21 menunjukkan perbandingan hasil konversi Rector pada kedua dataset.

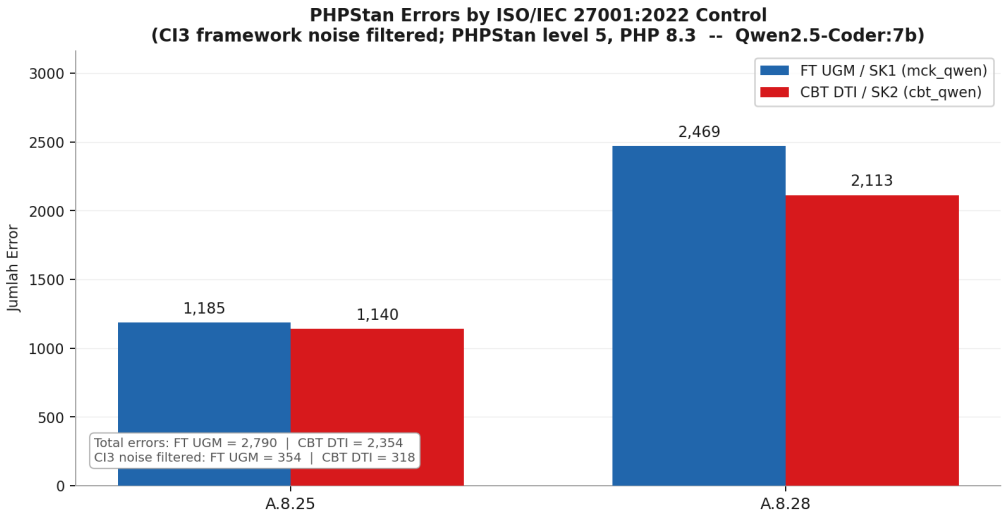



Gambar 4.21. Perbandingan hasil konversi Rector pada kedua studi kasus

FT UGM mencatat *conversion rate* 97,6% (239/245) sementara CBT DTI mencatat 91,7% (299/326). Perbedaan 5,9 poin persentase ini terutama disebabkan oleh proporsi *file* yang dilewati: CBT DTI memiliki 25 *file skipped* (7,7%) dibanding 4 *file* pada FT UGM (1,6%). Kedua dataset mengalami kegagalan pada 2 *file* yang sama (`captcha_helper.php` dan `form_helper.php`) akibat *bug* `FiberNodeScopeResolver` pada Rector, mengkonfirmasi bahwa kegagalan ini bersifat sistemik pada versi Rector yang digunakan, bukan spesifik pada salah satu dataset. Pada kedua studi kasus, validitas sintaks pasca-konversi mencapai 100%.

### 4.3.3 Perbandingan Temuan PHPStan

Gambar 4.22 menunjukkan perbandingan distribusi *error* PHPStan berdasarkan kontrol ISO/IEC 27001:2022 pada kedua dataset.




Gambar 4.22. Perbandingan *error* PHPStan berdasarkan kontrol ISO/IEC 27001:2022 pada kedua studi kasus

FT UGM menghasilkan 2.790 *error* PHPStan sementara CBT DTI menghasilkan 2.354 *error*. Keduanya dijalankan pada PHPStan *level* 5 dengan target PHP 8.3, sehingga angka ini dapat dibandingkan secara langsung. Sebagian besar *error* pada kedua dataset merupakan *false positive* akibat keterbatasan PHPStan dalam menelusuri arsitektur dinamis CodeIgniter 3.x, khususnya *magic property* dan fungsi *helper* yang diinjeksi saat *runtime*. Di samping itu, sebagian *error* mencerminkan masalah kompatibilitas dan kualitas kode aplikasi yang sesungguhnya, sebagaimana dibahas pada subbab sebelumnya. Jumlah *error* yang lebih tinggi pada FT UGM, meskipun ukuran datasetnya lebih kecil (245 vs 326 *file*), disebabkan oleh penggunaan dependensi pihak ketiga yang lebih beragam, PHPStan tidak dapat me-*resolve* kelas-kelas seperti `PhpOffice\PhpSpreadsheet` dan `Dotenv\Dotenv` karena dependensi tidak diinstal secara penuh di lingkungan analisis.

### 4.3.4 Ringkasan Perbandingan dan Analisis Dependensi Composer

Tabel 4.7 merangkum seluruh metrik utama kedua studi kasus dalam satu tampilan, termasuk status dependensi *composer* FT UGM.

Tabel 4.7. Ringkasan perbandingan metrik *pipeline* dan status dependensi *composer* kedua studi kasus

| **Metrik** | **FT UGM / SK1 (mck_qwen)** | **CBT DTI / SK2 (cbt_qwen)** |
|---|---|---|
| File PHP dianalisis | 245 | 326 |
| Semgrep *findings* (pre) | 11 | 11 |
| – SQL Injection | 1 | 1 (CI3 core) |
| – SSRF | 10 | 10 (CI3 core) |
| Semgrep delta (post–pre) | +0 (stabil) | +0 (stabil) |
| Rector *converted* | 239 (97,6%) | 299 (91,7%) |
| Rector *skipped* | 4 | 25 |
| Rector *failed* | 2 (*Fiber bug*) | 2 (*Fiber bug*) |
| PHPStan total *errors* | 2.790 | 2.354 |
| CI3 *noise filtered* | 354 | 318 |
| AI *recommendations* | 11 (Qwen2.5-Coder:7b) | 11 (Qwen2.5-Coder:7b) |
| ISO *overall status* | NON-COMPLIANT | NON-COMPLIANT |
| Kode aplikasi *custom* (Semgrep) | 0 *findings* | 0 *findings* |

PHPStan *errors* pada kedua studi kasus mayoritas merupakan *false positive* dari CI3 *framework* (magic props, runtime constants). Delta Semgrep bernilai 0 untuk kedua SK karena Rector tidak memperkenalkan maupun menghilangkan *vulnerability*. Seluruh

Semgrep *findings* yang terdeteksi berasal dari `CI3 system/core/Output.php` dan bukan merupakan bagian dari kode aplikasi *custom*.

Secara umum, kedua studi kasus menunjukkan pola yang konsisten: *pipeline* berhasil menjalankan seluruh tahapan secara otomatis, konversi Rector menghasilkan kode yang valid secara sintaksis di kedua dataset, dan status ISO/IEC 27001:2022 keduanya adalah `NON_COMPLIANT` yang disebabkan oleh *error* PHPStan. Perbedaan utama terletak pada tingkat konversi Rector (97,6% vs 91,7%), jumlah *error* PHPStan (2.790 vs 2.354), dan ketersediaan analisis dependensi *composer*—FT UGM memiliki berkas `composer.json` dengan lima dependensi yang diperiksa (tiga `SAFE`, dua `MANUAL REVIEW`), sementara CBT DTI tidak memiliki berkas tersebut sehingga langkah analisis dependensi dilewati secara otomatis. Dari sisi durasi eksekusi, kedua studi kasus sebanding, sekitar 3–4 menit, karena keduanya menggunakan Qwen2.5-Coder 7b sebagai *AI Engine* dengan satu kali inferensi per temuan; eksperimen komparasi multi-model (Kondisi A–C) pada studi kasus FT UGM merupakan prosedur evaluasi terpisah dan tidak termasuk dalam pengukuran durasi *pipeline*.

### 4.3.5 Hasil Validasi Efektivitas *AI Engine*

Validasi efektivitas komponen *AI Engine* dilakukan melalui eksperimen *blind rating* oleh dua orang rater independen. Tujuan eksperimen adalah mengukur secara empiris apakah keluaran *pipeline* dengan *AI Engine* aktif (kondisi T1) memberikan nilai tambah yang terukur dibandingkan keluaran tanpa *AI Engine* (kondisi B0). Dataset yang digunakan adalah 11 temuan Semgrep dari studi kasus FT UGM (FT-01 hingga FT-11), yang terdiri dari 1 SQL Injection dan 10 SSRF.

#### 4.3.5.1 Desain Eksperimen

Dua kondisi keluaran yang dibandingkan adalah sebagai berikut:

- Kondisi B0: keluaran *pipeline* tanpa *AI Engine*, hanya memuat *rule_id*, nama *file*, nomor baris, tingkat keparahan, potongan kode, dan pemetaan kontrol ISO/IEC 27001:2022 berbasis aturan deterministik.
- Kondisi T1: keluaran *pipeline* dengan *AI Engine* aktif menggunakan model Qwen2.5-Coder 7b, yang menambahkan penjelasan kerentanan dan saran remediasi secara otomatis di atas informasi yang tersedia pada B0.

Setiap temuan dikemas menjadi dua paket anonim (Paket A dan Paket B). Penetapan B0/T1 ke masing-masing paket diacak menggunakan Python `random` dengan `seed=42` sehingga rater tidak mengetahui kondisi mana yang sedang dinilai. Dua rater independen dari DTETI FT UGM yang tidak terlibat dalam penelitian ini memberikan penilaian, yaitu Muhammad Aidiel Rachman Putra, S.Kom., M.Kom.

(Rater 1) dan Dr. Warsun Najib, S.T., M.Sc. (Rater 2). Setiap paket dinilai pada empat dimensi menggunakan skala Likert 7 poin: Akurasi, Kejelasan, *Actionability* (kelayakan saran perbaikan), dan Ketepatan pemetaan ISO. Skor rata-rata per paket dihitung sebagai rata-rata keempat dimensi.

#### 4.3.5.2 Skor Rata-rata per Temuan

Tabel 4.8 menunjukkan skor rata-rata (rata-rata keempat dimensi) untuk kondisi B0 dan T1 per temuan, dirinci per rater dan gabungan keduanya.

Tabel 4.8. Skor rata-rata B0 dan T1 per temuan per rater (skala Likert 7, rata-rata 4 dimensi)

| **Temuan** | **Rater 1 (Aidiel)** | | **Rater 2 (Warsun)** | | **Gabungan** | |
|---|---|---|---|---|---|---|
| | **B0** | **T1** | **B0** | **T1** | **B0** | **T1** |
| FT-01 | 6,75 | 6,75 | 6,00 | 6,75 | 6,38 | 6,75 |
| FT-02 | 5,75 | 5,75 | 5,25 | 5,50 | 5,50 | 5,63 |
| FT-03 | 6,50 | 6,50 | 5,25 | 6,25 | 5,88 | 6,38 |
| FT-04 | 7,00 | 7,00 | 6,25 | 5,25 | 6,63 | 6,13 |
| FT-05 | 7,00 | 7,00 | 5,25 | 6,25 | 6,13 | 6,63 |
| FT-06 | 6,50 | 6,50 | 5,25 | 6,25 | 5,88 | 6,38 |
| FT-07 | 7,00 | 7,00 | 5,25 | 6,25 | 6,13 | 6,63 |
| FT-08 | 7,00 | 7,00 | 5,25 | 6,25 | 6,13 | 6,63 |
| FT-09 | 6,50 | 6,50 | 5,25 | 6,25 | 5,88 | 6,38 |
| FT-10 | 7,00 | 7,00 | 5,25 | 6,50 | 6,13 | 6,75 |
| FT-11 | 6,50 | 6,50 | 6,25 | 5,25 | 6,38 | 5,88 |
| **Rata-rata** | **6,68** | **6,68** | **5,50** | **6,07** | **6,09** | **6,38** |

Rater 1 memberikan skor yang identik antara B0 dan T1 pada semua temuan, sehingga perbedaan antarkondisi pada skor gabungan sepenuhnya berasal dari penilaian Rater 2. Rater 2 memberikan skor T1 lebih tinggi pada delapan dari sepuluh temuan uji (FT-02 hingga FT-11), dengan dua pengecualian pada FT-04 dan FT-11 di mana skor B0 lebih tinggi.

#### 4.3.5.3 Perbandingan per Dimensi

Tabel 4.9 merangkum rata-rata skor gabungan kedua rater untuk masing-masing dimensi penilaian.

Perbedaan paling besar terlihat pada dimensi Kejelasan (+0,591) dan *Actionability* (+0,500). Dimensi Akurasi hanya berbeda tipis (+0,045), dan Ketepatan pemetaan ISO tidak menunjukkan perbedaan sama sekali antara B0 dan T1.

Tabel 4.9. Rata-rata skor B0 dan T1 per dimensi penilaian (gabungan 2 rater, $n = 11$ temuan)

| **Dimensi** | **B0** | **T1** | **Selisih (T1 − B0)** |
|---|---|---|---|
| Akurasi | 6,000 | 6,045 | +0,045 |
| Kejelasan | 5,909 | 6,500 | +0,591 |
| *Actionability* | 5,955 | 6,455 | +0,500 |
| Ketepatan pemetaan ISO | 6,500 | 6,500 | 0,000 |
| **Keseluruhan** | **6,091** | **6,375** | **+0,284** |

#### 4.3.5.4 Analisis Statistik

**Reliabilitas antar-rater.** Reliabilitas antar-rater diukur menggunakan Cohen's kappa berdasarkan keputusan biner per dimensi per temuan (T1 menang atau tidak). Nilai yang diperoleh adalah $\kappa = -0{,}097$, yang termasuk dalam kategori *Poor* berdasarkan interpretasi Landis dan Koch. Nilai negatif menunjukkan bahwa tingkat kesepakatan antara kedua rater lebih rendah daripada yang diharapkan secara kebetulan.

**Uji Wilcoxon.** FT-01 merupakan satu-satunya temuan SQL Injection dalam dataset dan disajikan secara deskriptif terpisah dari 10 temuan SSRF. Skor gabungan FT-01 adalah B0 = 6,38 dan T1 = 6,75, menunjukkan selisih +0,38 poin.

Untuk FT-02 hingga FT-11 ($n = 10$ temuan SSRF), uji Wilcoxon *signed-rank* satu arah diterapkan dengan hipotesis T1 $>$ B0 menggunakan skor gabungan kedua rater. Hasil uji menghasilkan statistik $W = 44{,}0$ dan nilai $p = 0{,}0469$ (satu arah), atau $p = 0{,}0938$ (dua arah). Dengan ambang signifikansi $\alpha = 0{,}05$, uji satu arah menunjukkan bahwa skor T1 secara statistik lebih tinggi daripada skor B0 ($p < 0{,}05$).

#### 4.3.5.5 Diskusi

Hasil validasi menunjukkan bahwa kontribusi *AI Engine* paling terasa pada dimensi Kejelasan dan *Actionability*. Keluaran T1, yang menyertakan penjelasan kerentanan dan saran remediasi dari Qwen2.5-Coder, membantu rater memahami implikasi temuan dan langkah perbaikan yang perlu diambil dengan lebih mudah dibandingkan keluaran B0 yang hanya memuat informasi teknis mentah dari Semgrep. Perbedaan rata-rata sebesar 0,284 poin pada skala 7 dikonfirmasi signifikan secara statistik oleh uji Wilcoxon satu arah ($p = 0{,}0469$).

Tidak adanya perbedaan pada dimensi Ketepatan pemetaan ISO antara B0 dan T1 dapat dijelaskan secara konseptual: pemetaan kontrol ISO berbasis aturan deterministik yang diimplementasikan dalam `iso_mapper.py` sudah menghasilkan pemetaan yang lengkap dan konsisten tanpa memerlukan penguatan dari model bahasa. Hal yang serupa terjadi pada dimensi Akurasi: Semgrep sudah mengidentifikasi kerentanan secara tepat

sehingga kehadiran penjelasan AI tidak mengubah persepsi rater terhadap keakuratan identifikasi temuan. Dengan kata lain, nilai tambah *AI Engine* bersifat komplementer terhadap komponen lain *pipeline*, bukan pengganti.

#### 4.3.5.6 Limitasi

Nilai Cohen's kappa yang sangat rendah ($\kappa = -0{,}097$) merupakan keterbatasan utama yang perlu diperhatikan dalam interpretasi hasil ini. Analisis data mentah menunjukkan bahwa Rater 1 secara konsisten memberikan skor yang sama antara Paket A dan Paket B untuk hampir semua temuan, sedangkan pola penilaian Rater 2 pada dimensi Kejelasan dan *Actionability* menunjukkan indikasi *position bias*: paket yang disajikan pada posisi tertentu cenderung menerima skor lebih tinggi pada beberapa temuan, terlepas dari kondisi B0 atau T1 yang dikandungnya. Perbedaan gaya respons ini menyebabkan kappa negatif dan mengindikasikan bahwa kedua rater menggunakan standar penilaian yang berbeda secara struktural. Oleh karena itu, kesimpulan bahwa T1 lebih baik dari B0 pada dimensi Kejelasan dan *Actionability* harus dibaca dengan kehati-hatian yang sesuai.

Keterbatasan lain yang perlu dicatat adalah jumlah sampel yang kecil ($n = 10$ untuk uji statistik) dan homogenitas dataset (10 dari 11 temuan merupakan SSRF dari satu pola kode yang berulang dalam `Output.php`). Keberagaman konteks kerentanan yang terbatas ini mengurangi kemampuan generalisasi hasil ke jenis kerentanan lain, sehingga studi lanjutan dengan dataset yang lebih beragam dan melibatkan lebih banyak rater diperlukan untuk mengkonfirmasi temuan ini secara lebih kuat.

# BAB V
# KESIMPULAN DAN SARAN

## 5.1 Kesimpulan

Penelitian ini berhasil membangun *pipeline* berbasis Python yang mengintegrasikan konversi otomatis kode PHP lama, pemindaian keamanan, dan pemetaan kepatuhan ISO/IEC 27001:2022 dalam satu alur kerja yang dapat dijalankan sepenuhnya secara lokal. Berdasarkan pengujian pada dua studi kasus nyata, berikut kesimpulan yang menjawab tujuan penelitian.

**Pertama**, konversi otomatis PHP berbasis Rector berhasil diimplementasikan dengan tingkat keberhasilan 97,6% pada dataset FT UGM dan 91,7% pada dataset CBT DTI UGM, dengan validitas sintaks pasca-konversi mencapai 100% pada keduanya. Kegagalan pada dua *file* di kedua dataset disebabkan oleh *bug* internal Rector, bukan dari kode sumber.

Validasi berbasis *sampling* terhadap 12 *file* aplikasi menunjukkan rata-rata *similarity score* 0,9214 antara keluaran Rector dan referensi konversi manual, mengindikasikan tingkat kesesuaian yang tinggi antara pendekatan otomatis dan manual.

**Kedua**, mekanisme pemeriksaan keamanan yang dipetakan ke kontrol ISO/IEC 27001:2022 Annex A berhasil diimplementasikan secara deterministik dan dapat diaudit sepenuhnya. Nilai $\Delta F = 0$ pada kedua studi kasus mengonfirmasi bahwa konversi Rector tidak memperkenalkan kerentanan baru ke dalam basis kode, sebuah konfirmasi yang penting mengingat transformasi otomatis berskala besar berpotensi mengubah perilaku kode secara tidak terduga. Status `NON_COMPLIANT` yang dihasilkan bukan cerminan kegagalan *pipeline*, melainkan potret kondisi nyata kode warisan yang memang sudah mengandung permasalahan keamanan sebelum dilakukan remediasi. Seluruh hasil analisis ini disatukan dalam laporan JSON yang dibuat secara otomatis pada setiap eksekusi *pipeline*. Laporan otomatis ini dapat dimanfaatkan langsung sebagai artefak teknis pendukung dalam proses audit sertifikasi standar keamanan tanpa memerlukan rekapitulasi manual tambahan.

**Ketiga**, komponen validasi PHPStan memberikan manfaat konkret dengan mengekspos ribuan potensi masalah kompatibilitas tipe yang tidak akan terdeteksi sebelum kode benar-benar dijalankan di lingkungan produksi. Mayoritas temuan merupakan *technical debt* yang sudah ada jauh sebelum migrasi dilakukan dan baru terlihat karena sistem tipe PHP 8.3 jauh lebih ketat dibanding versi sebelumnya. Hal ini menunjukkan bahwa *pipeline* tidak hanya menjalankan transformasi sintaksis, tetapi

juga memetakan kondisi aktual basis kode yang sebelumnya tidak terdeteksi akibat sifat permisif sistem tipe pada PHP versi lama.

**Keempat**, *AI Engine* berbasis LLM *open source* lokal terbukti menghasilkan keluaran yang secara terukur lebih informatif dibandingkan keluaran *pipeline* tanpa komponen AI. Eksperimen *blind rating* oleh dua rater independen dari DTETI FT UGM terhadap 11 temuan Semgrep menunjukkan bahwa kondisi T1 (dengan *AI Engine* aktif) unggul atas kondisi B0 (tanpa *AI Engine*) pada dimensi Kejelasan (+0,591 poin) dan *Actionability* (+0,500 poin) dari skala Likert 7. Perbedaan rata-rata keseluruhan sebesar 0,284 poin dikonfirmasi signifikan secara statistik melalui uji Wilcoxon *signed-rank* satu arah ($W = 44{,}0$, $p = 0{,}047$, $n = 10$ temuan SSRF). Sebaliknya, tidak ada perbedaan pada dimensi Akurasi maupun Ketepatan pemetaan ISO, yang menunjukkan bahwa pemetaan ISO berbasis aturan deterministik yang telah diimplementasikan sudah memadai dan tidak memerlukan penguatan dari model bahasa. Hasil ini perlu dibaca dengan kehati-hatian mengingat nilai Cohen's kappa antar-rater yang rendah ($\kappa = -0{,}097$), yang mengindikasikan perbedaan gaya respons antara kedua rater.

Sebagai bagian dari evaluasi ini, komparasi performa lima model LLM *open source* menunjukkan bahwa *Format Compliance Rate* dan *Syntax Validity Rate* merupakan dua dimensi yang independen dan tidak berkorelasi. Qwen2.5-Coder direkomendasikan sebagai model utama karena secara konsisten menghasilkan SVR 100% di seluruh kondisi eksperimen. SVR terbukti sebagai metrik evaluasi yang paling sahih untuk sistem berbasis LLM lokal karena hasilnya dapat diverifikasi secara independen menggunakan `php -l`, berbeda dari *confidence score self-reported* yang tidak berkorelasi dengan kualitas kode yang sebenarnya dihasilkan.

## 5.2 Saran

Berdasarkan keterbatasan yang teridentifikasi, berikut arah pengembangan yang disarankan untuk penelitian selanjutnya.

1. Evaluasi LLM perlu diperkuat dari dua sisi: validasi semantik menggunakan *test suite* otomatis untuk memverifikasi kebenaran logika perbaikan, dan perluasan dataset dengan keragaman jenis kerentanan yang lebih tinggi agar kesimpulan perbandingan model dapat digeneralisasi lebih luas. Sebagai kelanjutan jangka panjang, eksplorasi *fine-tuning* dengan LoRA atau QLoRA dapat dilakukan dengan menjadikan penelitian ini sebagai *baseline*.
2. Akurasi laporan PHPStan pada basis kode CodeIgniter 3.x dapat ditingkatkan melalui dua pendekatan: (1) *custom stubs* yang mendefinisikan tipe untuk *magic property* CI3 seperti `$this->db` dan `$this->load` secara eksplisit, sehingga PHPStan tidak perlu bergantung pada *runtime resolution*; dan (2) penggantian

mekanisme *noise filter* berbasis *substring matching* yang digunakan saat ini dengan konfigurasi *baseline* PHPStan yang lebih akurat dan tidak berisiko menyaring temuan nyata secara tidak sengaja.

3. Validasi pemetaan ISO/IEC 27001:2022 oleh auditor atau praktisi bersertifikat perlu dilakukan untuk meningkatkan kredibilitas laporan kepatuhan yang dihasilkan.
4. Integrasi ke CI/CD dan perluasan ke bahasa pemrograman lain dapat dieksplorasi lebih lanjut. Arsitektur modular *pipeline* memudahkan penggantian komponen Rector dan Semgrep dengan alat setara untuk Python, JavaScript, atau bahasa lain, sekaligus membuka kemungkinan integrasi ke GitHub Actions atau GitLab CI.
5. Pengujian terhadap PHP 8.4 dan 8.5 serta perbandingan dengan alat komersial seperti SonarQube atau Snyk pada dataset yang sama perlu dilakukan untuk memvalidasi cakupan dan akurasi *pipeline* secara lebih komprehensif.
6. Validasi efektivitas *AI Engine* perlu direplikasi dengan jumlah rater yang lebih banyak dan desain eksperimen yang mengendalikan *position bias*. Nilai Cohen's kappa yang rendah ($\kappa = -0{,}097$) pada eksperimen *blind rating* mengindikasikan perbedaan gaya respons antar-rater yang substansial, sehingga kesimpulan tentang keunggulan T1 atas B0 memerlukan konfirmasi lebih lanjut. Selain itu, keberagaman dataset yang lebih tinggi – mencakup lebih banyak jenis kerentanan selain SSRF – diperlukan agar hasil validasi dapat digeneralisasi ke konteks yang lebih luas.

# DAFTAR PUSTAKA


[1] W3Techs, "Usage statistics and market share of PHP for websites," W3Techs Web Technology Surveys, 2026, [Online]. Available: https://w3techs.com/technologies/details/pl-php. [Accessed: Apr. 8, 2026].

[2] The PHP Group, "PHP: Supported versions," php.net, 2025, [Online]. Available: https://www.php.net/supported-versions.php. [Accessed: Apr. 2026].

[3] National Institute of Standards and Technology, "National vulnerability database (NVD)," NIST, 2024, [Online]. Available: https://nvd.nist.gov/. [Accessed: Apr. 2026].

[4] Semgrep, "Semgrep — lightweight static analysis for many languages," semgrep.dev, 2024, [Online]. Available: https://semgrep.dev. [Accessed: Apr. 2026].

[5] PHPStan, "PHPStan — php static analysis tool," phpstan.org, 2024, [Online]. Available: https://phpstan.org. [Accessed: Apr. 2026].

[6] Rector, "Rector — instant upgrades and automated refactoring," getrector.com, 2024, [Online]. Available: https://getrector.com. [Accessed: Apr. 2026].

[7] D. Chen, D. Lawler, Y. Zhang, P. Kuchhal, D. Westcott, and G. Yang, "AST-based source code migration through symbols replacement," in *Proc. 2022 IEEE Asia-Pacific Conf. on Computer Science and Data Engineering (CSDE)*, Brisbane, Australia, 2022, pp. 1–6.

[8] S. Strobl, C. Zoffi, C. Haselmann, M. Bernhart, and T. Grechenig, "Automated code transformations: Dealing with the aftermath," in *Proc. 2020 IEEE 27th Int. Conf. on Software Analysis, Evolution and Reengineering (SANER)*, London, Canada, 2020, pp. 627–631.

[9] V. Akuthota, R. Kasula, S. T. Sumona, M. Mohiuddin, M. T. Reza, and M. M. Rahman, "Vulnerability detection and monitoring using LLM," in *Proc. 2023 IEEE 9th Int. Women in Engineering (WIE) Conf. on Electrical and Computer Engineering (WIECON-ECE)*, Thiruvananthapuram, India, 2023, pp. 309–314.

[10] C. Li, B. Guan, Q. Zheng, B. Liu, and Z. Zhang, "Software vulnerability detection based on LLM," in *Proc. 2025 IEEE 7th Advanced Information Management, Communications, Electronic and Automation Control Conf. (IMCEC)*, Chongqing, China, 2025, pp. 1534–1539.

[11] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in *Proc. 13th Int. Conf. on Learning Representations (ICLR)*, Singapore, 2025, [Online]. Available: https://arxiv.org/abs/2405.17238.

[12] DeepSeek-AI, "DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024, [Online]. Available: https://arxiv.org/abs/2401.14196.

[13] B. Hui *et al.*, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024, [Online]. Available: https://arxiv.org/abs/2409.12186.

[14] B. Rozière *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2024, [Online]. Available: https://arxiv.org/abs/2308.12950.

[15] A. Q. Jiang *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023, [Online]. Available: https://arxiv.org/abs/2310.06825.

[16] A. Dubey *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024, [Online]. Available: https://arxiv.org/abs/2407.21783.

[17] Ollama, "Ollama — get up and running with large language models locally," ollama.com, 2024, [Online]. Available: https://ollama.com. [Accessed: Apr. 2026].

[18] International Organization for Standardization and International Electrotechnical Commission, "ISO/IEC 27001:2022 information security, cybersecurity and privacy protection — information security management systems: Requirements, 3rd ed." ISO/IEC, Geneva, 2022.

[19] E. Merlo, D. Letarte, and G. Antoniol, "Automated protection of PHP applications against SQL-injection attacks," in *Proc. 11th European Conf. on Software Maintenance and Reengineering (CSMR)*, Amsterdam, Netherlands, 2007, pp. 191–202.

[20] J. Stümpfle, D. Atray, N. Jazdi, and M. Weyrich, "Large language model assisted transformation of software variants into a software product line," in *Proc. 2025 IEEE/ACM 22nd Int. Conf. on Software and Systems Reuse (ICSR)*, Ottawa, Canada, 2025, pp. 12–20.

[21] Z. Cai, L. Zhao, X. Wang, X. Yang, J. Qin, and K. Yin, "A pattern-based code transformation approach for cloud application migration," in *Proc. 2015 IEEE 8th Int. Conf. on Cloud Computing (CLOUD)*, New York, USA, 2015, pp. 33–40.

[22] The PHP Group, "PHP: History of PHP," php.net, 2024, [Online]. Available: https://www.php.net/manual/en/history.php. [Accessed: Apr. 2026].

[23] OWASP Foundation, "OWASP top 10:2021 — the ten most critical web application security risks," OWASP Foundation, 2021, [Online]. Available: https://owasp.org/Top10/. [Accessed: Apr. 2026].

[24] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach, 9th ed.* New York: McGraw-Hill Education, 2019.

[25] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-following eval for large language models," arXiv preprint arXiv:2311.07911, 2023, [Online]. Available: https://arxiv.org/abs/2311.07911.

[26] K. Yan, H. Guo, X. Shi, S. Cao, D. Di, and Z. Li, "CodeIF: Benchmarking the instruction-following capabilities of large language models for code generation," arXiv preprint arXiv:2502.19166, 2025, [Online]. Available: https://arxiv.org/abs/2502.19166.

[27] Element Labs Inc., “LM Studio: Discover, download, and run local LLMs,” lmstudio.ai, 2024, [Online]. Available: https://lmstudio.ai. [Accessed: Apr. 2026].

[28] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” GitHub, 2023, [Online]. Available: https://github.com/ggml-org/llama.cpp. [Accessed: Apr. 2026].

[29] J. W. Ratcliff and D. E. Metzener, “Pattern matching: The gestalt approach,” p. 46, Jul. 1988, [Online]. Available: https://xlinux.nist.gov/dads/HTML/ratcliffObershelp.html. [Accessed: Jun. 2026].

[30] R. Likert, “A technique for the measurement of attitudes,” *Archives of Psychology*, vol. 22, no. 140, pp. 1–55, 1932.

[31] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[32] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

# LAMPIRAN

## A.1 Tautan Repositori GitHub

Kode sumber alat *pipeline* migrasi PHP yang dikembangkan dalam penelitian ini tersedia secara publik di repositori GitHub berikut:

`https://github.com/MFarrelAkbar1/skripsi-php-migration`

Repositori ini memuat seluruh modul Python (`pipeline/`), konfigurasi Rector dan PHPStan, *ruleset* Semgrep lokal (`rules/`), berkas contoh pengujian (`tests/`), serta dokumentasi pendukung (`docs/`) yang digunakan selama proses penelitian.

## A.2 Struktur Folder Proyek

Berikut adalah struktur direktori lengkap dari repositori *pipeline* migrasi PHP.

```
skripsi-php-migration/
|-- README.md                          <- Dokumentasi utama
|-- requirements.txt                   <- Dependensi Python
|-- composer.json                      <- Dependensi PHP (Rector,
                                          PHPStan)
|
|-- pipeline/                          <- Semua modul Python
|   |-- main.py                        <- Entry point & orkestrator
                                          6 step
|   |-- scanner.py                     <- Semgrep security scanning
|   |-- converter.py                   <- Rector PHP conversion
|   |-- analyzer.py                    <- PHPStan static analysis
|   |-- ai_engine.py                   <- Ollama / LLM integration
|   |-- iso_mapper.py                  <- Pemetaan ISO/IEC 27001:2022
|   |-- composer_analyzer.py           <- Analisis kompatibilitas
                                           composer.json
|   |-- llm_quality_checker.py         <- Syntax validity rate
                                          per model
|   |-- accuracy_checker.py            <- Metrik akurasi pipeline
|   |-- accuracy_validator.py          <- Validasi hasil migrasi
|   |-- capture_responses.py           <- Capture raw LLM responses
|   |-- llm_comparison_chart.py        <- Chart perbandingan LLM
|   |-- generate_condition_a_charts.py <- Chart eksperimen kondisi A
|   |-- generate_condition_b_charts.py <- Chart eksperimen kondisi B
|   `-- pipeline_result_chart.py       <- Chart hasil pipeline
                                          keseluruhan
|
|-- input/                             <- PHP 7.4 asli
                                          (tidak dimodifikasi)
```

```
|-- output/                              <- PHP 8.x hasil konversi Rector
|-- reports/                               <- JSON reports + PNG charts
|-- rules/                                 <- Semgrep rules lokal (pinned)
|
|-- tests/
|   `-- sample_php74/                      <- Sample PHP untuk
|                                             pengujian cepat
|       |-- sample_login.php
|       |-- sample_realistic.php
|       `-- sample_upload.php
|
|-- docs/
|   |-- architecture.md
|   |-- iso_controls.md
|   `-- pseudocode_appendix.md
|
`-- vendor/                                 <- Paket Composer
```

Keterangan singkat setiap direktori utama:

| **Direktori / Berkas** | **Keterangan** |
|---|---|
| `pipeline/` | Seluruh modul Python yang mengimplementasikan enam tahap pipeline: pra-pindai, konversi, pasca-pindai, analisis statis, rekomendasi AI, dan pemetaan ISO. |
| `input/` | Folder tempat file PHP 7.4 asli ditempatkan sebelum diproses. Isi folder ini **tidak pernah dimodifikasi** oleh pipeline. |
| `output/` | Hasil konversi Rector: file PHP 8.x yang sudah diperbarui sintaksnya. |
| `reports/` | Laporan JSON dan grafik PNG yang dihasilkan setelah pipeline selesai, termasuk laporan kepatuhan ISO dan perbandingan model LLM. |
| `rules/` | *Ruleset* Semgrep yang telah di-*pin* secara lokal agar hasil eksperimen dapat direproduksi tanpa bergantung pada versi *ruleset* online. |
| `tests/ sample_php74/` | Tiga berkas PHP sampel untuk pengujian cepat tanpa harus menyiapkan seluruh dataset studi kasus. |
| `docs/` | Dokumentasi arsitektur sistem, daftar kontrol ISO yang dipetakan, dan pseudocode dalam format Markdown. |
| `vendor/` | Paket PHP yang diinstall oleh Composer (Rector, PHPStan). |

**Pseudocode Pipeline Migrasi PHP**

Lampiran ini menyajikan pseudocode setiap modul pada pipeline migrasi PHP yang dikembangkan. Pseudocode merupakan representasi logika program dalam bahasa semi-formal yang lebih mudah dibaca dibandingkan kode program sesungguhnya, tanpa terikat pada sintaks bahasa pemrograman tertentu.

**Panduan Membaca Pseudocode**

Berikut keterangan notasi yang digunakan pada seluruh pseudocode di lampiran ini.

| **Notasi** | **Keterangan** |
|---|---|
| `PROGRAM: NamaProgram` | Nama modul/program yang didefinisikan. |
| `FUNCTION nama(param): ...END FUNCTION` | Definisi fungsi beserta parameternya. Kode di dalam blok ini dijalankan saat fungsi dipanggil. |
| `param <- nilai` | Penugasan nilai ke variabel (dibaca: *param diisi dengan nilai*). |
| `IF kondisi THEN ...ELSE...END IF` | Percabangan: blok dalam `THEN` dijalankan jika kondisi benar, blok `ELSE` jika salah. |
| `FOR each item in daftar DO ...END FOR` | Perulangan: blok dijalankan satu kali untuk setiap elemen dalam daftar. |
| `RETURN nilai` | Mengembalikan hasil fungsi ke pemanggil. |
| `DATA STRUCTURE Nama:` | Definisi struktur data (kumpulan atribut yang membentuk satu entitas). |
| `CONSTANT NAMA: nilai` | Nilai tetap yang tidak berubah selama program berjalan. |
| `// komentar` | Keterangan tambahan, **tidak dieksekusi** oleh program. |
| `NULL` | Tidak ada nilai / kosong. |
| `A -> B` | Menunjukkan keluaran atau pemetaan: *A menghasilkan B*. |
| `A <- B` (di luar penugasan) | Menunjukkan asal data: *A berasal dari B*. |

### B.1 Modul `main.py` — Orkestrator Pipeline Utama

Modul ini adalah titik masuk (*entry point*) program. Ketika pengguna menjalankan pipeline dari *command line*, modul inilah yang pertama kali dieksekusi. Tugasnya adalah membaca argumen yang diberikan pengguna (seperti lokasi folder PHP yang akan dimigrasi, model AI yang dipakai, dan opsi lainnya), kemudian memanggil fungsi `run_pipeline()` yang mengorkestrasi seluruh tahapan migrasi secara berurutan. Di akhir proses, program keluar dengan kode `0` jika sistem dinyatakan patuh ISO, atau kode `1` jika belum.

```
PROGRAM: MigrationPipeline

FUNCTION main():
    args <- parse_command_line_arguments()
        // argumen: --input, --output, --reports, --skip-ai, --model,
        //          --compare-all, --target-php, --php-version, dll.

    IF output_dir is NOT empty THEN
        prompt user "Hapus isi output/? [Y/n]"
        IF user confirms THEN
            delete contents of output_dir
        ELSE
            exit program
        END IF
    END IF

    result <- run_pipeline(args)

    IF result.iso_status == COMPLIANT THEN
        exit with code 0   // sistem dinyatakan patuh ISO
    ELSE
        exit with code 1   // masih terdapat temuan keamanan
    END IF
END FUNCTION


FUNCTION run_pipeline(input_dir, output_dir, reports_dir, options):
    record start_time

    // Step 1: Pindai keamanan kode SEBELUM konversi (kondisi awal)
    pre_scan <- run_scan(input_dir)

    // Step 2: Konversi sintaks PHP lama ke PHP 8.x menggunakan Rector
    conversion <- run_conversion(input_dir, output_dir, target_version)

    // Step 2b: Analisis dependensi library di composer.json
    composer_analysis <- run_composer_analysis(input_dir,
```

```
    target_version)

    // Step 3: Pindai keamanan kode SESUDAH konversi (kondisi akhir)
    IF conversion succeeded THEN
        post_scan <- run_scan(output_dir)
    END IF

    // Step 4: Analisis statis PHPStan untuk mendeteksi kesalahan tipe
    //         dan referensi kode yang tidak valid
    IF conversion succeeded THEN
        analysis <- run_analysis(output_dir, phpstan_level,
        php_version)
    END IF

    // Step 5: Analisis AI untuk rekomendasi perbaikan kerentanan
    IF skip_ai == FALSE THEN
        IF compare_all == TRUE THEN
            // Mode perbandingan: uji semua model LLM,
            3 kali per temuan
            run_llm_comparison(pre_scan.findings, all_models,
            3_runs_each)
        ELSE
            // Mode normal: gunakan satu model terpilih
            ai_recs <- run_ai_analysis(pre_scan.findings,
            selected_model)
        END IF
    END IF

    // Step 6: Petakan semua temuan ke kontrol ISO/IEC 27001:2022
    iso_report <- run_iso_mapping(
        pre_scan, analysis, ai_recs, output_path)

    total_duration <- now() - start_time

    print_summary_table(pre_scan, post_scan, conversion, analysis,
    iso_report)
    save_pipeline_result_json(reports_dir, timestamp)

    RETURN PipelineResult(pre_scan, conversion, post_scan, analysis,
                          ai_recs, iso_report, composer_analysis,
                          duration)
END FUNCTION
```

### B.2 Modul `scanner.py` — Pemindaian Keamanan (Semgrep)

Modul ini bertanggung jawab menjalankan *tool* Semgrep untuk memindai seluruh file PHP dalam sebuah direktori dan mendeteksi potensi kerentanan keamanan. Semgrep adalah alat analisis kode statis berbasis pola (*pattern matching*): ia mencocokkan kode sumber dengan aturan-aturan yang mendefinisikan pola kode berbahaya. Setiap temuan kemudian diklasifikasikan ke dalam jenis kerentanan (misalnya SQL Injection atau XSS), diberi tingkat prioritas dari 1 (kritis) hingga 6 (rendah), dan dipetakan ke kontrol ISO/IEC 27001:2022 yang relevan.

```
PROGRAM: SemgrepScanner

DATA STRUCTURE ScanFinding:
    // Satu temuan kerentanan dari hasil Semgrep
    rule_id, file_path, line_start, line_end
    message, severity // ERROR | WARNING | INFO
    code_snippet  // potongan kode yang bermasalah
    vuln_type     // SQL Injection | XSS | Deprecated Function | ...
    iso_controls  // kontrol ISO terkait, e.g. ["A.8.28", "A.8.26"]
    priority      // 1..6 (1 = tertinggi / paling kritis)

DATA STRUCTURE ScanResult:
  findings   : list of ScanFinding // diurutkan berdasarkan prioritas
  summary    : ScanSummary         // jumlah total per jenis/severity
  raw_output : JSON dict           // keluaran mentah dari Semgrep

CONSTANT VULNERABILITY_RULES:
    // Pemetaan jenis kerentanan ke prioritas dan kontrol ISO
    priority 1: SQL Injection        -> ISO [A.8.28, A.8.26]
    priority 2: XSS                  -> ISO [A.8.28, A.8.26]
    priority 3: Deprecated Function  -> ISO [A.8.25, A.8.28]
    priority 4: Hardcoded Credentials -> ISO [A.5.17, A.8.28]
    priority 5: SSRF / Path Traversal -> ISO [A.8.28, A.8.29]
    priority 6: Weak Cryptography    -> ISO [A.8.24, A.8.28]


FUNCTION run_scan(target_path):
// Pilih ruleset Semgrep: lokal (reproducible) atau registry online
    IF local rules/ directory exists THEN
        rulesets <- local rules/ directory
    ELSE
        rulesets <- ["p/php", "p/owasp-top-ten"]
    END IF

    cmd <- build_semgrep_command(target_path, rulesets, "--json")
    raw_json <- invoke_subprocess(cmd, timeout=300s)

```

```
    findings <- []
    FOR each result in raw_json["results"] DO
        rule_id  <- result.check_id
        severity <- result.extra.severity
        message  <- result.extra.message
        snippet  <- result.extra.lines

      // Klasifikasi ke jenis kerentanan, kontrol ISO, dan prioritas
      (vuln_type, iso_controls, priority) <- classify(rule_id, message)

        findings.append(ScanFinding(...))
    END FOR

    // Urutkan dari yang paling kritis
    findings.sort(by priority ASC, then severity, then file_path)

    summary <- aggregate(findings)
    print_summary_table(summary)

    RETURN ScanResult(findings, summary, raw_json)
END FUNCTION


FUNCTION classify(rule_id, message):
// Cocokkan ID aturan dan pesan ke dalam VULNERABILITY_RULES
  haystack <- lowercase(rule_id + message[:100])

  FOR each (keywords, label, controls, priority) in
            VULNERABILITY_RULES DO
        IF any keyword in haystack THEN
            RETURN (label, controls, priority)
        END IF
    END FOR

    RETURN ("Other", ["A.8.28"], priority=99)
    // kategori tidak dikenal
END FUNCTION
```

### B.3 Modul `converter.py` — Konversi PHP (Rector)

Modul ini melakukan konversi otomatis kode PHP lama ke PHP 8.x menggunakan *tool* Rector. Rector adalah *tool* refaktorisasi PHP berbasis aturan (*rule-based*): ia membaca kode sumber, membangun representasi pohon sintaks (*Abstract Syntax Tree*), menerapkan aturan transformasi yang sesuai, lalu menuliskan kode yang sudah diperbarui. Modul ini terlebih dahulu mendeteksi versi PHP tiap file (PHP 5 atau PHP

7), memilih rantai aturan migrasi yang tepat, kemudian memverifikasi file mana yang benar-benar berubah menggunakan sidik jari SHA-256.

```
PROGRAM: RectorConverter

DATA STRUCTURE ConversionFile:
    source_path, output_path
    php_version      // PHP5 | PHP7 | UNKNOWN
    status           // CONVERTED | SKIPPED | FAILED
    changes_count    // jumlah perubahan yang diterapkan
    applied_rectors  // daftar nama aturan Rector yang aktif
    iso_controls     // ["A.8.25"] - Secure Development Lifecycle

DATA STRUCTURE ConversionResult:
    files             : list of ConversionFile
    summary           : ConversionSummary
    rector_raw_output : string (JSON)  // keluaran mentah Rector

CONSTANT LEVEL_SETS_PHP5:
    // Rantai migrasi penuh untuk kode PHP 5 (setiap versi minor)
    UP_TO_PHP_53, UP_TO_PHP_54, ..., UP_TO_PHP_85

CONSTANT LEVEL_SETS_PHP7:
    // Rantai lebih pendek untuk kode PHP 7
    UP_TO_PHP_70, ..., UP_TO_PHP_85

CONSTANT QUALITY_SETS:
    // Aturan peningkatan kualitas kode (selalu diterapkan)
    CODE_QUALITY, DEAD_CODE, EARLY_RETURN, TYPE_DECLARATION


FUNCTION run_conversion(input_path, output_path, target_version):
    php_files <- collect_all_php_files(input_path)

// Step 1: Deteksi versi PHP per file dari pola kode yang digunakan
    version_map <- {}
    FOR each file in php_files DO
        version_map[file] <- detect_php_version(file)
    END FOR

// Step 2: Pilih rantai aturan Rector sesuai versi PHP yang
    ditemukan
    IF PHP5 in version_map.values() THEN
        level_sets <- LEVEL_SETS_PHP5 truncated at target_version
    ELSE
        level_sets <- LEVEL_SETS_PHP7 truncated at target_version
    END IF
    all_sets <- level_sets + QUALITY_SETS
```

```
46
47 // Step 3: Salin input/ ke output/ agar input asli tidak pernah
       diubah
48     copy_tree(input_path, output_path)
49
50
51 // Step 4: Catat hash SHA-256 semua file SEBELUM Rector dijalankan
52     pre_hashes <- snapshot_sha256(output_path)
53
54 // Step 5: Tulis konfigurasi Rector sementara ke direktori temp
       sistem
55     config_path <- write_rector_config(output_path, all_sets)
56
57
58 // Step 6: Jalankan Rector sebagai subprocess
59     (exit_code, stdout, stderr) <- invoke_rector(config_path,
       timeout=600s)
60
61
62 // Step 7: Catat hash SHA-256 semua file SESUDAH Rector
63     post_hashes <- snapshot_sha256(output_path)
64
65 // Step 8: Hapus konfigurasi sementara
66     delete(config_path)
67
68 // Step 9: Tentukan status tiap file dengan membandingkan hash
69     rector_data <- parse_rector_json(stdout)
70     FOR each source_file in php_files DO
71         pre  <- pre_hashes[output_copy_of(source_file)]
72         post <- post_hashes[output_copy_of(source_file)]
73
74         IF file is in rector error list THEN
75             status <- FAILED     // Rector gagal memproses file ini
76         ELSE IF pre != post THEN
77             status <- CONVERTED  // file berhasil dikonversi
78         ELSE
79             status <- SKIPPED    // tidak ada perubahan yang perlu
80             diterapkan
81         END IF
82
83         files.append(ConversionFile(source_file, status, ...))
84     END FOR
85
86     summary <- aggregate(files, elapsed_time)
87     print_summary_table(summary)
88
89     RETURN ConversionResult(files, summary, stdout)
90 END FUNCTION
91
92
```

```
FUNCTION detect_php_version(file_path):
    // Baca 8KB pertama file untuk efisiensi
    content <- read first 8KB of file

    // Cek pola fungsi/sintaks yang hanya ada di PHP 5
    IF any PHP5 pattern matches (mysql_connect, ereg, split) THEN
        RETURN PHP5
    END IF

    // Cek pola yang diperkenalkan di PHP 7
    IF any PHP7 pattern matches (fn =>, ??, intdiv) THEN
        RETURN PHP7
    END IF

    RETURN UNKNOWN
END FUNCTION
```

### B.4 Modul `analyzer.py` — Analisis Statis (PHPStan)

Modul ini menjalankan PHPStan, sebuah alat analisis statis untuk PHP yang memeriksa kode tanpa mengeksekusinya. PHPStan mendeteksi kesalahan seperti pemanggilan fungsi yang tidak ada, ketidakcocokan tipe data, kode yang tidak pernah dieksekusi (*dead code*), dan referensi variabel yang tidak valid. Karena proyek target menggunakan kerangka kerja CodeIgniter 3 (CI3) yang mengandalkan konvensi *magic method* dan konstanta global yang tidak dikenali PHPStan, modul ini secara khusus menyaring pesan *false positive* dari CI3 sebelum hasilnya dilaporkan.

```
PROGRAM: PHPStanAnalyzer

DATA STRUCTURE AnalysisError:
    file_path, line
    message, severity   // ERROR | WARNING
    iso_controls        // kontrol ISO yang relevan

DATA STRUCTURE AnalysisResult:
    errors     : list of AnalysisError  // diurutkan per file lalu
                 per baris
    summary    : AnalysisSummary
    raw_output : JSON dict

CONSTANT CI3_FRAMEWORK_NOISE:
    // False positive khas arsitektur CodeIgniter 3 -- diabaikan:
    "extends unknown class CI_*"
    "Constant ENVIRONMENT/FCPATH/APPPATH not found"
    "$this->db, $this->input, $this->session, ..."
    "Function base_url/redirect/form_open not found"
```

```
20
21
22 FUNCTION run_analysis(target_path, level, php_version):
23     // Cari binary PHPStan (vendor/bin/ atau PATH global)
24     phpstan_exe <- find_phpstan(vendor/bin/ OR global PATH)
25
26     cmd <- build_phpstan_command(target_path, level, php_version,
27                             "--error-format=json", "--no-progress")
28     (exit_code, stdout, stderr) <- invoke_subprocess(cmd, timeout=300s)
29
30     raw_json <- parse_phpstan_json(stdout)
31
32     errors <- []
33     noise_count <- 0
34     FOR each (file_path, file_data) in raw_json["files"] DO
35         FOR each msg_entry in file_data["messages"] DO
36             message <- msg_entry.message
37             line    <- msg_entry.line
38
39             // Lewati pesan yang merupakan false positive CI3
40             IF message matches CI3_FRAMEWORK_NOISE THEN
41                 noise_count <- noise_count + 1
42                 CONTINUE
43             END IF
44
45             (severity, iso_controls) <- classify_error(message)
46             errors.append(AnalysisError(file_path, line, message,
47                                         severity, iso_controls))
48         END FOR
49     END FOR
50
51     errors.sort(by file_path, then line)
52
53     summary <- aggregate(errors, noise_count, elapsed_time)
54     print_results_table(errors, summary)
55
56     RETURN AnalysisResult(errors, summary, raw_json)
57 END FUNCTION
58
59
60 FUNCTION classify_error(message):
61     lower_msg <- lowercase(message)
62
63     // Kode mati atau jalur eksekusi yang tidak dapat dicapai
64     IF "dead code" OR "unreachable" OR "never returns"
65         in lower_msg THEN
66         RETURN ("WARNING", ["A.8.25"])
```

```
    END IF

    // Referensi ke fungsi, kelas, atau konstanta yang tidak ada
    IF "undefined" OR "not found" OR "unknown class" in lower_msg THEN
        RETURN ("ERROR", ["A.8.28", "A.8.25"])
    END IF

    // Ketidakcocokan tipe data atau nilai null yang tidak ditangani
  IF "type" OR "null" OR "incompatible" OR "expects" in lower_msg THEN
        RETURN ("ERROR", ["A.8.28"])
    END IF

    RETURN ("ERROR", ["A.8.28"])
    // fallback untuk pesan yang tidak dikenal
END FUNCTION
```

### B.5 Modul `ai_engine.py` — Analisis AI (LLM via Ollama)

Modul ini mengintegrasikan model bahasa besar (*Large Language Model* / LLM) lokal yang berjalan melalui Ollama untuk memberikan rekomendasi perbaikan kode terhadap setiap kerentanan yang ditemukan Semgrep. Modul mendukung dua mode: mode normal (satu model) dan mode perbandingan (lima model, tiga kali pengujian per temuan). Setiap potongan kode rentan dikirim ke LLM beserta konteks jenis kerentanan dan kontrol ISO yang berlaku, kemudian respons LLM diurai untuk mengekstrak penjelasan, kode perbaikan, dan skor kepercayaan (*confidence score*).

```
PROGRAM: PHPAIEngine

DATA STRUCTURE AIRecommendation:
    original_code, suggested_fix, explanation
    confidence    // skor kepercayaan LLM: 0.0 - 1.0
    iso_controls, vuln_type
    file_path, line_start
    model_used, fim_used  // apakah menggunakan Fill-in-Middle

CONSTANT COMPARISON_MODELS:
    ["deepseek-coder:6.7b", "qwen2.5-coder:7b",
     "codellama:7b", "mistral:7b", "llama3.1:8b"]

// Hanya proses temuan dengan prioritas 1-5 (abaikan kategori "Other")
CONSTANT PRIORITY_THRESHOLD: 5
// Batasi panjang kode yang dikirim ke LLM (karakter)
CONSTANT MAX_CODE_CHARS: 2000


FUNCTION run_ai_analysis(findings, model):
```

```
    // Saring hanya temuan yang cukup kritis untuk dianalisis AI
    eligible <- [f for f in findings WHERE f.priority <=
    PRIORITY_THRESHOLD]

    // Pastikan server Ollama lokal sedang berjalan
    IF Ollama NOT reachable at localhost:11434 THEN
        print warning "Ollama offline -- tahapan AI dilewati"
        RETURN []
    END IF

    recommendations <- []
    FOR each finding in eligible DO
        rec <- analyze_snippet(finding.code_snippet, finding.vuln_type,
                               finding.iso_controls, finding.file_path,
                               finding.line_start, model)
        recommendations.append(rec)
    END FOR

    print_batch_summary(recommendations)
    RETURN recommendations
END FUNCTION


FUNCTION analyze_snippet(code, vuln_type, iso_controls,
                         file_path, line_start, model):
    truncated <- code[:MAX_CODE_CHARS]

    // Kode pendek (<= 3 baris): gunakan Fill-in-Middle (FIM) prompt
    // Kode lebih panjang: gunakan chat biasa
    IF lines(truncated) <= 3 THEN
      prompt   <- build_fim_prompt(truncated, vuln_type, iso_controls)
      fim_used <- TRUE
    ELSE
    messages <- build_chat_messages(truncated, vuln_type, iso_controls)
    fim_used <- FALSE
    END IF

    // temperature=0.1: output deterministik, minim variasi acak
    raw_text <- call_ollama_chat(messages, model, temperature=0.1)

    RETURN parse_response(raw_text, truncated, iso_controls,
                          vuln_type, file_path, line_start, fim_used)
END FUNCTION


FUNCTION build_chat_messages(code, vuln_type, iso_controls):
    // Kondisi A (standar): prompt identik untuk semua model
```

```
    system_msg <- "You are a PHP security and migration expert
                   controls [{iso_controls}]."
    user_msg   <- "TASK: Analyze [{vuln_type}] vulnerability...
                   VULNERABLE CODE: (kode PHP disisipkan di sini)
                   1. Explain the vulnerability.
                   2. Provide corrected PHP 8.x code.
                   Last line: CONFIDENCE: [0-100]"
    RETURN [system_msg, user_msg]
END FUNCTION


FUNCTION build_optimized_chat_messages(code, vuln_type, iso_controls,
model):
    // Kondisi B: prompt dioptimalkan khusus per model
    IF model == "qwen2.5-coder:7b" THEN
        // format diperlunak; CONFIDENCE tidak diwajibkan
    ELSE IF model == "deepseek-coder:6.7b" THEN
        // instruksi pendek dan langsung
    ELSE IF model == "mistral:7b" THEN
        // satu contoh few-shot untuk mengajarkan format keluaran
    ELSE IF model == "llama3.1:8b" THEN
        // role-play persona kuat + 4 langkah prosedural
    ELSE IF model == "codellama:7b" THEN
        // sama dengan Kondisi A + instruksi validitas sintaks php -l
    ELSE
        // fallback ke Kondisi A standar
    END IF
    RETURN [system_msg, user_msg]
END FUNCTION


FUNCTION parse_response(raw_text, original_code, iso_controls, ...):
    IF raw_text is empty THEN
        RETURN placeholder_recommendation(confidence=0.0)
    END IF

    // --- Ekstraksi penjelasan ---
    // Coba tag XML <explanation>, lalu "1. EXPLANATION:", lalu
       teks bebas

    // --- Ekstraksi kode perbaikan ---
    // Coba tag XML <fix>,
    // lalu "2. FIXED CODE:", lalu blok ```php ... ```

    // --- Ekstraksi skor kepercayaan (confidence) ---
    // Coba tag <confidence>N</confidence>
    // Coba kata kunci "CONFIDENCE: N" (integer 0-100)
```

```
    // Coba desimal 0.x di dekat kata "confidence"
    // Coba persentase di dekat kata "confidence"
    // Coba bahasa natural: "not sure"->0.30, "confident"->0.75
    // Jika tidak ditemukan -> kembalikan 0.0

    RETURN AIRecommendation(original_code, suggested_fix, explanation,
                            confidence, iso_controls, ...)
END FUNCTION


FUNCTION run_llm_comparison(findings, models, runs_per_finding,
                            prompt_mode, reports_dir):
    // Saring temuan yang memenuhi ambang prioritas
    eligible <- [f for f in findings WHERE
                 f.priority <= PRIORITY_THRESHOLD]

    report <- {models_compared, runs_per_finding, prompt_mode,
               findings: []}

    FOR each model in models DO
        IF Ollama NOT reachable THEN
            mark model as unavailable
            CONTINUE
        END IF

        FOR each finding in eligible DO
            IF prompt_mode == "standard" THEN
             // Kondisi A: prompt identik untuk semua model
                messages <- build_chat_messages(finding)
            ELSE
               // Kondisi B: prompt dioptimalkan per model
               messages <- build_optimized_chat_messages
                           (finding, model)
            END IF

            confidences <- []
            times       <- []
            FOR run_index = 1 to runs_per_finding DO
                (raw_text, elapsed) <- call_ollama_chat_timed
                                       (messages, model)
                confidence <- extract_confidence(raw_text)
                format_ok  <- check_format_valid(raw_text)
                confidences.append(confidence)
                times.append(elapsed)
            END FOR

            // Statistik agregat per temuan per model
```

```
            finding_result <- {
            confidence_mean       : mean(confidences),
            confidence_variance   : variance(confidences),
            confidence_stdev      : stdev(confidences),
            format_compliance_rate: count(format_ok)/runs_per_finding,
            mean_inference_time_sec: mean(times)
            }
            report.append(finding_result)
        END FOR
    END FOR

    save_json(report, reports_dir / "llm_comparison_{timestamp}.json")
    print_comparison_table(report)
    RETURN report
END FUNCTION
```

### B.6 Modul `iso_mapper.py` — Pemetaan ISO/IEC 27001:2022

Modul ini adalah "hakim akhir" pipeline yang mengumpulkan seluruh temuan dari Semgrep, PHPStan, dan rekomendasi AI, kemudian memetakannya ke enam kontrol keamanan ISO/IEC 27001:2022 Annex A yang relevan. Setiap kontrol diberi status: *COMPLIANT* (tidak ada temuan), *PARTIAL* (hanya peringatan), atau *NON_COMPLIANT* (terdapat kesalahan kritis). Status terburuk di antara semua kontrol menjadi status keseluruhan sistem. Rekomendasi AI dicatat sebagai bukti (*evidence*) namun **tidak** memengaruhi penentuan status kepatuhan.

```
PROGRAM: ISOMapper

DATA STRUCTURE ISOControl:
    control_id    // e.g. "A.8.28"
    title, description
    status        // COMPLIANT | PARTIAL | NON_COMPLIANT
    findings_count
    evidence      // daftar string bukti temuan (maks. 30 entri)

DATA STRUCTURE ISOReport:
    controls          : dict of ISOControl (kunci: control_id)
    overall_status    : status terburuk dari semua kontrol
    total_findings    : total temuan (Semgrep + PHPStan)
    critical_findings : jumlah temuan ERROR-severity
    generated_at      : waktu laporan dibuat

CONSTANT CONTROL_REGISTRY:
    A.5.17  Authentication Information (pengelolaan kredensial)
    A.8.24  Use of Cryptography (penggunaan enkripsi)
    A.8.25  Secure Development Lifecycle (siklus pengembangan aman)
```

```
    A.8.26  Application Security Requirements
            (persyaratan keamanan aplikasi)
    A.8.28  Secure Coding (praktik penulisan kode aman)
    A.8.29  Security Testing in Development  (pengujian keamanan)

STATUS RULES:
    NON_COMPLIANT : terdapat >= 1 temuan ERROR pada kontrol ini
    PARTIAL       : hanya temuan WARNING, tidak ada ERROR
    COMPLIANT     : tidak ada temuan sama sekali pada kontrol ini


FUNCTION run_iso_mapping(scan_result, analysis_result, ai_recs,
                         output_path):
    // Step 1: Inisialisasi wadah temuan untuk setiap kontrol
    findings_map <- {}
    FOR each control in CONTROL_REGISTRY DO
        findings_map[control] <- []
    END FOR

    // Step 2: Masukkan temuan Semgrep ke peta kontrol
    IF scan_result != NULL THEN
        FOR each finding in scan_result.findings DO
            // INFO dinaikkan ke WARNING agar tidak ada status
            // COMPLIANT palsu
            severity <- normalize(finding.severity)
            evidence <- "[Semgrep][vuln_type] file:baris -- pesan"
            FOR each ctrl in finding.iso_controls DO
                findings_map[ctrl].append((severity, evidence))
            END FOR
        END FOR
    END IF

    // Step 3: Masukkan kesalahan PHPStan ke peta kontrol
    IF analysis_result != NULL THEN
        FOR each error in analysis_result.errors DO
            IF error.file_path == "<phpstan>" THEN CONTINUE END IF
            evidence <- "[PHPStan] file:baris -- pesan"
            FOR each ctrl in error.iso_controls DO
                findings_map[ctrl].append((severity, evidence))
            END FOR
        END FOR
    END IF

    // Step 4: Kumpulkan bukti rekomendasi AI (hanya informatif)
    ai_evidence_map <- {}
    FOR each rec in ai_recs DO
        evidence <- "[AI/model][vuln_type] file:baris -- fix
```

```
                    (confidence)"
        FOR each ctrl in rec.iso_controls DO
            ai_evidence_map[ctrl].append(evidence)
        END FOR
    END FOR

    // Step 5: Bangun objek ISOControl untuk setiap kontrol
    controls <- {}
    FOR each (ctrl_id, title, description) in CONTROL_REGISTRY DO
        entries <- findings_map[ctrl_id]
        ai_evs  <- ai_evidence_map[ctrl_id]
        status  <- determine_status(entries)
        // Gabungkan bukti ERROR + WARNING + AI, batasi 30 entri
        evidence <- (ERROR entries + WARNING entries + ai_evs)[:30]
        controls[ctrl_id] <- ISOControl(ctrl_id, title, description,
                                status, count(entries), evidence)
    END FOR

    // Step 6: Status keseluruhan = status terburuk di
    // antara semua kontrol
    overall_status <- max(ctrl.status for ctrl in controls)

    report <- ISOReport(controls, overall_status,
                        total_findings, critical_findings, now())

    print_control_table(report)
    save_json(report, output_path)
    RETURN report
END FUNCTION


FUNCTION determine_status(entries):
    severities <- {sev for (sev, _) in entries}
    IF "ERROR"   in severities THEN RETURN NON_COMPLIANT END IF
    IF "WARNING" in severities THEN RETURN PARTIAL       END IF
    RETURN COMPLIANT
END FUNCTION
```

### B.7 Modul `composer_analyzer.py` — Analisis Dependensi

Composer adalah manajer dependensi standar PHP. File `composer.json` mendaftarkan semua *library* pihak ketiga yang digunakan proyek. Modul ini membaca file tersebut dan mengevaluasi kompatibilitas setiap *library* terhadap PHP 8.x menggunakan basis data paket yang sudah diketahui. Hasilnya berupa rekomendasi: apakah *library* aman digunakan (*safe*), perlu diperbarui versinya (*upgrade*), perlu diganti

dengan alternatif yang didukung (*replace*), atau perlu diperiksa secara manual karena datanya belum tersedia (*manual_review*).

```
PROGRAM: ComposerAnalyzer

DATA STRUCTURE DependencyRecommendation:
    package                // nama library, e.g. "guzzlehttp/guzzle"
    current_version_constraint  // versi yang digunakan, e.g. "^6.0"
    status    // "safe" | "upgrade" | "replace" | "manual_review"
    suggestion  // teks rekomendasi aksi yang harus dilakukan
    iso_control // "A.8.25" - Secure Development Lifecycle

DATA STRUCTURE ComposerAnalysisResult:
    composer_found    : bool
    php_constraint    // e.g. ">=7.4" dari composer.json
    recommendations   : list of DependencyRecommendation
    issues_count      // jumlah library yang berstatus bukan "safe"

CONSTANT KNOWN_PACKAGES:
    // Basis data kompatibilitas library terhadap PHP 8.x
    phpoffice/phpexcel      -> replace  ke phpoffice/phpspreadsheet
    dompdf/dompdf           -> upgrade  ke versi >=2.0
    swiftmailer/swiftmailer -> replace  ke symfony/mailer
    codeigniter/framework   -> replace  ke codeigniter4/framework
    guzzlehttp/guzzle       -> upgrade  ke versi >=7.0
    monolog/monolog         -> safe     (>=2.0 mendukung PHP 8.x)
    firebase/php-jwt        -> safe     (>=6.0 mendukung PHP 8.x)
    ... (dan lainnya)


FUNCTION run_composer_analysis(input_dir, target_php):
    composer_path <- input_dir / "composer.json"

    // Jika proyek tidak menggunakan Composer, lewati tahapan ini
    IF composer_path does NOT exist THEN
        RETURN ComposerAnalysisResult(composer_found=FALSE)
    END IF

    data <- parse_json(composer_path)
    php_constraint <- data["require"]["php"]  // e.g. ">=7.4 <8.0"

    // Kumpulkan semua dependensi (runtime + development),
    // kecuali entri meta "php" dan "ext-*"
    deps <- merge(data["require"], data["require-dev"])
    deps <- filter(deps, exclude php and ext-*)

    recommendations <- []
    FOR each (package, version) in deps DO
```

```
        rec <- evaluate_package(package, version, target_php)
        recommendations.append(rec)
    END FOR

    print_dependency_table(recommendations)
    RETURN ComposerAnalysisResult(composer_found=TRUE,
                                  php_constraint, recommendations)
END FUNCTION


FUNCTION evaluate_package(package, version, target_php):
    IF package in KNOWN_PACKAGES THEN
        (action, suggestion, php8_safe) <- KNOWN_PACKAGES[package]
        status <- "safe" IF php8_safe ELSE action
    ELSE
    // Library tidak ada dalam basis data: minta pemeriksaan manual
        status     <- "manual_review"
        suggestion <- "Tidak ada data PHP 8.x untuk paket ini. "
                    + "Periksa secara manual."
    END IF

RETURN DependencyRecommendation(package, version, status, suggestion)
END FUNCTION
```

### B.8 Ringkasan Alur Pipeline

Berikut adalah gambaran keseluruhan alur eksekusi pipeline dari awal hingga akhir. Setiap langkah menghasilkan sebuah objek hasil (*result object*) yang digunakan oleh langkah berikutnya. Kode keluar (*exit code*) `0` berarti sistem dinyatakan patuh ISO, sedangkan `1` berarti masih terdapat temuan yang perlu ditangani.

```
INPUT: File PHP sumber (PHP 5.x / 7.x) di direktori input/

Tahap 1  run_scan(input/)
         -> ScanResult  [Semgrep, kondisi SEBELUM konversi]

Tahap 2  run_conversion(input/, output/)
         -> ConversionResult  [Rector: PHP lama ke PHP 8.x]

Tahap 2b run_composer_analysis(input/)
         -> ComposerAnalysisResult  [kompatibilitas library]

Tahap 3  run_scan(output/)
         -> ScanResult  [Semgrep, kondisi SESUDAH konversi]

Tahap 4  run_analysis(output/)
         -> AnalysisResult  [PHPStan: analisis tipe & referensi]
```

```
17
18 Tahap 5  run_ai_analysis(pre_scan.findings)
19         -> list[AIRecommendation]  [Ollama LLM: rekomendasi perbaikan]
20    ATAU  run_llm_comparison(pre_scan.findings, 5_models, 3_runs)
21         -> LLMComparisonReport  [perbandingan antar model]
22
23 Tahap 6  run_iso_mapping(pre_scan, analysis, ai_recs)
24         -> ISOReport  [pemetaan ke 6 kontrol ISO/IEC 27001:2022]
25
26 OUTPUT:
27   output/   -- file PHP 8.x hasil konversi
28   reports/
29     iso_report_{timestamp}.json          // laporan kepatuhan ISO
30     pipeline_result_{timestamp}.json     // ringkasan seluruh pipeline
31     llm_comparison_{timestamp}.json      // (jika mode --compare-all)
32
33 KODE KELUAR:
34   0 -> ISO overall_status == COMPLIANT   (semua kontrol terpenuhi)
35   1 -> NON_COMPLIANT / PARTIAL / ada tahapan yang gagal
```

Berikut teks lengkap *prompt* Kondisi B untuk masing-masing model, diambil dari metode `_build_optimized_chat_messages()` di `ai_engine.py`. Variabel `{vuln_type}`, `{controls_str}`, `{context_block}`, dan `{code}` diisi secara dinamis saat *runtime* berdasarkan temuan Semgrep yang sedang diproses.

### C.1 Qwen2.5-Coder 7b

Strategi: format diperlunak — `CONFIDENCE` tidak diwajibkan, fokus pada kualitas kode.

***System message*:**

```
You are a PHP security expert specializing in PHP 7.x to PHP 8.x
migration. Your primary goal is to produce clean, secure, and
syntactically correct PHP 8.x code. Apply ISO/IEC 27001:2022
secure coding principles (A.8.28, A.8.29).
```

***User message*:**

````
Analyze this PHP code for a [{vuln_type}] vulnerability and
provide a secure fix.
ISO/IEC 27001:2022 Controls: {controls_str}
{context_block}
VULNERABLE CODE:
```php
{code}
```
Please:
1. Briefly explain the security issue.
2. Provide the corrected PHP 8.x code in a ```php code block.

If you wish, rate your confidence (0-100) at the end as:
CONFIDENCE: [number]
````

### C.2 DeepSeek Coder 6.7b

Strategi: instruksi pendek dan langsung, satu format *requirement*.

***System message*:**

```
You are a PHP security expert. Fix PHP vulnerabilities with
secure PHP 8.x code. ISO/IEC 27001:2022 A.8.28 applies.
```

***User message*:**

```
Fix this [{vuln_type}] in PHP 8.x.
ISO controls: {controls_str}
{context_block}
CODE:
{code}

Write:
- Why it is vulnerable.
- Fixed PHP 8.x code in a ```php block.
- Last line: CONFIDENCE: [0-100]
```

### C.3 Mistral 7b

Strategi: satu contoh *few-shot* konkret untuk mengajarkan struktur keluaran.

***System message*:**

```
You are a PHP security expert and migration specialist. Analyze
PHP vulnerabilities and provide secure PHP 8.x replacements.
Follow ISO/IEC 27001:2022 controls A.8.28 (Secure Coding) and
A.8.29 (Security Testing in Development).
```

***User message*:**

```
TASK: Fix a [{vuln_type}] vulnerability.
ISO controls: {controls_str}
{context_block}
=== EXAMPLE ===
VULNERABLE CODE:
$id = $_GET['id'];
$result = mysql_query('SELECT * FROM users WHERE id=' . $id);

EXPECTED OUTPUT:
The code uses deprecated mysql_query() with direct user-input
concatenation, enabling SQL injection (ISO/IEC 27001:2022 A.8.28).

$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([$_GET['id']]);
$result = $stmt->fetchAll();

CONFIDENCE: 90
=== END EXAMPLE ===
```

```
Now fix this code using the same format:
VULNERABLE CODE:
{code}

Provide: explanation, then ```php fix block,
then CONFIDENCE: [0-100].
```

### C.4 Llama 3.1 8b

Strategi: *role-play* persona kuat dengan empat langkah prosedural eksplisit.

***System message*:**

```
You are an expert PHP security auditor with 10 years of
experience in web application security and PHP migration.
Your role is to identify vulnerabilities and write secure
PHP 8.x code. You strictly follow ISO/IEC 27001:2022 Annex A
controls for secure software development. You always respond
in exactly the format requested, without deviation.
```

***User message*:**

```
Act as a PHP security expert and follow these steps exactly:

Step 1: Read the vulnerable PHP code below.
Step 2: Identify why it is a [{vuln_type}] security risk
        (ISO/IEC 27001:2022: {controls_str}).
Step 3: Write a secure PHP 8.x replacement that fully fixes
        the vulnerability.
Step 4: End your response with this exact line:
        CONFIDENCE: [number 0-100]
{context_block}
VULNERABLE CODE:
{code}

Your response must follow this exact order:
1. Explanation of the vulnerability.
2. Secure PHP 8.x code fix.
3. CONFIDENCE: [number]
```

### C.5 CodeLlama 7b

Strategi: format identik dengan Kondisi A dengan tambahan instruksi validitas sintaks `php -l` secara eksplisit.

***System message*:**

```
You are a PHP security expert and migration specialist. You
analyze PHP code for security vulnerabilities and provide secure
PHP 8.x replacements. Follow ISO/IEC 27001:2022 secure coding
standards (A.8.28, A.8.29). CRITICAL: Every PHP code block you
generate must be syntactically valid and pass php -l
(lint check). Mentally verify your code compiles without
errors before writing it. Never output PHP with syntax errors.
```

***User message*:**

```
TASK: Analyze the following PHP code for a [{vuln_type}]
vulnerability and provide a secure PHP 8.x replacement.
ISO/IEC 27001:2022 Controls: {controls_str}
{context_block}
VULNERABLE CODE:
{code}

Respond in English:
1. Explain the vulnerability and its security risk.
2. Provide the corrected PHP 8.x code in a ```php code block.
IMPORTANT: Before writing the code, verify it is syntactically
correct (as if running php -l). No syntax errors are allowed.

After your explanation and code fix, write exactly this on the
last line:
CONFIDENCE: [number between 0 and 100]
Example last line: CONFIDENCE: 85
```

Lampiran ini menyajikan instrumen evaluasi yang digunakan dalam eksperimen *blind rating* untuk mengukur efektivitas komponen *AI Engine* sebagaimana dirancang pada Subbab 3.8 dan dilaporkan hasilnya pada Subbab 4.3.5. Dua temuan representatif disertakan secara lengkap: **FT-01** (SQL Injection, satu-satunya temuan non-SSRF) dan **FT-02** (SSRF, representatif untuk FT-02 hingga FT-11). Temuan FT-03 hingga FT-11 mengikuti format yang identik dengan FT-02 dengan potongan kode dan nomor baris yang berbeda sesuai posisi masing-masing temuan di `Output.php`.

Penugasan kondisi ke Paket A atau Paket B diacak menggunakan `random.seed(42)` sehingga rater tidak mengetahui kondisi mana yang sedang dinilai. Pada FT-01, Paket A berisi keluaran kondisi T1 (*dengan AI Engine*) dan Paket B berisi kondisi B0 (*tanpa AI Engine*). Pada FT-02 hingga FT-11, penugasan diacak sehingga posisi T1 dan B0 tidak selalu konsisten antar temuan.

### D.1 Instruksi Penilai dan Dimensi Penilaian

**Tujuan:** Menilai kualitas analisis temuan keamanan dari dua pendekatan berbeda (Paket A dan Paket B) untuk setiap temuan. Penilai tidak perlu mengetahui pendekatan mana yang menghasilkan paket tertentu.

**Petunjuk umum:**

- Penilai diminta menilai Paket A dan Paket B untuk setiap temuan secara *independen*.
- Skor 1 menunjukkan kualitas sangat rendah dan skor 7 menunjukkan kualitas sangat tinggi.
- Penilai tidak perlu melakukan verifikasi teknis terhadap kode sumber asli. Penilaian difokuskan pada kualitas analisis yang disajikan.
- Beberapa paket memiliki bagian **Catatan Tambahan** yang berisi analisis lebih lanjut. Pertimbangkan kualitas keseluruhan paket saat memberikan penilaian.
- Nilailah setiap paket secara independen, bukan relatif terhadap paket lainnya.

**Terdapat 11 temuan keamanan (FT-01 s.d. FT-11)** dari `Output.php` pada aplikasi web PHP berbasis CodeIgniter 3. Setiap temuan memiliki dua paket (A dan B) yang perlu dinilai secara independen.

**Dimensi Penilaian (Skala Likert 1–7):**

| Dimensi | Deskripsi dan Jangkar Skala |
|---|---|
| Akurasi | Seberapa akurat identifikasi dan deskripsi kerentanan?<br>1 = sepenuhnya salah; 7 = sepenuhnya akurat. |
| Kejelasan | Seberapa jelas dan mudah dipahami penjelasan yang diberikan?<br>1 = sangat membingungkan; 7 = sangat jelas. |
| *Actionability* | Seberapa *actionable* saran perbaikan yang diberikan?<br>1 = tidak ada saran yang berguna; 7 = dapat langsung diterapkan. |
| Ketepatan Pemetaan ISO | Seberapa tepat pemetaan ke kontrol ISO/IEC 27001:2022?<br>1 = sepenuhnya salah; 7 = sepenuhnya tepat. |

**Interpretasi skala:** 1 = Sangat Buruk 2 = Buruk 3 = Cukup Buruk 4 = Netral 5 = Cukup Baik 6 = Baik 7 = Sangat Baik

### D.2 Temuan FT-01 — SQL Injection

**Paket A (Temuan FT-01)**

| | |
|---|---|
| **Rule ID** | `php.lang.security.injection.tainted-sql-string` |
| **File** | `Output.php` |
| **Line** | 768 |
| **Severity** | ERROR |
| **Vuln Type** | SQL Injection |

**Deskripsi Aturan Semgrep:**

```
User data flows into this manually-constructed SQL string. User
data can be safely inserted into SQL strings using prepared
statements or an ORM. Manually-constructed SQL strings is a
possible indicator of SQL injection, which could let an attacker
steal or manipulate data from the database.
CWE: CWE-89 - Improper Neutralization of Special Elements used
     in an SQL Command ('SQL Injection')
OWASP: A03:2021 - Injection
```

**Potongan Kode:**

```
763
764  $cache_path .= md5($CI->config->item('base_url')
765                  .$CI->config->item('index_page')
766                  .ltrim($uri, '/'));
767
768  if ( ! @unlink($cache_path))
769  {
770 >> log_message('error', 'Unable to delete cache file for
       '.$uri);
771     return FALSE;
772  }
773
774  return TRUE;
775 }
```

**Kontrol ISO/IEC 27001:2022:** A.8.28, A.8.26

**Catatan Tambahan:**

*Penjelasan Kerentanan:* The provided PHP code constructs an SQL query manually using user input without proper sanitization or parameterized queries. Specifically, the `$uri` variable is used in a file path construction that involves concatenation with other configuration values and user-provided data. If this user-provided data is not properly sanitized, it could lead to a SQL Injection vulnerability.

*Security Risk:* SQL Injection occurs when an attacker can manipulate SQL queries by injecting malicious SQL code through input fields. This can result in unauthorized access, data theft, or even complete database compromise.

*Corrected PHP 8.x Code:*

```
public function delete_cache($uri = '')
{
    $CI =& get_instance();
    $cache_path = $CI->config->item('cache_path');
    if ($cache_path === '') {
        $cache_path = APPPATH . 'cache/';
    }
    if (!is_dir($cache_path)) {
        log_message('error', 'Unable to find cache path:
        '.$cache_path);
        return FALSE;
    }
```

```
    if (empty($uri)) {
        $uri = $CI->uri->uri_string();
        if (($cache_query_string = $CI->config->
        item('cache_query_string'))
            && !empty($_SERVER['QUERY_STRING'])) {
            if (is_array($cache_query_string)) {
                $uri .= '?' . http_build_query(
                    array_intersect_key($_GET,
                    array_flip($cache_query_string)));
            } else {
                $uri .= '?' . $_SERVER['QUERY_STRING'];
            }
        }
    }
    // Sanitize the URI to prevent directory traversal attacks
    $safe_uri = filter_var($uri, FILTER_SANITIZE_URL);
    // ... (terpotong)
```

**Penilaian Paket A:**

| **Dimensi** | **Deskripsi** | **Skor (1–7)** |
|---|---|---|
| Akurasi | Seberapa akurat identifikasi kerentanan? | |
| Kejelasan | Seberapa jelas penjelasan yang diberikan? | |
| *Actionability* | Seberapa *actionable* rekomendasi perbaikan? | |
| Ketepatan ISO | Seberapa tepat pemetaan kontrol ISO 27001? | |
| Komentar | | |

**Paket B (Temuan FT-01)**

| | |
|---|---|
| **Rule ID** | `php.lang.security.injection.tainted-sql-string` |
| **File** | `Output.php` |
| **Line** | 768 |
| **Severity** | ERROR |
| **Vuln Type** | SQL Injection |

**Deskripsi Aturan Semgrep:**

```
User data flows into this manually-constructed SQL string. User
data can be safely inserted into SQL strings using prepared
statements or an ORM. Manually-constructed SQL strings is a
possible indicator of SQL injection, which could let an attacker
steal or manipulate data from the database.
CWE: CWE-89 - Improper Neutralization of Special Elements used
     in an SQL Command ('SQL Injection')
OWASP: A03:2021 - Injection
```

**Potongan Kode:**

```
763
764  $cache_path .= md5($CI->config->item('base_url')
765                   .$CI->config->item('index_page')
766                   .ltrim($uri, '/'));
767
768  if ( ! @unlink($cache_path))
769  {
770 >> log_message('error', 'Unable to delete cache file for
       '.$uri);
771     return FALSE;
772  }
773
774  return TRUE;
775 }
```

**Kontrol ISO/IEC 27001:2022:** A.8.28, A.8.26

**Saran Remediasi:** *Manual review required.*

**Penilaian Paket B:**

| Dimensi | Deskripsi | Skor (1–7) |
|---|---|---|
| Akurasi | Seberapa akurat identifikasi kerentanan? | |
| Kejelasan | Seberapa jelas penjelasan yang diberikan? | |
| *Actionability* | Seberapa *actionable* rekomendasi perbaikan? | |
| Ketepatan ISO | Seberapa tepat pemetaan kontrol ISO 27001? | |
| Komentar | | |

### D.3 Temuan FT-02 — SSRF (Representatif FT-02 s.d. FT-11)

**Paket A (Temuan FT-02)**

| | |
|---|---|
| **Rule ID** | `php.lang.security.injection.tainted-filename` |
| **File** | `Output.php` |
| **Line** | 583 |
| **Severity** | WARNING |
| **Vuln Type** | SSRF |

**Deskripsi Aturan Semgrep:**

```
File name based on user input risks server-side request forgery.
CWE: CWE-918 - Server-Side Request Forgery (SSRF)
OWASP: A10:2021 - Server-Side Request Forgery (SSRF)
```

**Potongan Kode:**

```
578   }
579   }
580   $cache_path .= md5($uri);
581
582   if ( ! $fp = @fopen($cache_path, 'w+b'))
583   {
584 >> log_message('error', 'Unable to write cache file:
      '.$cache_path);
585       return;
586   }
587
```

```
588   }
```

**Kontrol ISO/IEC 27001:2022:** A.8.28, A.8.29

**Penilaian Paket A:**

| **Dimensi** | **Deskripsi** | **Skor (1–7)** |
|---|---|---|
| Akurasi | Seberapa akurat identifikasi kerentanan? | |
| Kejelasan | Seberapa jelas penjelasan yang diberikan? | |
| *Actionability* | Seberapa *actionable* rekomendasi perbaikan? | |
| Ketepatan ISO | Seberapa tepat pemetaan kontrol ISO 27001? | |
| Komentar | | |

**Paket B (Temuan FT-02)**

| | |
|---|---|
| **Rule ID** | `php.lang.security.injection.tainted-filename` |
| **File** | `Output.php` |
| **Line** | 583 |
| **Severity** | WARNING |
| **Vuln Type** | SSRF |

**Deskripsi Aturan Semgrep:**

```
File name based on user input risks server-side request forgery.
CWE: CWE-918 - Server-Side Request Forgery (SSRF)
OWASP: A10:2021 - Server-Side Request Forgery (SSRF)
```

**Potongan Kode:**

```
578   }
579   }
580   $cache_path .= md5($uri);
581
582   if ( ! $fp = @fopen($cache_path, 'w+b'))
583   {
584 >> log_message('error', 'Unable to write cache file:
```

```
8           '.$cache_path);
9   585      return;
10  586   }
11  587
12  588   }
```

**Kontrol ISO/IEC 27001:2022:** A.8.28, A.8.29

**Catatan Tambahan:**

*Penjelasan Kerentanan:* The vulnerable code is located in the `_write_cache` method within `Output.php`. The vulnerability lies in the way the cache path is constructed using user input from the URI and query string. The `$cache_path` is built by appending an MD5 hash of `$uri`, where `$uri` may contain unsanitized user-controlled data from `$_SERVER['QUERY_STRING']`. This allows an attacker to influence the file path used in `fopen()`, potentially leading to unauthorized access to server-side resources.

**Saran Remediasi:** *Manual review required.*

**Penilaian Paket B:**

| **Dimensi** | **Deskripsi** | **Skor (1–7)** |
|---|---|---|
| Akurasi | Seberapa akurat identifikasi kerentanan? | |
| Kejelasan | Seberapa jelas penjelasan yang diberikan? | |
| *Actionability* | Seberapa *actionable* rekomendasi perbaikan? | |
| Ketepatan ISO | Seberapa tepat pemetaan kontrol ISO 27001? | |
| Komentar | | |

### D.4 Catatan: Temuan FT-03 hingga FT-11

Temuan FT-03 hingga FT-11 seluruhnya merupakan kerentanan SSRF (*Server-Side Request Forgery*) yang bersumber dari pola kode yang sama di `Output.php`, yaitu penggunaan `$uri` atau `$filepath` yang berasal dari data pengguna sebagai argumen operasi *file system*. Struktur instrumen untuk setiap temuan identik dengan FT-02, dengan perbedaan hanya pada nomor baris (*line number*) dan

potongan kode yang ditampilkan. Penugasan kondisi B0/T1 ke Paket A atau Paket B pada FT-03 hingga FT-11 diacak secara independen menggunakan `random.seed(42)`.

Tabel berikut merangkum nomor baris dan penugasan paket untuk setiap temuan FT-03 hingga FT-11:

| **Temuan** | **Baris** | **Paket A** | **Paket B** |
|---|---|---|---|
| FT-03 | 632 | B0 | T1 |
| FT-04 | 637 | T1 | B0 |
| FT-05 | 677 | T1 | B0 |
| FT-06 | 677 | T1 | B0 |
| FT-07 | 684 | T1 | B0 |
| FT-08 | 684 | B0 | T1 |
| FT-09 | 698 | B0 | T1 |
| FT-10 | 704 | B0 | T1 |
| FT-11 | 766 | T1 | B0 |

*Keterangan: B0 = keluaran pipeline tanpa AI Engine (hanya data Semgrep mentah); T1 = keluaran pipeline dengan AI Engine aktif (Qwen2.5-Coder 7b), disertai penjelasan kerentanan dan saran remediasi.*